

软件工程基础

第5章：软件详细设计

杨溢龙

北航软件学院

目　录

Contents

软件工程基础

第5章：软件详细设计

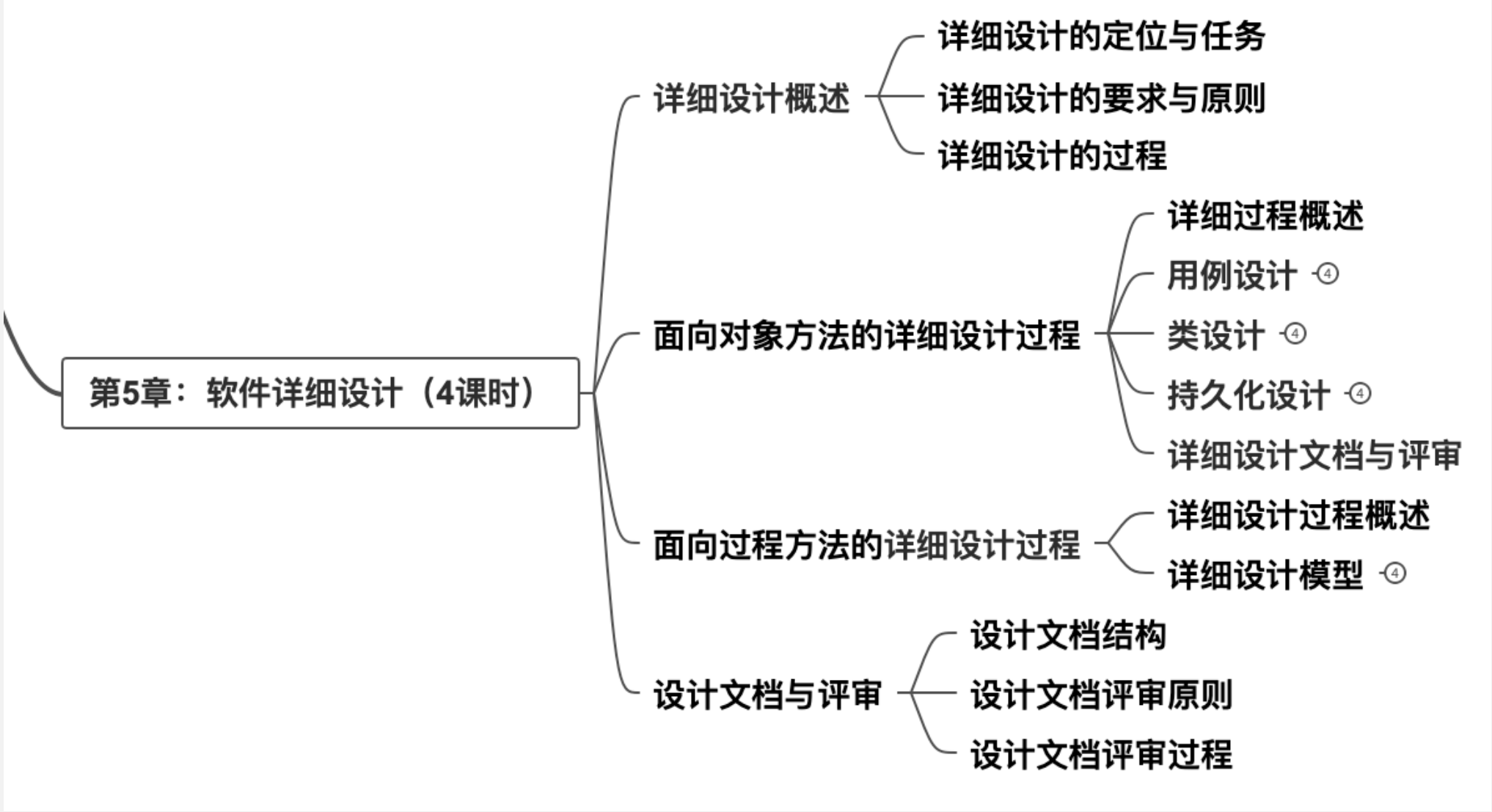
Part 1 详细设计概述

Part 2 面向对象的详细设计过程

Part 3 面向过程的详细设计过程

Part 4 设计文档与评审

课程大纲

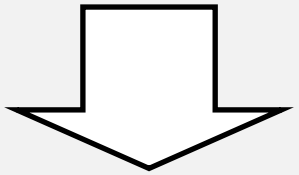

第5章：软件详细设计（4课时）
详细设计概述
详细设计的定位与任务
详细设计的要求与原则
详细设计的过程
面向对象方法的详细设计过程
详细过程概述
用例设计 ④
类设计 ④
持久化设计 ④
详细设计文档与评审
面向过程方法的详细设计过程
详细设计过程概述
详细设计模型 ④
设计文档与评审
设计文档结构
设计文档评审原则
设计文档评审过程

面向对象方法的软件设计过程（回顾）

系统需求分析

系统模型

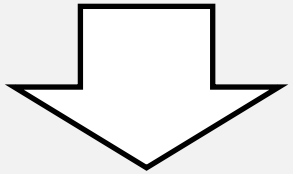
用例图+
系统顺序图（系统状态图/活动图）
+系统操作合约 + 概念类图



概要（架构）设计

组件模型

组件图 + 组件顺序图
(组件状态图/活动图)



详细（类）设计

类模型

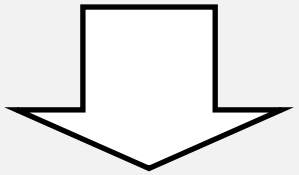
类图 + 类顺序图
(对象状态图/活动图)

面向过程方法的软件设计过程（回顾）

系统需求分析

系统模型

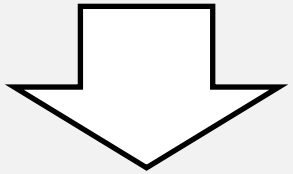
数据流图DFD + 数据字典DD + + 实体关系图E-R



概要（架构）设计

组件模型

层次图H-IPO + 结构图SD

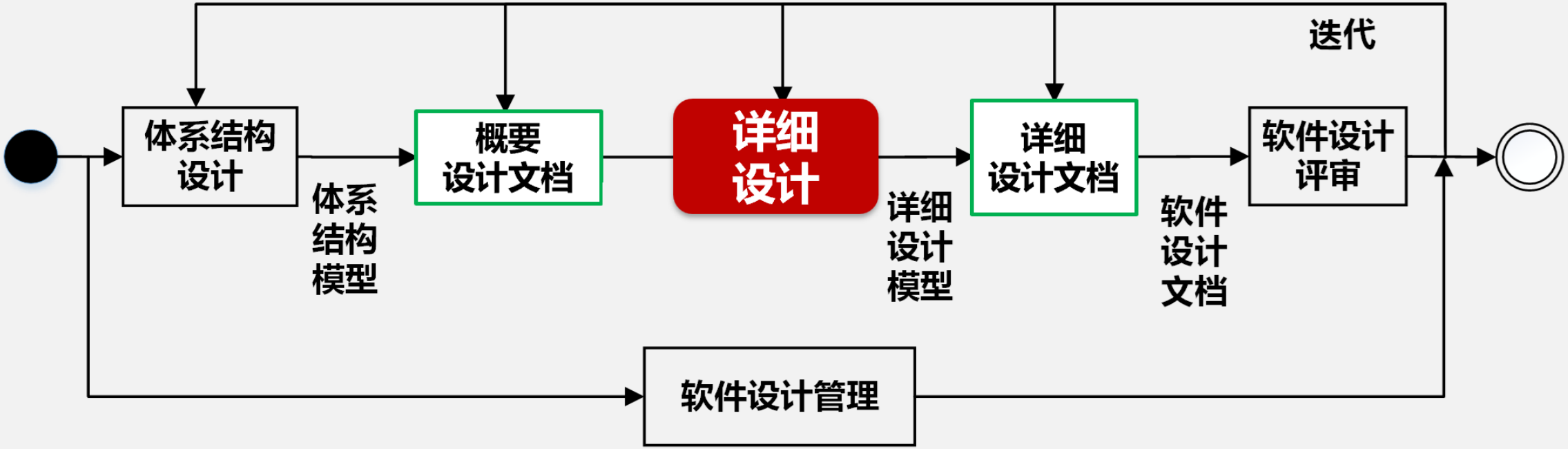


详细（函数）设计

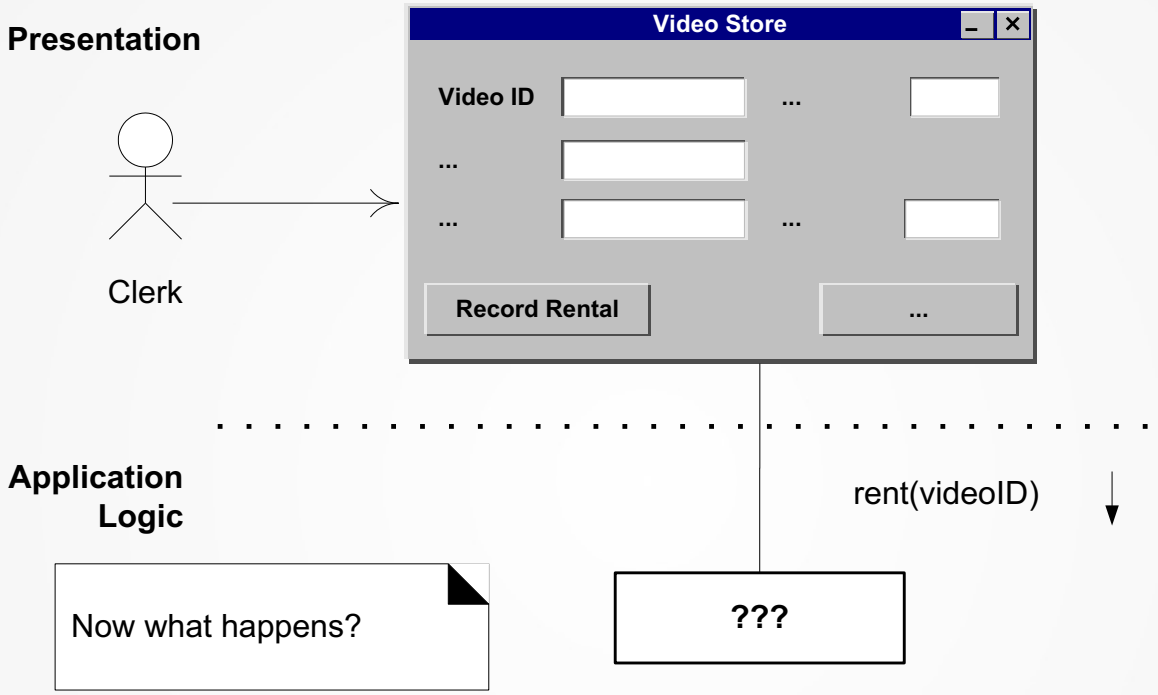
过程模型

流程图 + N-S图 + PAD图

软件设计的过程（回顾）


迭代
体系结构设计
概要设计文档
详细设计
详细设计文档
软件设计评审
体系结构模型
详细设计模型
软件设计文档
软件设计管理

详细设计的动机



- **面向对象**
  - What object should receive this message?
  - What objects should interact to fulfill the request, and how?
- **面向过程**
  - What function should receive this message?
  - What function should be composed to fulfill the request, and how?

北京航空航天大學
BEIHANG UNIVERSITY



Part 1

详细设计概述

Part 1 详细设计概述

- 详细设计的任务与定位
- 详细设计的目标与原则
- 详细设计的主要活动

Part 1 详细设计概述

- 详细设计的任务与定位
- 详细设计的目标与原则
- 详细设计的主要活动

详细设计的定位

落实和细化

辅助和支持

高层软件体系结构设计

软件详细设计

软件实现

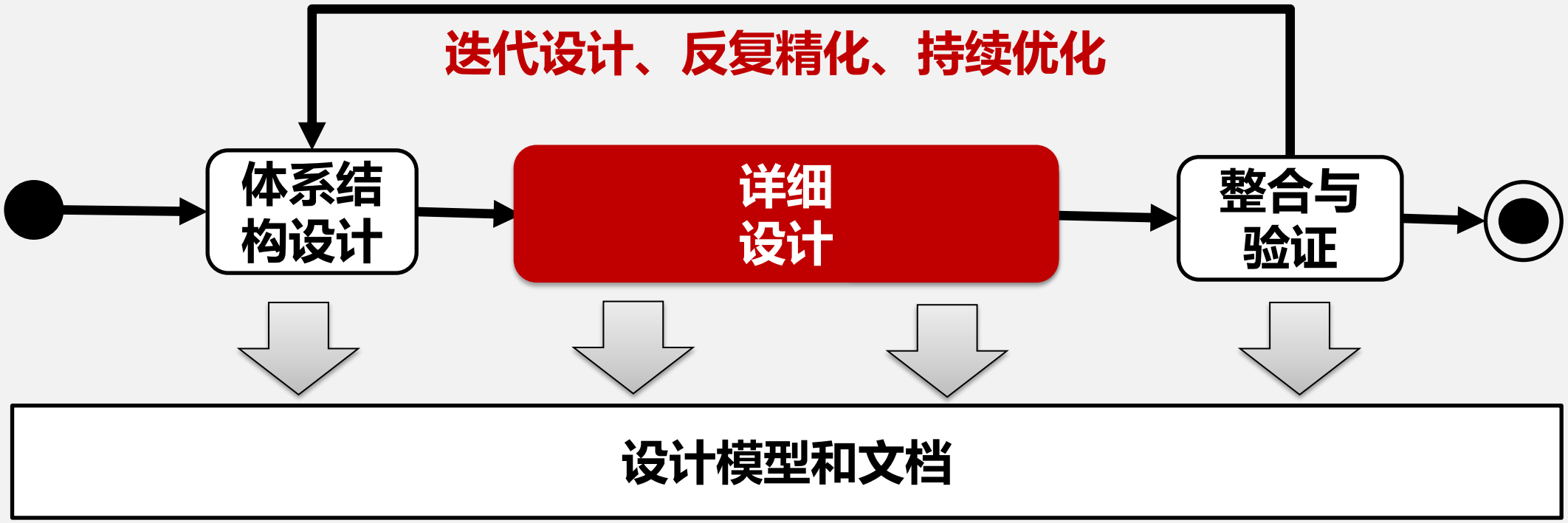




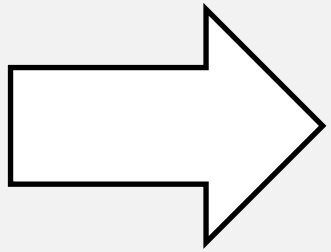


详细设计是高层设计和底层实现间的桥梁

概要设计 vs 详细设计



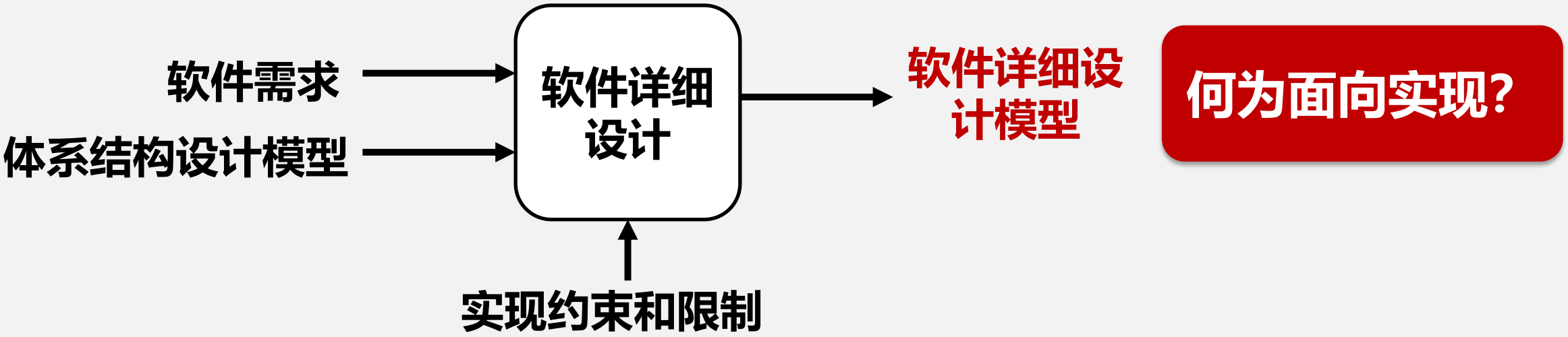
- 结构性
- 全局性
- 关键性
- 粗粒度



- 过程性
- 局部性
- 细节性
- 细粒度

详细设计的任务

- **输入：软件体系结构设计、软件需求**
- **任务：对体系结构设计成果进行细化和精化，获得高质量、面向实现的设计模型**
  - ✓**面向实现**：直接支持编码和程序设计
  - ✓精化和细化的具体对象：**子系统、组件**



何为面向实现?

Part 1 详细设计概述

- 详细设计的任务与定位
- 详细设计的目标与原则
- 详细设计的主要活动

软件详细设计的要求和原则

- **针对软件需求**
  - ✓**从软件需求出发**，确保每一项软件需求都有相应的详细设计元素加以实现
- **深入优化设计**
  - ✓精心设计，以充分优化软件系统的**性能、效能**等，提高软件**系统的可靠性、可重用性和可维护性**等
- **设计足够详细**
  - ✓得到**详实程度**足以支持程序员编码的软件设计模型
- **充分软件重用**
  - ✓从多个**不同的维度和层次**进行充分的软件重用，以提高软件开发的效率和质量，减低开发成本

Part 1 详细设计概述

- 详细设计的任务与定位
- 详细设计的目标与原则
- 详细设计的主要活动

软件详细设计的活动（1/2）

- **用例（面向对象）/顶层过程（面向过程）的详细设计**
  - ✓**面向对象**：给出**用例的具体实现解决方案**，描述用例是如何通过各个**设计元素**（设计类）的交互和协作来完成的
  - ✓**面向过程**：**给出顶层过程的详细分解过程**，描述顶层过程是如何通过各个**子过程**（函数）的复合来完成的

- **子系统/软构件设计**
  - ✓针对粗粒度的子系统和软构件，给出其**细粒度的设计元素**，如子子系统、设计类等，明确这些设计元素之间的协作关系，使得它们能够实现子系统/软构件接口所规定的相关功能和服务

软件详细设计的活动（2/2）

- **类/函数设计**
  - ✓**面向对象**：给出每个设计类的**具体实现细节**，包括类的属性定义、方法的实现算法等，使得**程序员**能够基于类设计**给出这些类的实现代码**
  - ✓**面向过程**：给出每个**子函数（叶子结点）**的**具体实现细节**，包括函数的具体的实现算法等，使得**程序员**能够基于函数设计**给出这些函数的实现代码**

- **持久化设计**
  - ✓对软件所涉及的**持久数据及其操作**进行设计，**明确持久数据**的**存储方式和格式**，细化数据操作的**实现细节**

北京航空航天大學
BEIHANG UNIVERSITY



Part 2

面向对象的详细设计过程

Part 2 详细设计过程

- 面向对象的过程概述
- 用例设计
- 类设计
- 持久化设计
- 设计整合与验证

Part 2 详细设计过程

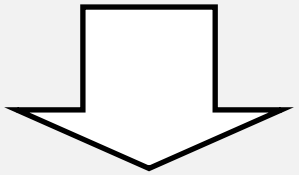
- 面向对象的过程概述
- 用例设计
- 类设计
- 持久化设计
- 设计整合与验证

面向对象方法的软件设计过程（回顾）

系统需求分析

系统模型

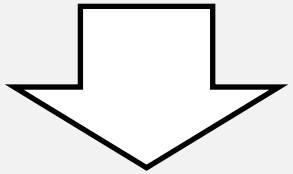
用例图+
系统顺序图（系统状态图/活动图）
+系统操作合约 + 概念类图



概要（架构）设计

组件模型

组件图 + 组件顺序图
(组件状态图/活动图)



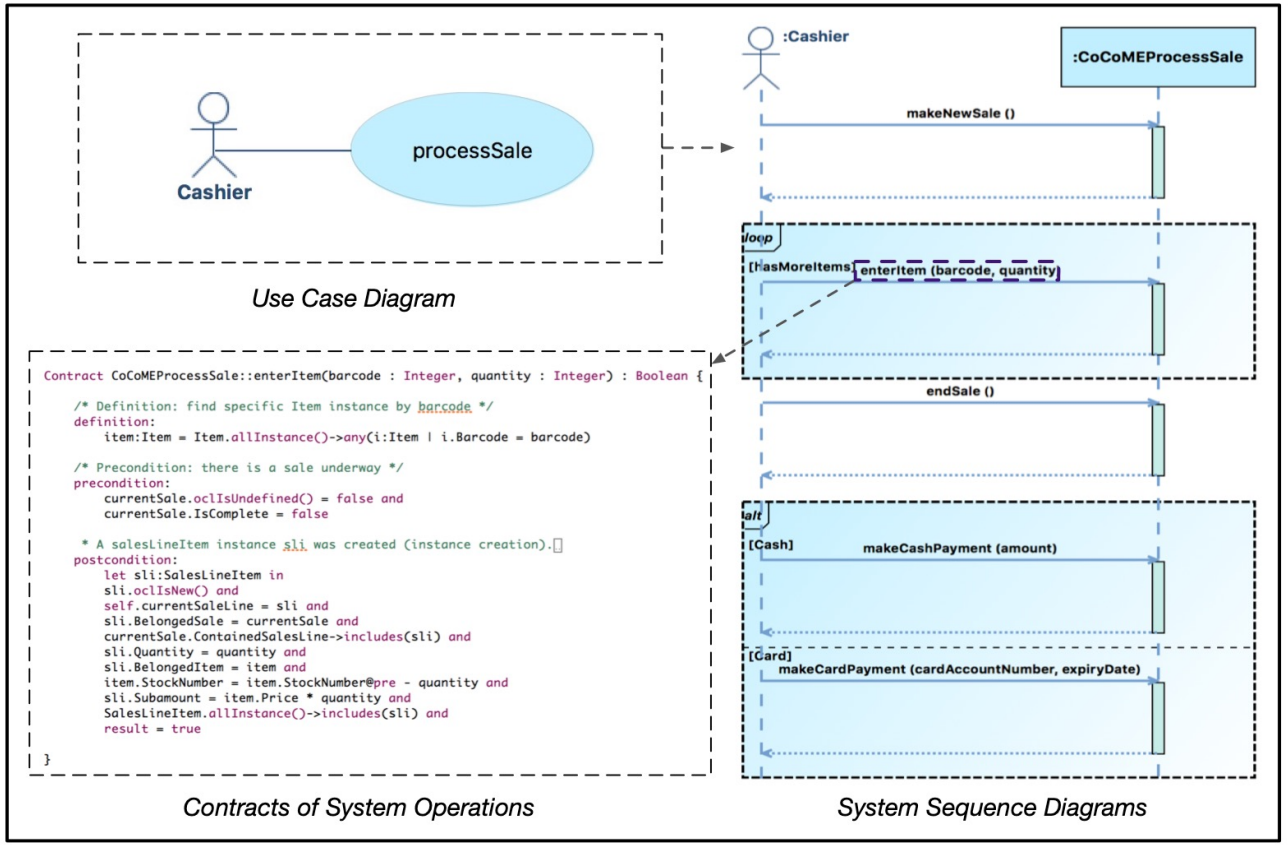
详细（类）设计

类模型

类图 + 类顺序图
(对象状态图/活动图)

需求模型 - 超市管理系统

Requirements Model

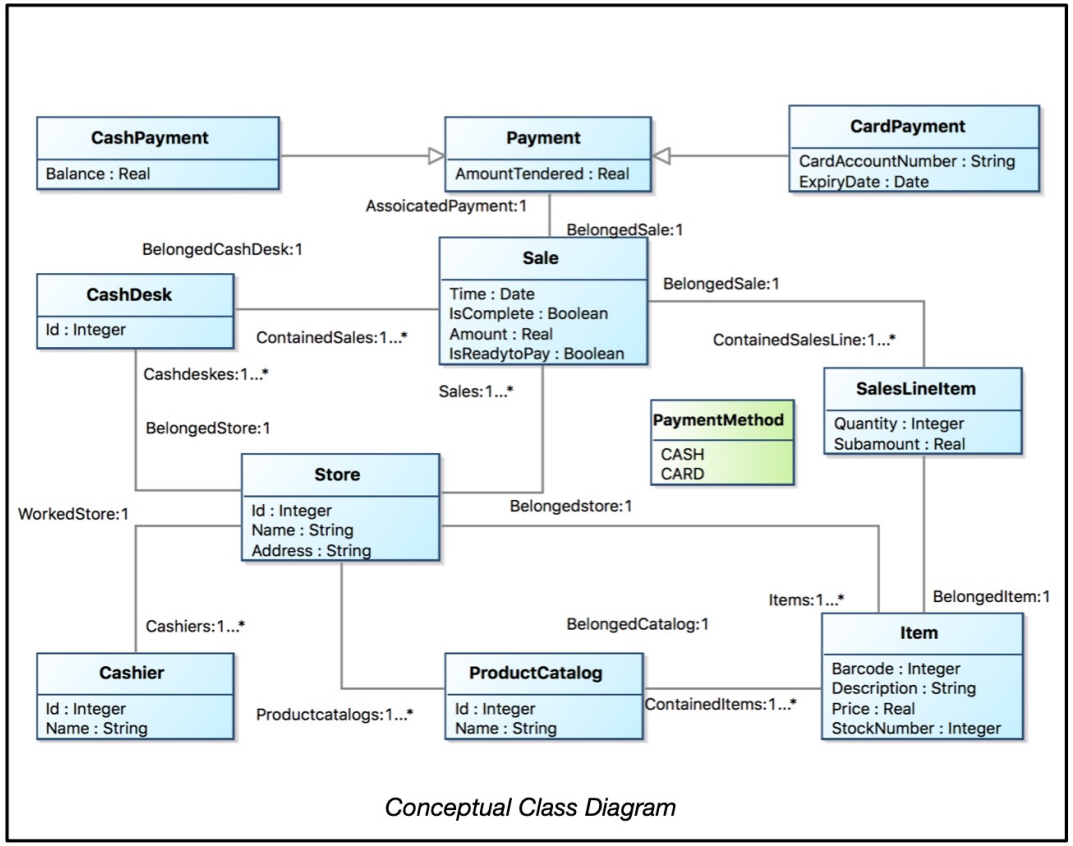


Use Case Diagram

Contracts of System Operations

System Sequence Diagrams

Use Case Model



Conceptual Class Diagram

Conceptual Class Model

概要设计 – 组件图（带接口/概念信息）

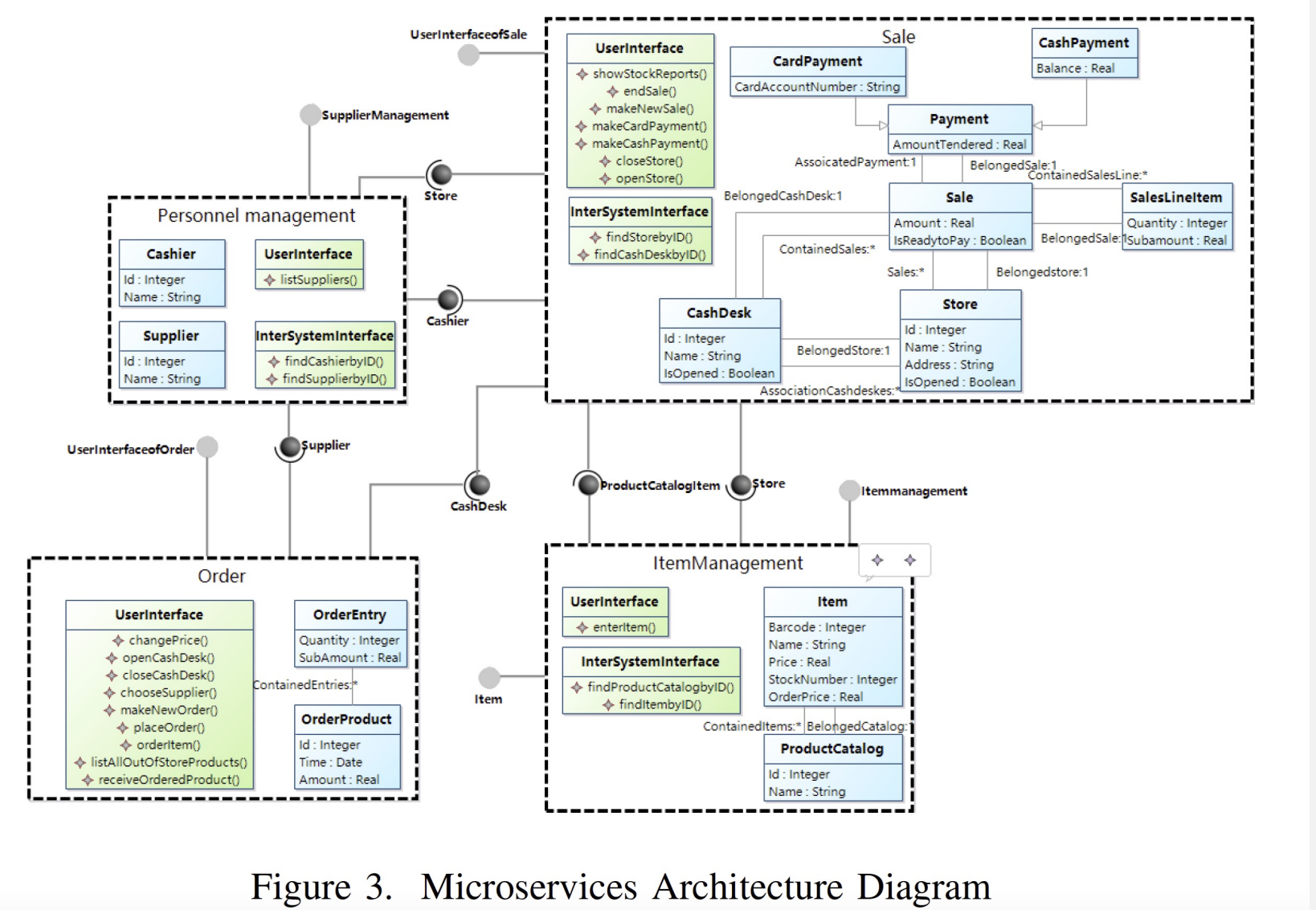
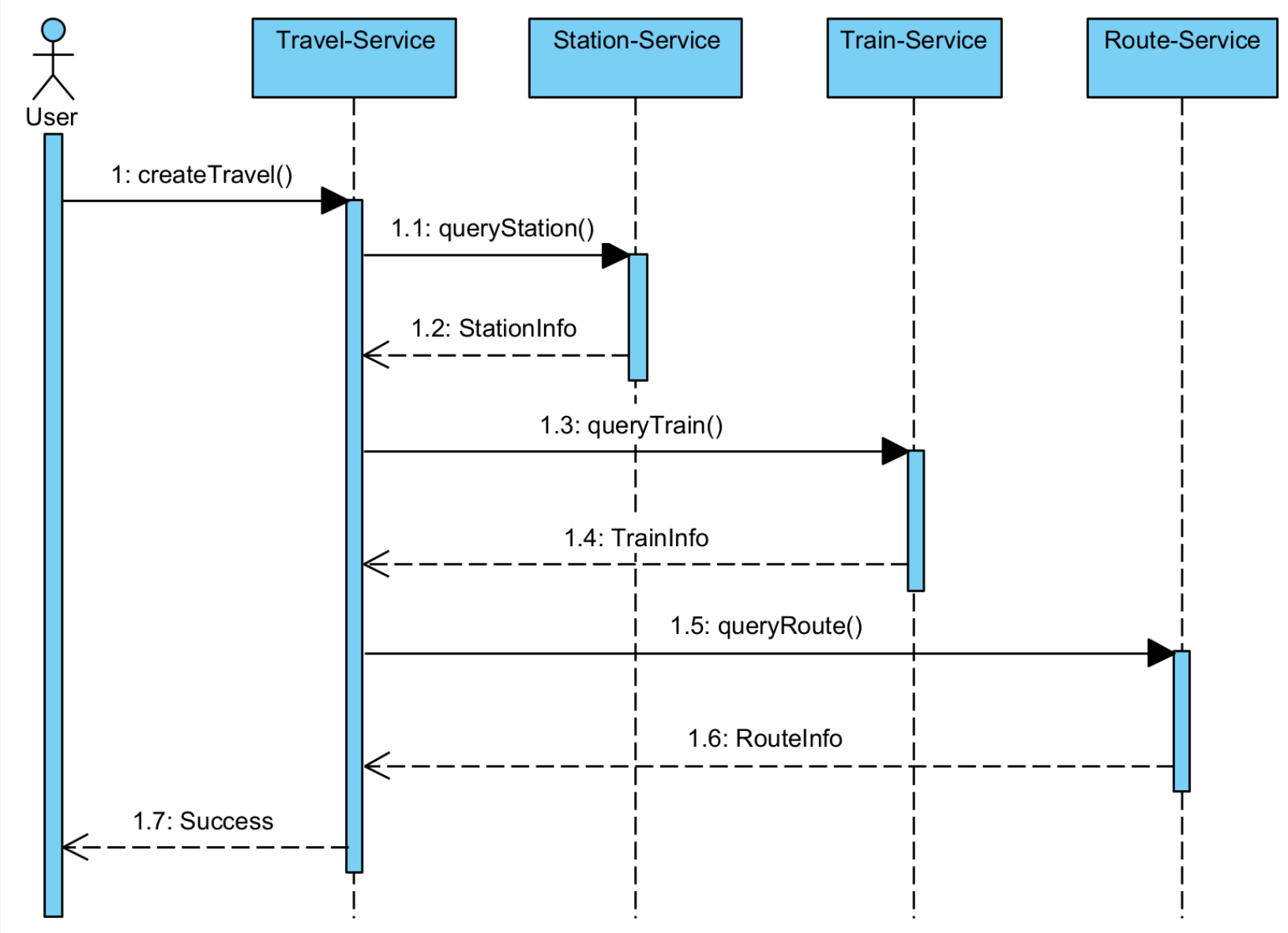


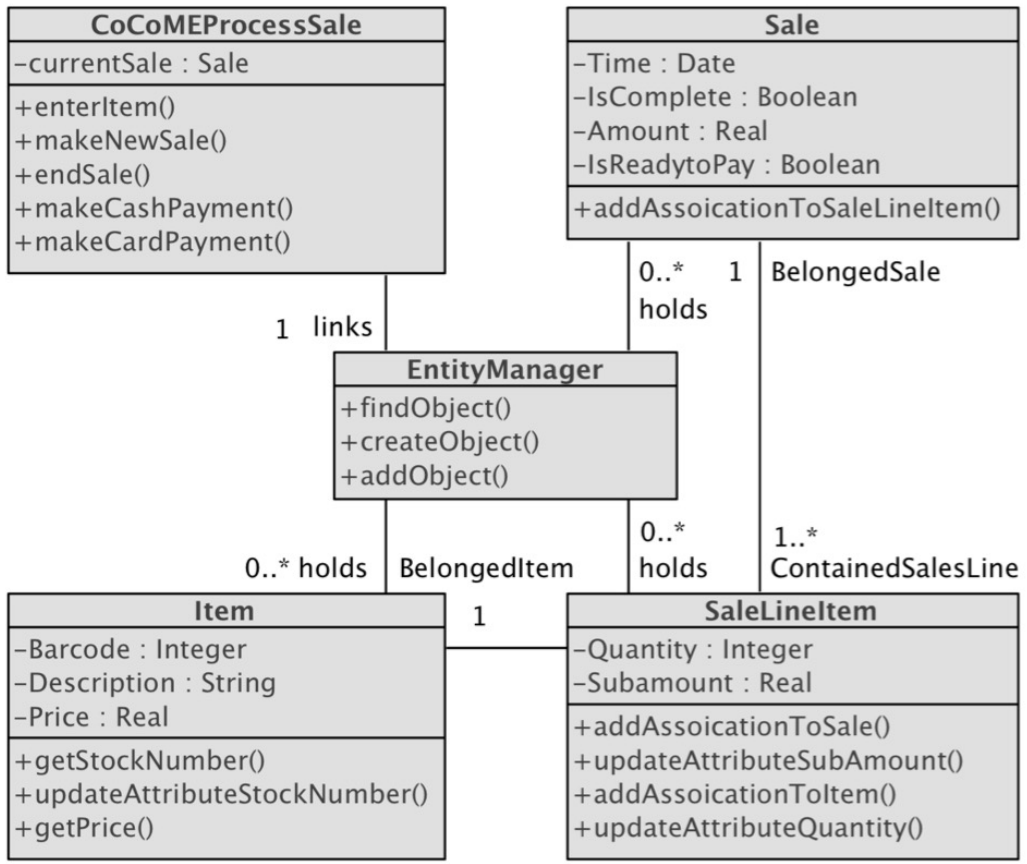
Figure 3. Microservices Architecture Diagram

概要设计 – 组件顺序图

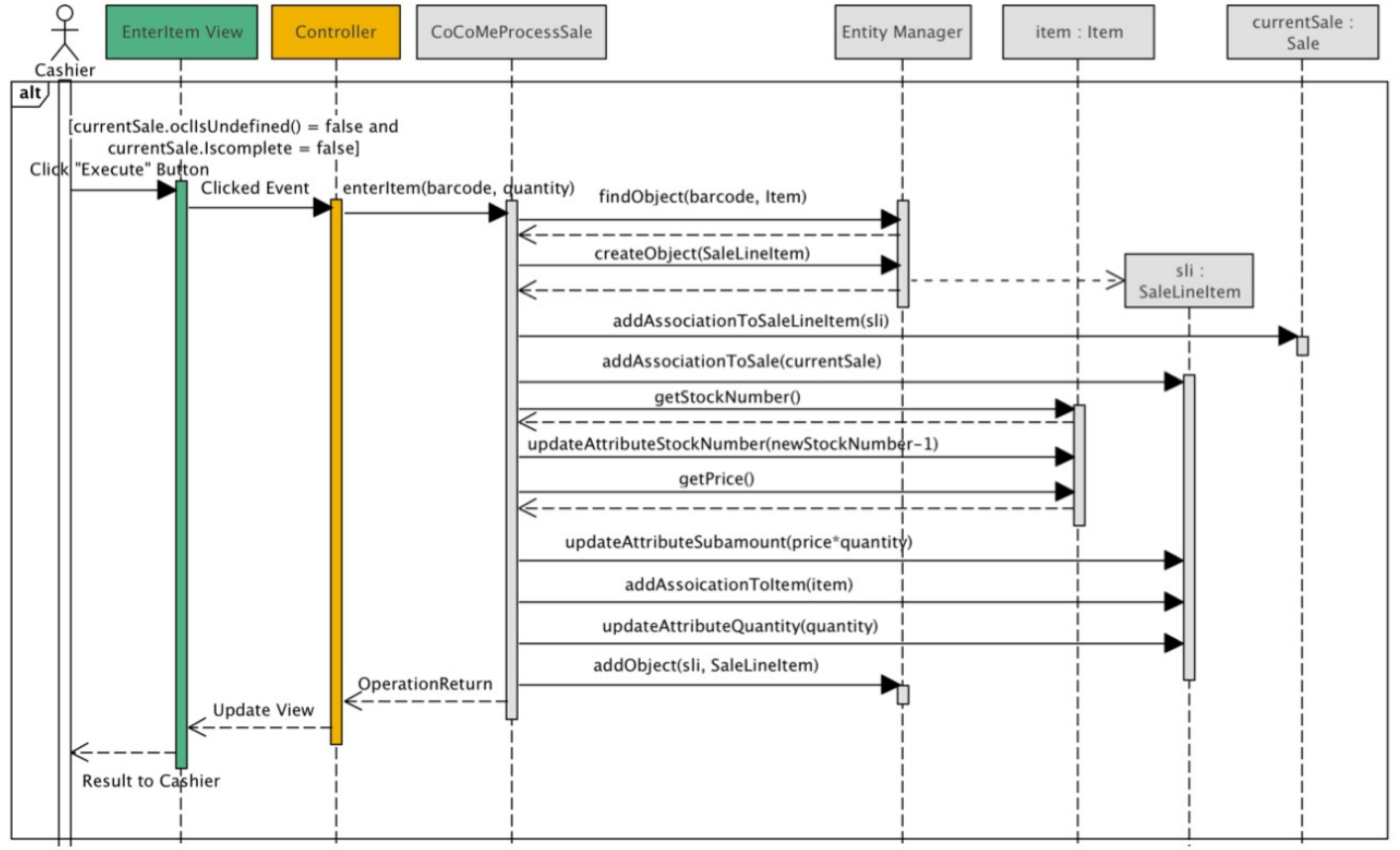

User
Travel-Service
Station-Service
Train-Service
Route-Service
1: createTravel()
1.1: queryStation()
1.2: StationInfo
1.3: queryTrain()
1.4: TrainInfo
1.5: queryRoute()
1.6: RouteInfo
1.7: Success

详细设计 （类图 + 顺序图）

Implemented MVC Design

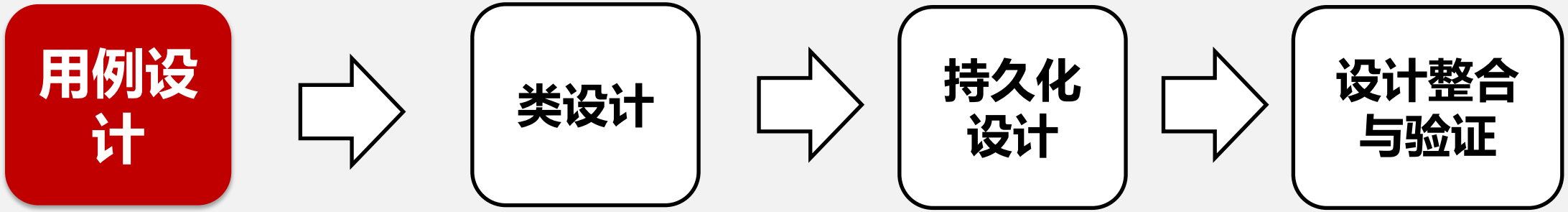


Class Diagram



Sequence Diagram

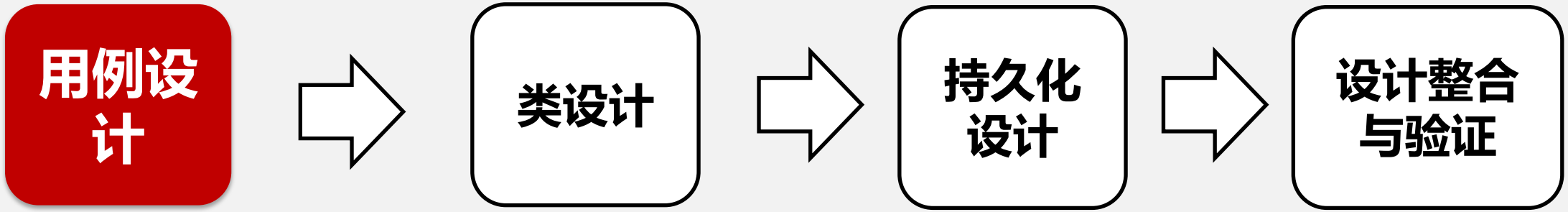
详细设计的过程



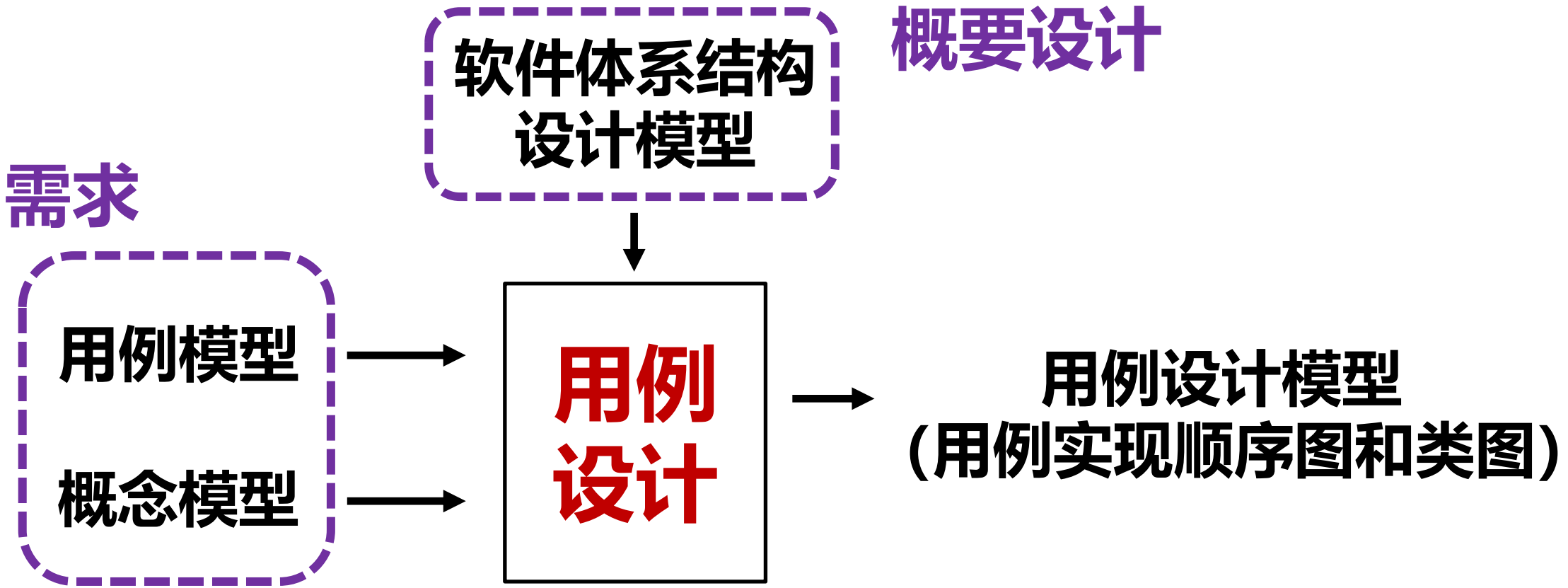
Part 2 详细设计过程

- 面向对象的过程概述
- 用例设计
- 类设计
- 持久化设计
- 设计整合与验证

详细设计的过程（1/4） – 用例设计



用例设计的任务



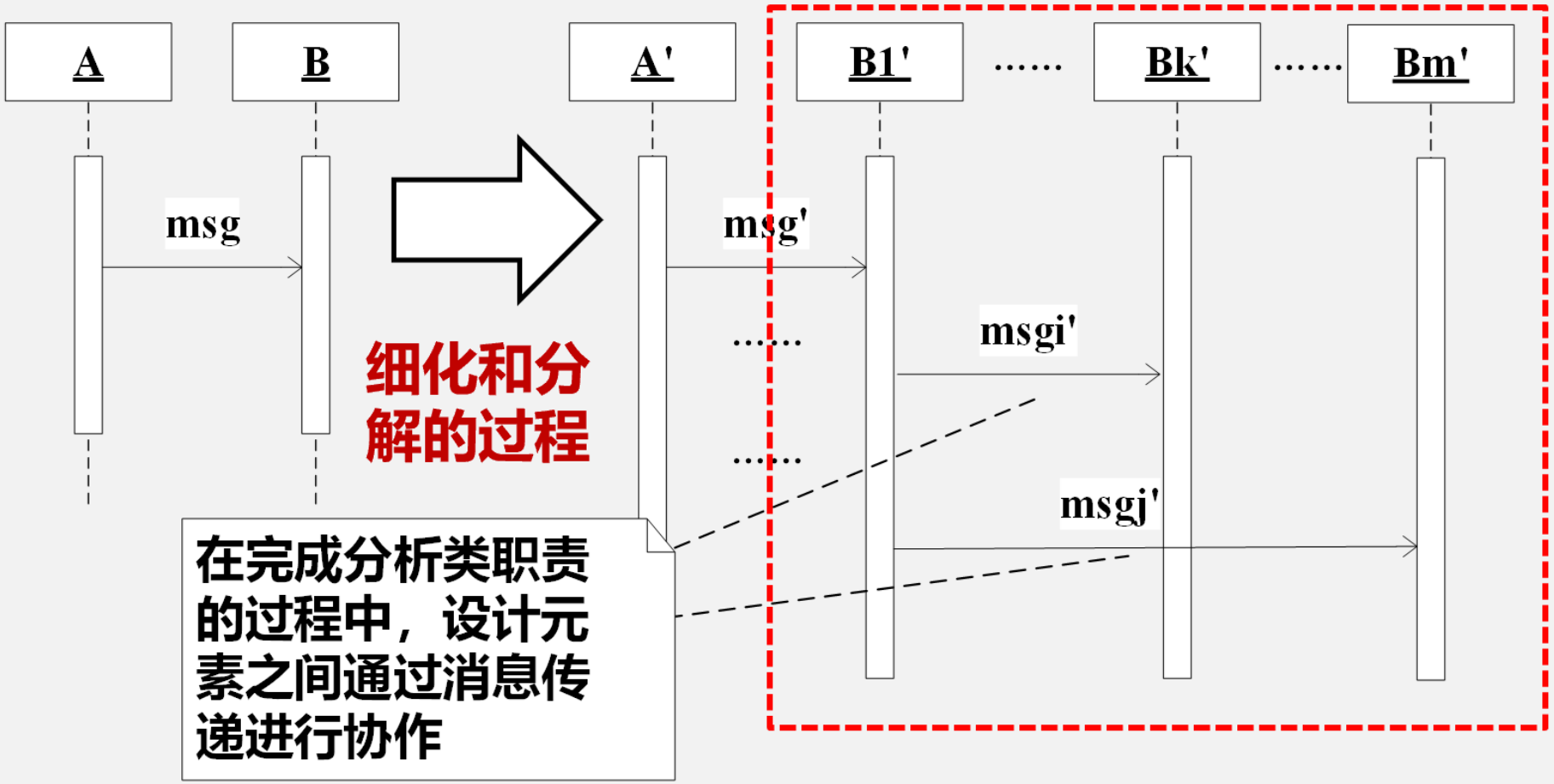
- **结果**
  - ✓ 产出能够描述用例的设计模型（类图、类顺序图、类状态图等）

用例设计的任务

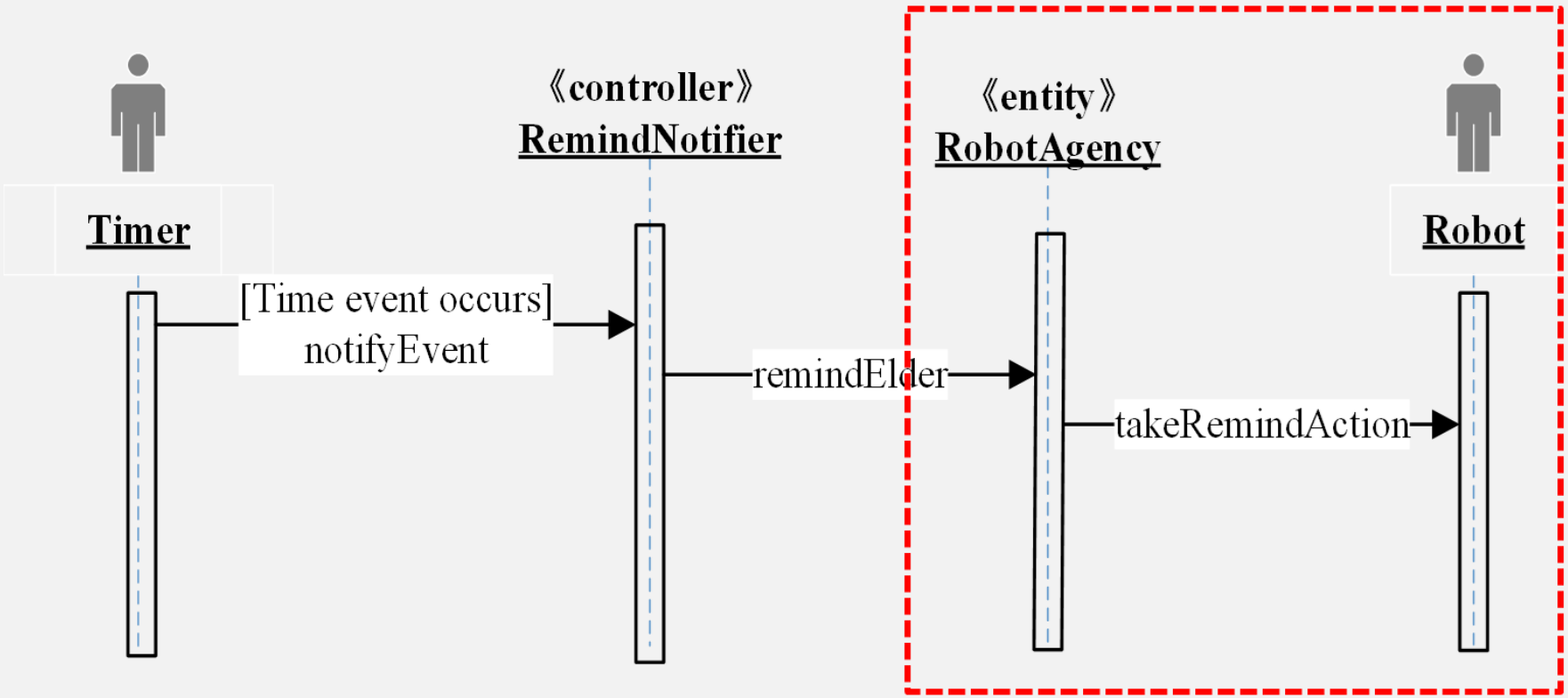
- **任务**
  - ✓针对需求分析模型中的每个用例，基于体系结构给出的设计元素，**设计用例的软件实现方案**
- **设计和建模**
  - ✓各个设计元素如何通过协作完成用例
  - ✓关联各个设计元素，包括**子系统、构件、对象类、界面**等
  - ✓产生详细的设计信息，如**新的设计元素、细节、关联**
- **结果**
  - ✓描述用例设计的顺序图、类图等设计模型

系统与设计类间对应关系

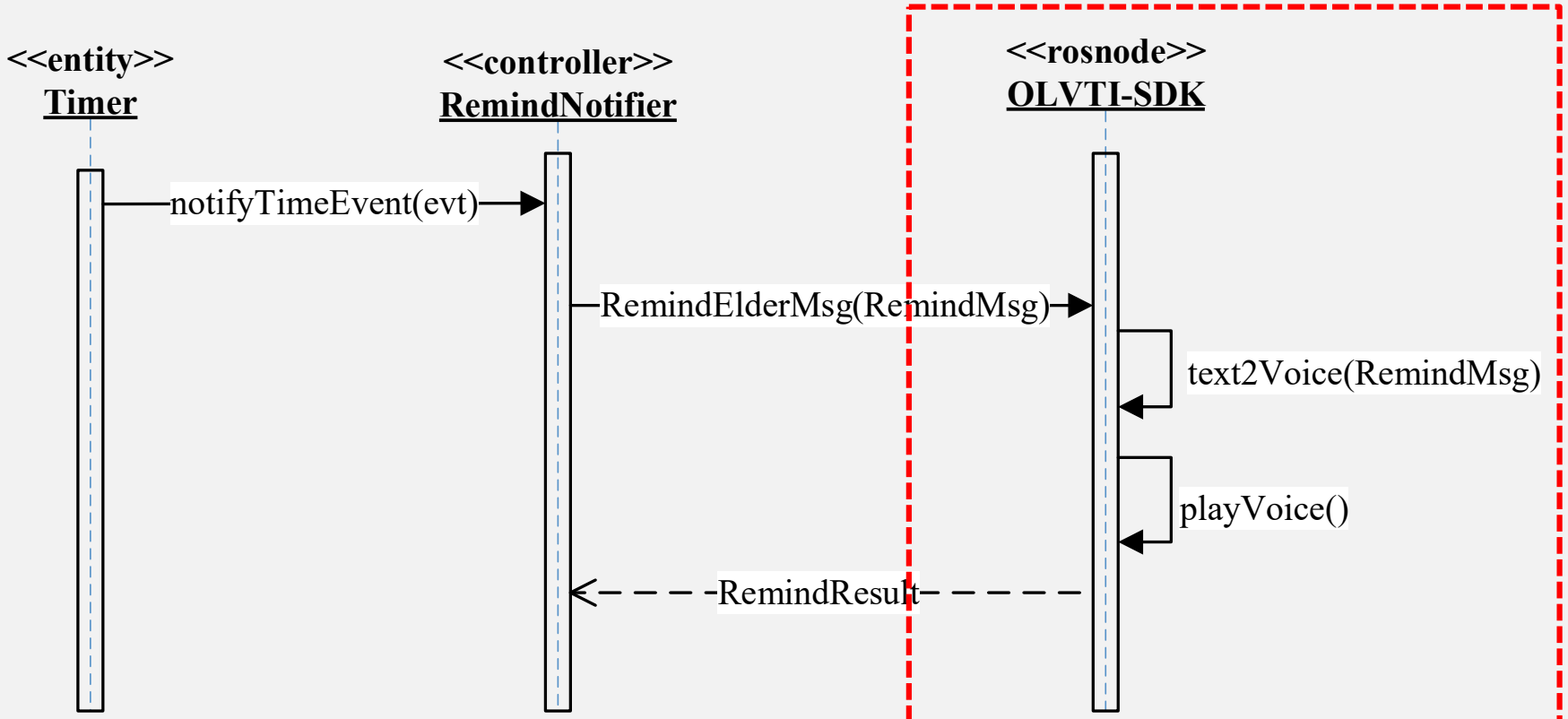
- 系统的一项职责由多个设计元素协同完成



示例1：“提醒服务”用例的顺序图（需求）



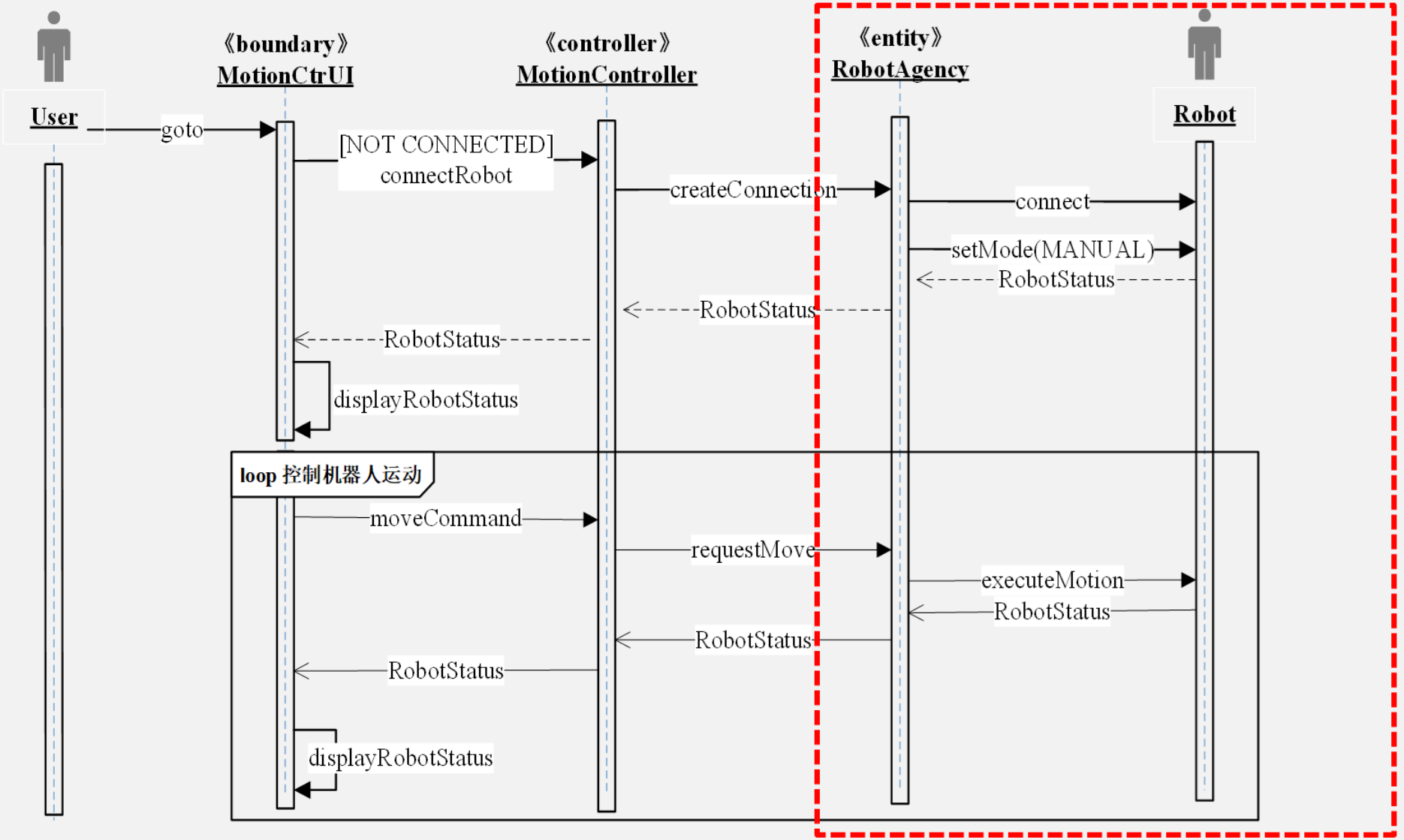
示例1：“提醒服务”用例设计方案（设计）



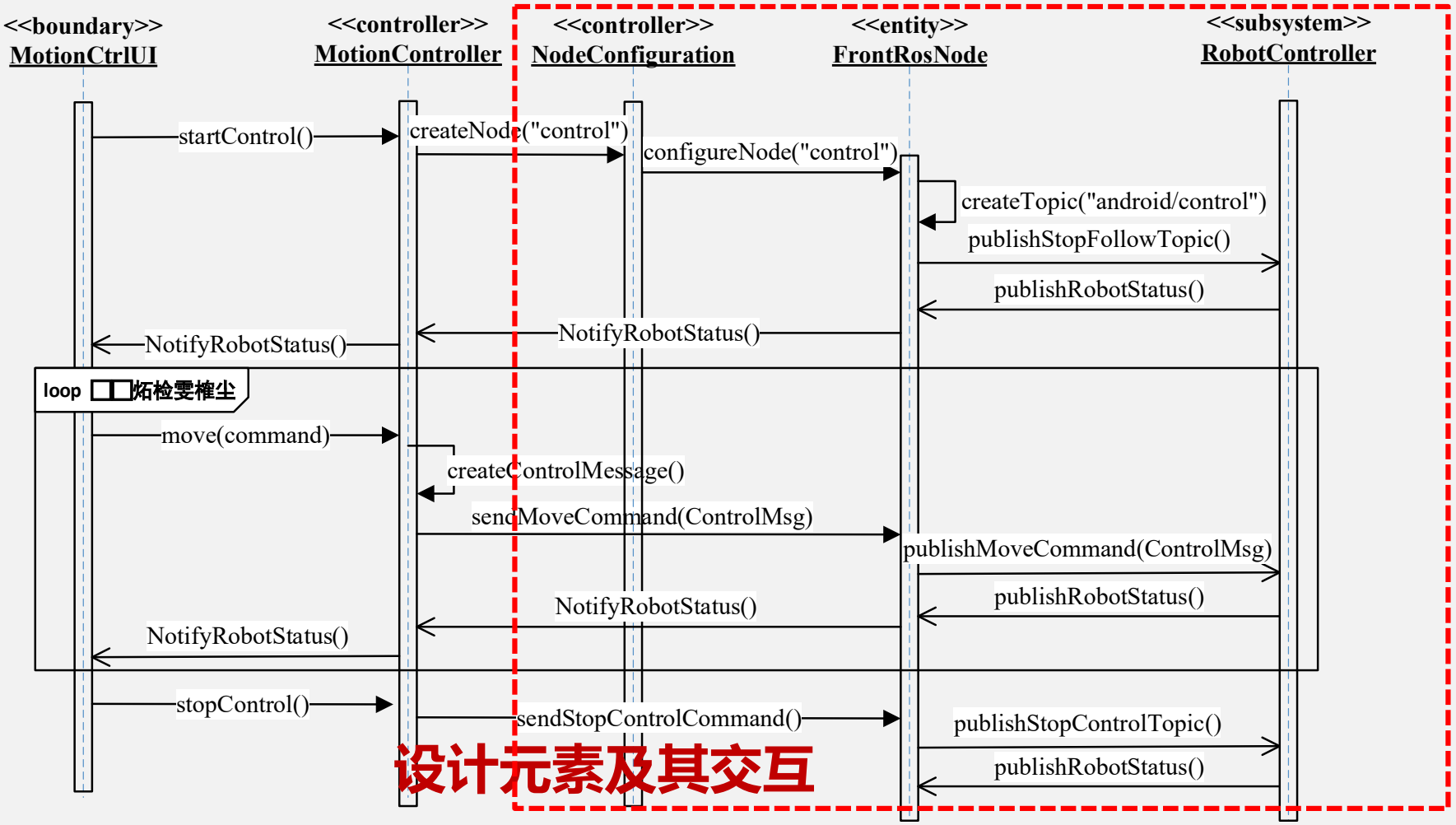
设计元素及其交互

所有设计元素最终通过代码加以实现

示例2：”远程控制机器人”用例顺序图



示例2:“远程控制机器人”用例设计方案（设计）



所有设计元素最终通过代码加以实现

主观题 10分

设置

- 为什么需要用例设计?
- 需求阶段的用例交互模型与详细设计阶段的用例设计模型有何本质区别?

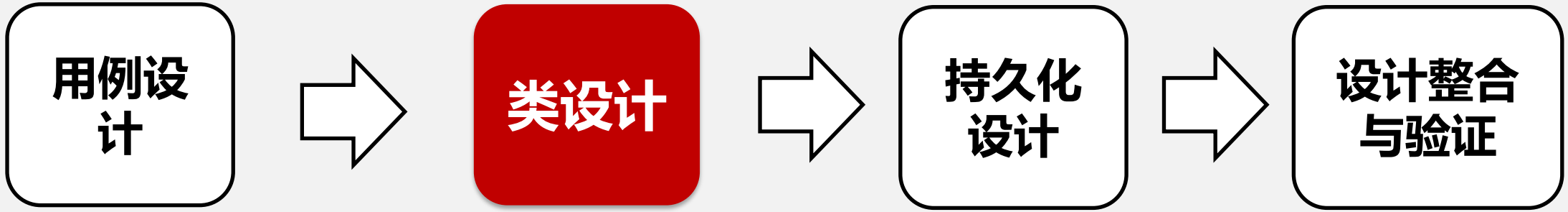


Answer

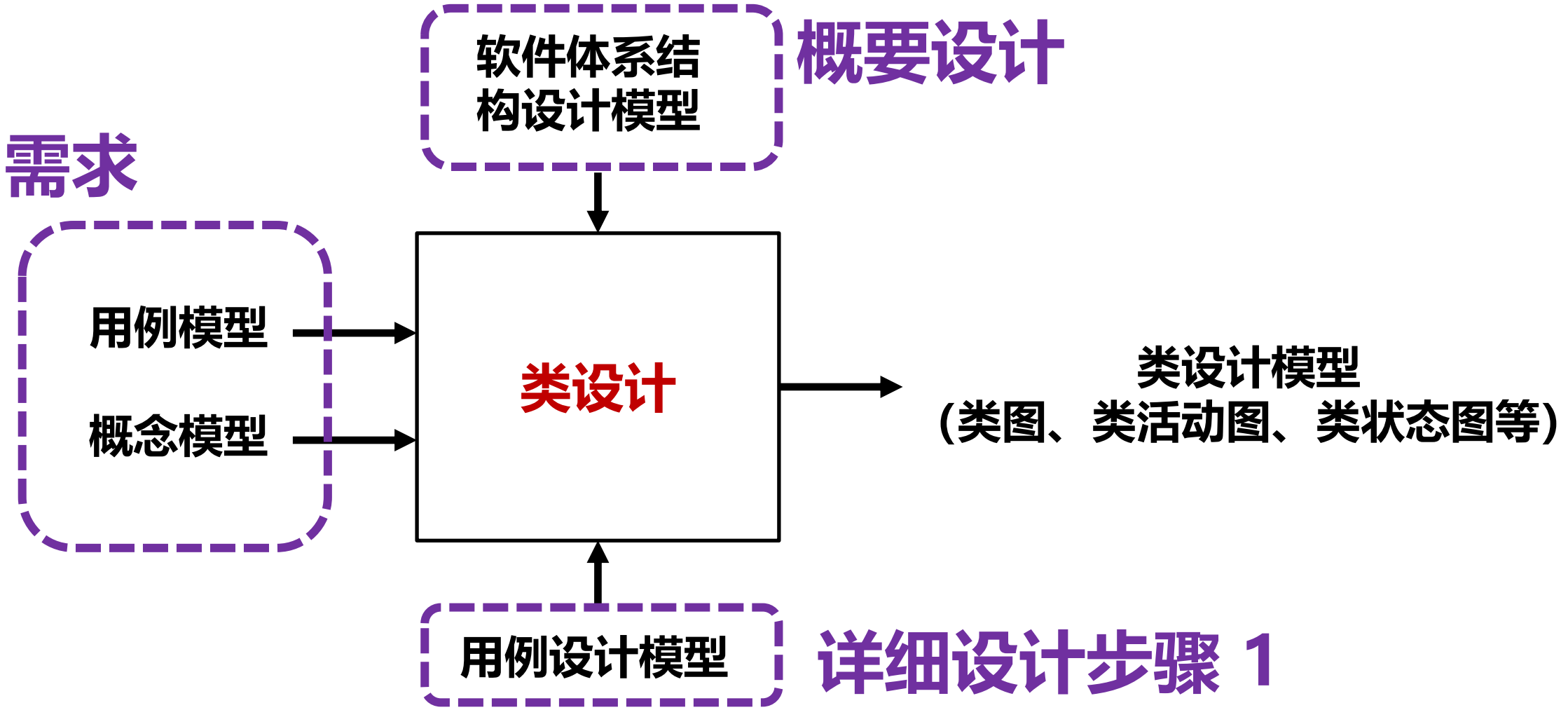
Part 2 详细设计过程

- 面向对象的过程概述
- 用例设计
- 类设计
- 持久化设计
- 设计整合与验证

详细设计的过程（2/4）



类设计的任务



“承上” （类设计要充分考虑软件需求，基于体系结构设计、用例设计、子系统/构件设计的成果）

“启下” （类设计要为后续阶段的编码和实现奠定基础，需要产生足够详细的设计结果）

类设计的任务

❑任务

- ✓对初步识别的**界面类、关键设计类、设计类**等进行设计优化和精化，
- ✓明确设计类的内部实现细节
- ✓精化到可以提交软件实现的程度

❑设计与建模

- ✓明确类的**可见范围，类的操作和属性，类之间的关系**等
- ✓对类设计进行建模

❑结果

- ✓类图、顺序图、状态图、活动图等

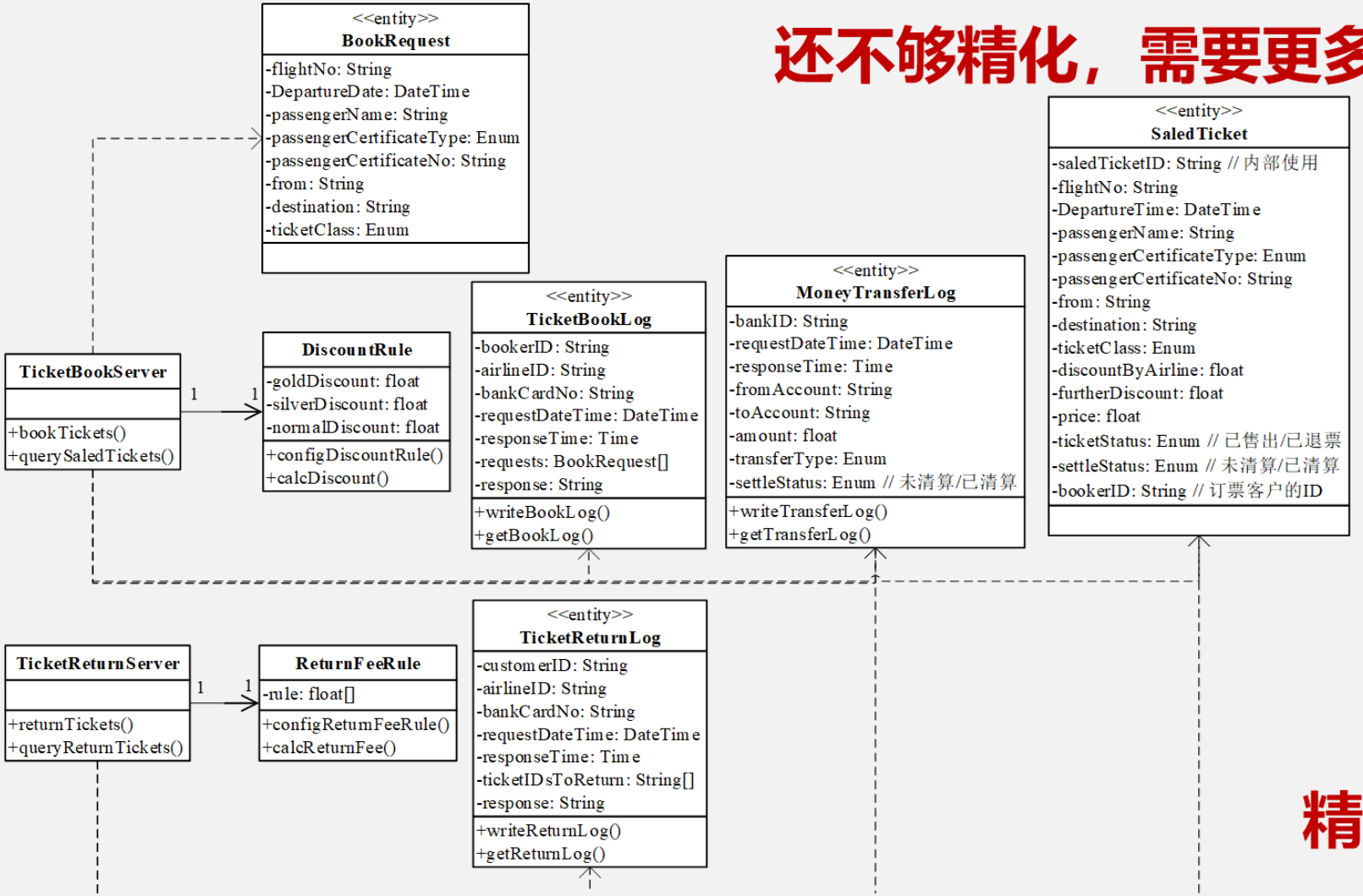
类设计原则

- **准确化**
  - ✓ 对类的内部结构、行为等给予**准确的表达**，以支持程序员精准地理解类设计，进而编写出类的程序代码
- **细节化**
  - ✓ 对类的接口、属性、方法等方面给予**足够详细**的设计，以便程序员能够对类进行编程
- **一致性**
  - ✓ 要确保类的**关系、属性、方法**等的设计是相互一致的，类的内部属性、方法等设计与类的职责、关系等是**相互一致**的
- **遵循软件设计的基本原则**
  - ✓ 按照**模块化、高内聚度、低耦合度、信息隐藏**等基本原则来进行类设计，必要时需要基于这些原则对所设计的类进行必要的拆分和合并，以提高类设计的质量

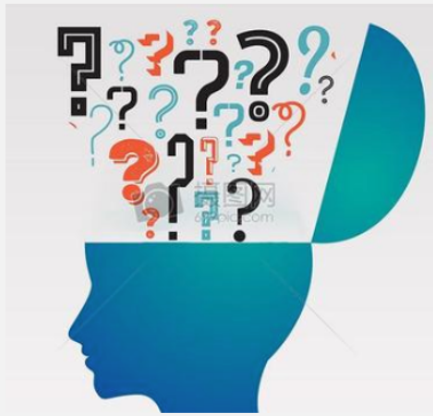
注意事项

- **确保设计类图与概念类图之间、设计类图与用例模型之间的一致性**
  - ✓概念类图中的类在设计类图中有相应的对应物
  - ✓用例模型中的设计元素（消息与系统操作）在设计类图中要有相应的对应物（主要是指设计类及其方法）

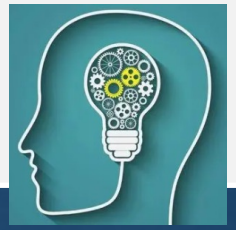
类设计示例



还不够精化，需要更多的详细设计模型

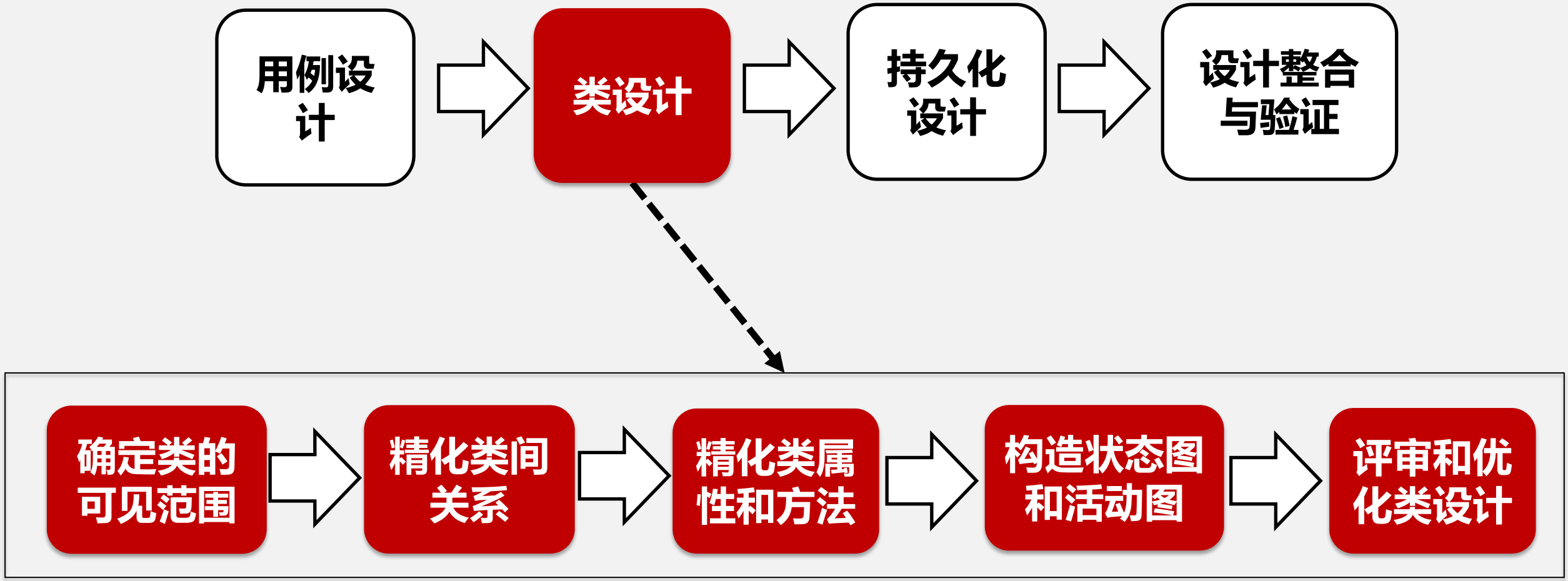


精化类图



基于该类图，程序员指导如何进行编码吗？

类设计过程


用例设计
类设计
持久化设计
设计整合与验证
确定类的可见范围
精化类间关系
精化类属性和方法
构造状态图和活动图
评审和优化类设计

类设计过程



确定类的可见范围

❑可见范围

✓如果类仅仅被其所在的包所使用，那么该类是“私有的”，否则是“公开的”

❑原则

✓尽量缩小类的可见范围，除非确有必要，否则应将类“隐藏”于包的内部

- **public: 公开级范围，软件系统中所有包中的类均可见和可访问该类**
- **protected：保护级范围，只对其所在包中的类以及该类的子类可见和访问**
- **private：私有级范围，只对其所在包中的类可见和访问**

为什么要缩小类的可见范围?



类设计过程



精化类间的关系

❑明确类间的语义关系

- ✓类间关系的语义强度从高到低依次是：**继承，组合，聚合，（普通）关联，依赖**

❑类间关系定义的原则

- ✓ “**自然抽象**”原则，类间关系应该自然、直观地反映软件需求及其实现模型
- ✓ “**强内聚、松耦合**”的原则，即、尽量采用语义连接强度较小的关系

精化类间的关系

- 1:1，即一对一
- 1:n，即一对多
- 0:n，即0对多
- n:m，即多对多等等

类设计过程



精化类属性的设计

❑类属性的命名

- ✓用业务领域的名词或者名词短语来命名

❑类属性的可见范围

- ✓public，对软件系统中的所有类均可见
- ✓protected：仅对本类及其子类可见
- ✓private：仅对本类可见
- ✓遵循“信息隐藏” 原则，尽可能地缩小作用范围

示例：精化LoginUI类属性的设计

- **静态元素、用户输入元素和命令界面元素**
  - ✓ “loginPicture” 类型为静态元素
  - ✓ “account”类型为用户输入元素，如文本框
  - ✓ “password” 类型为用户输入元素（如文本框）
- **可见范围**
  - ✓均为“private”
- **初始值**
  - ✓ “loginPicture”属性的初始值不为空，要有一个预加载的图标；“account”和“password”的初始值为空串

精化类方法的设计

- **细化和明确类中各个方法的以下设计信息**
  - ✓方法名称
  - ✓参数表（含参数的名称和类型）
  - ✓返回类型
  - ✓作用范围
  - ✓功能描述
  - ✓实现算法
  - ✓**前提条件（pre-condition）、出口断言（post-condition）等**

类设计过程



构造类对象的状态图

❑状态图

- ✓如果一个类的**对象**具有**较为复杂的状态**，在其生命周期中**需要针对外部和内部事件**实施一系列的**活动以变迁其状态**，那么可以考虑构造和绘制类的状态图

❑活动图

- ✓如果某个类在实现其职责过程中需要**执行一系列的方法**、与其他的对象进行诸多的交互，那么可以考虑构造和绘制针对该类某些职责的活动图

活动图基本概念

- **活动图类似流程图**
  - 活动图 = 流程图 + 大量软件工程术语
  - 活动图的表达能力包括了逻辑判断、分支甚至并发
- **活动图的表达能力要远超流程图**
  - 流程图仅展示一个固定的过程
  - 活动图可展示**并发和控制分支**，并且可以对**活动之间的信息流动**进行建模
  - 活动图在表达流程的基础上继承了一部分协作图的特点，即可以适当表达**活动之间的关系**

活动图组成元素

- 动作和活动节点
- 开始和终止
- 控制流
- 判断节点
- 分叉节点
- 合并节点
- 结合节点

动作和活动节点

- **动作和活动节点**
- **开始和终止**
- **控制流**
- **判断节点**
- **分叉节点**
- **合并节点**
- **结合节点**

- 动作代表一个**原子操作**，操作可能是任何**合法行为**
- 动作包括但不限于：创建或删除对象、发送消息、调用接口，甚至数学运算以及返回表达式的求值结果
- 活动节点是**一系列动作**，主要用于实现动作序列的简化和动作图的嵌套。活动节点在图例上的表达方式和动作相同

x=5

开始和终止

- 动作和活动节点
- **开始和终止**
- 控制流
- 判断节点
- 分叉节点
- 合并节点
- 结合节点

- 活动图中的**开始和终止**是两个标记符号，分别标记了**业务流程**的**起始位置**和**结束位置**
- 活动图中必须**有且仅有一个开始标记**，一般**至少有一个结束标记**（i.e., 存在一些特殊的无穷过程不存在终止标记）



开始

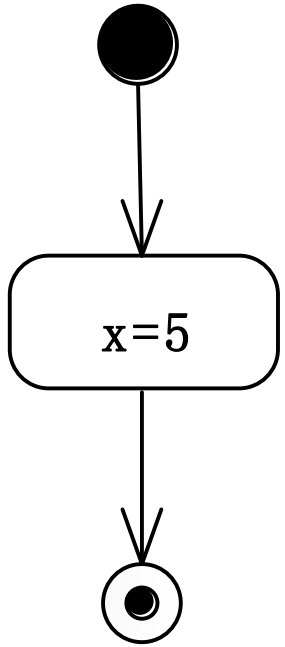


终止

控制流

- 动作和活动节点
- 开始和终止
- **控制流**
- 判断节点
- 分叉节点
- 合并节点
- 结合节点

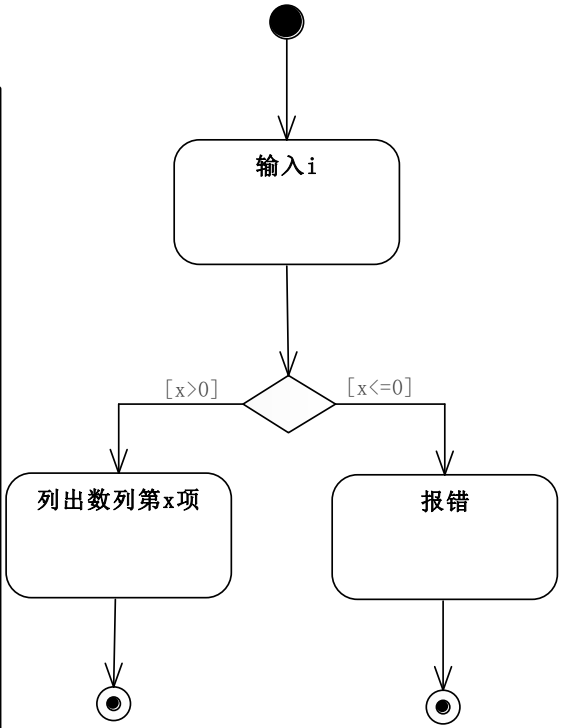
- 控制流是活动图中用于标示**控制路径**的一种符号。它负责当一个动作或活动节点**执行完毕**后，将执行主体从当前已完毕的节点转移到过程的**下一个动作**或**动作节点**
- 控制流从活动图的开始标记开始运行，经过**顺序、分支**等结构**引导**着各个动作的**连续执行**



判断节点 （Decision Node）

- 动作和活动节点
- 开始和终止
- 控制流
- 判断节点
- 分叉节点
- 合并节点
- 结合节点

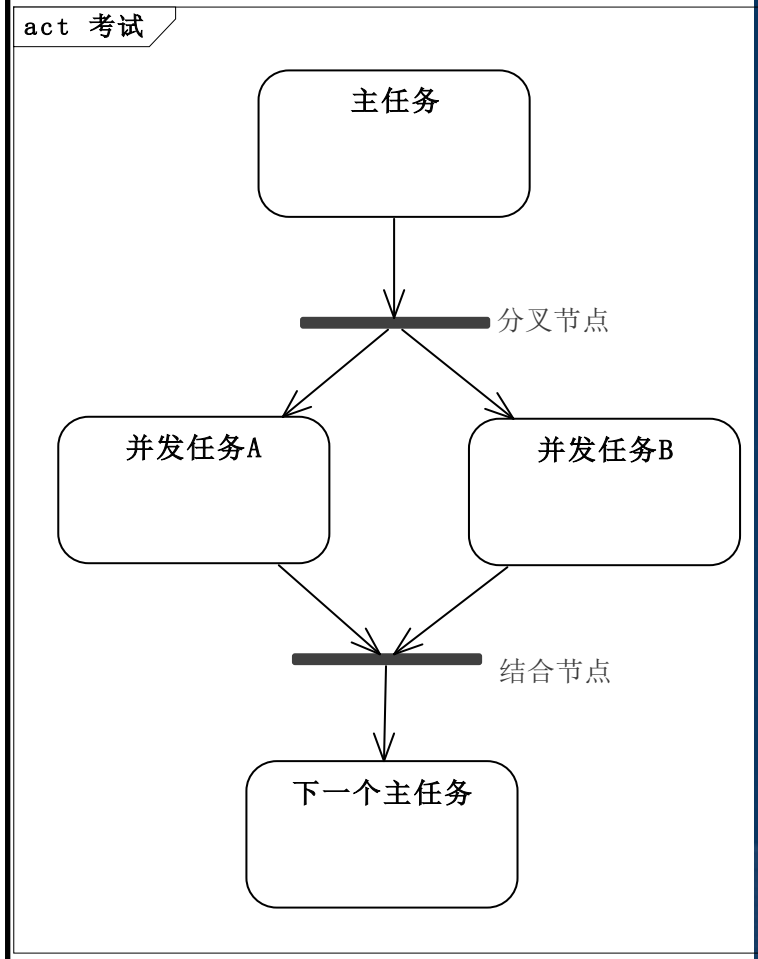
- **判断节点**是活动图中进行**逻辑判断**、**创造分支**的一种方法
- 判断节点具有**一个进入控制流**和**至少两个导出控制流**
- 判断节点具有多个导出流，对于每条导出流而言，应当在表示该控制流的箭头上**附加控制条件**



分叉节点 (Fork Node)

- 动作和活动节点
- 开始和终止
- 控制流
- 判断节点
- **分叉节点**
- 合并节点
- 结合节点

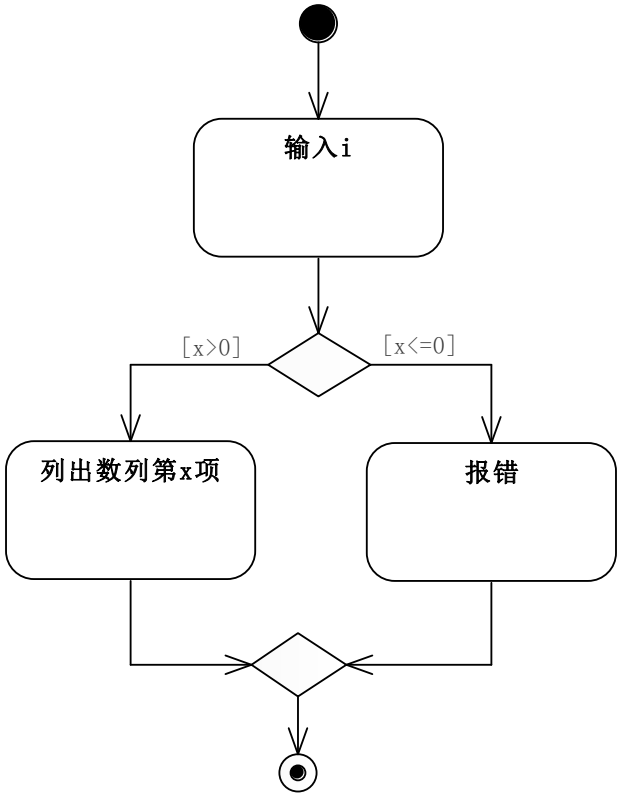
- 分叉节点是从线性流程**进入并发过程的过渡节点**，它拥有一个进入控制流和多个离开控制流
- 不同于判断节点，分叉节点的所有离开流程是**并发关系**，即分叉节点使执行过程进入**多个动作并发**的状态
- 分叉节点在活动图中表示为一根**粗横线**，粗横线上方的进入箭头表示进入**并发状态**，离开粗横线的箭头指向的各个动作**将并发发生**



合并节点 (Merged Node)

- 动作和活动节点
- 开始和终止
- 控制流
- 判断节点
- 分叉节点
- **合并节点**
- 结合节点

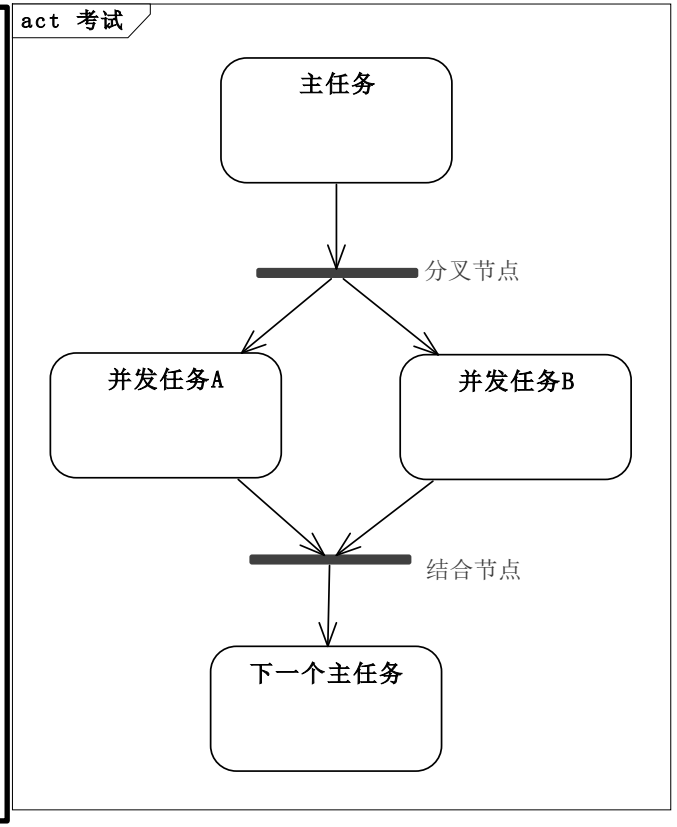
- 合并节点将**多个控制流**进行合并，并**统一导出**到同一个**离开控制流**
- 合并节点仅有逻辑意义而**没有时间和数据上的意义**：几个动作都指向同一个合并节点也并不意味着这些动作要在进入之后互相等待或进行同步数据之类的操作
- 在各个进入合并节点的控制流间**不存在并发关系**，所以也没有等待和同步过程



结合节点 (Join Node)

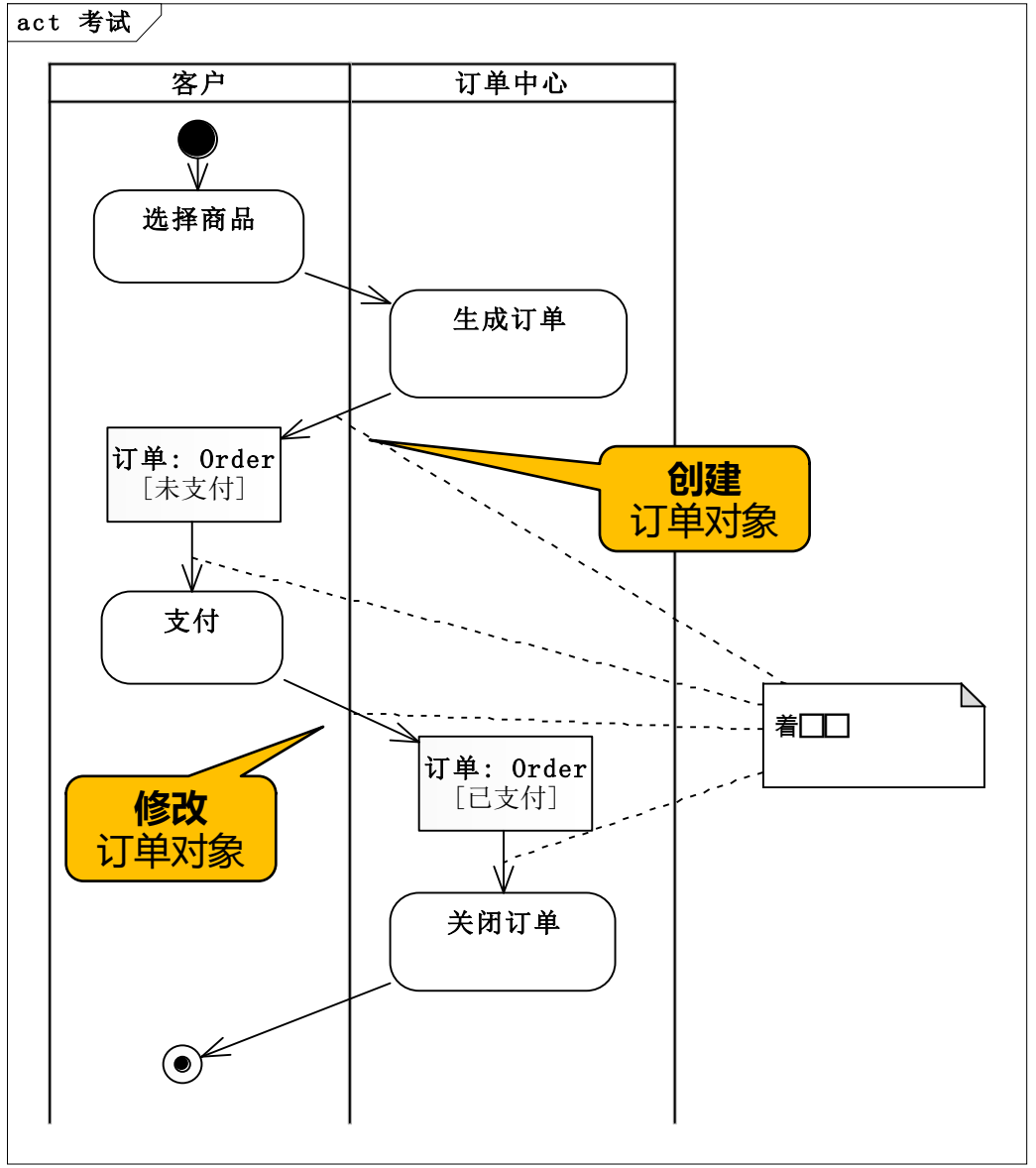
- 动作和活动节点
- 开始和终止
- 控制流
- 判断节点
- 分叉节点
- 合并节点
- **结合节点**

- 结合节点的各个“进入控制流”之间**具有并发关系**，它们在系统中同时运行
- 在各个支流收束时，为了保证数据的统一性，先到达结合节点的控制流都必须等待直到**所有的流程全部到达这个结合节点**后才会继续进行，转移到“离开控制流”所指向的动作开始运行



对象流

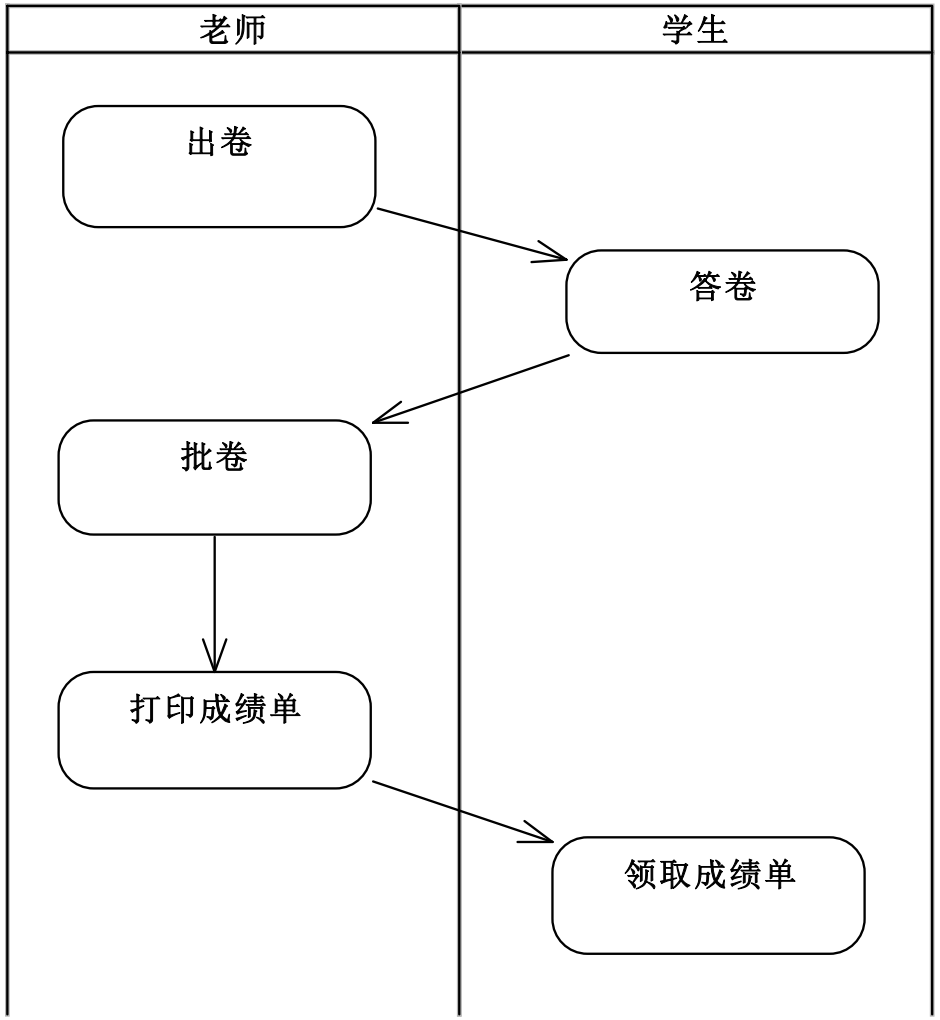
- 对象也可被包含在一个活动图相关的控制流中
- 对象流是一种连接两个节点的活动边，这两个节点通常是一个**可执行节点**和一个**对象节点**
- 对象流用来表示**源活动生产了一个对象** or **目标活动消费了一个对象**
- **例子：某系统支付订单时的活动图**
  - ✓ 图中连接活动节点与对象节点的连线均为对象流
  - ✓ 对象流可能会**创建对象**（例如“生成订单”活动创建了一个订单对象）或**使用或修改对象状态**（例如“支付”活动将订单对象的状态修改为“已支付”）

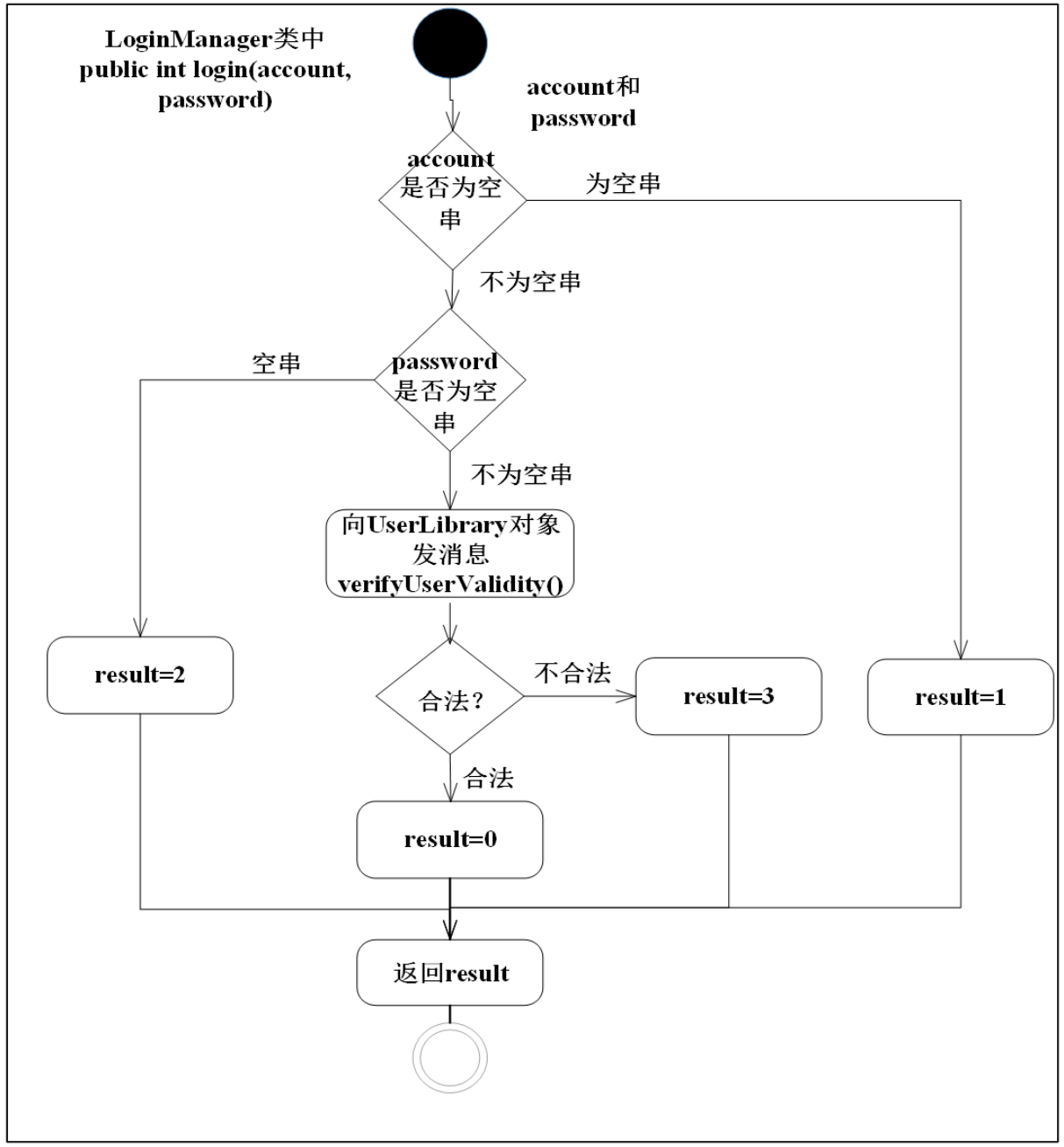


泳道（UML 1.x / UML 2.x Partition）

- 泳道是将活动中的具体活动按照负责进行该活动的对象**进行分区**，一条泳道中的所有活动由**同一个对象执行**

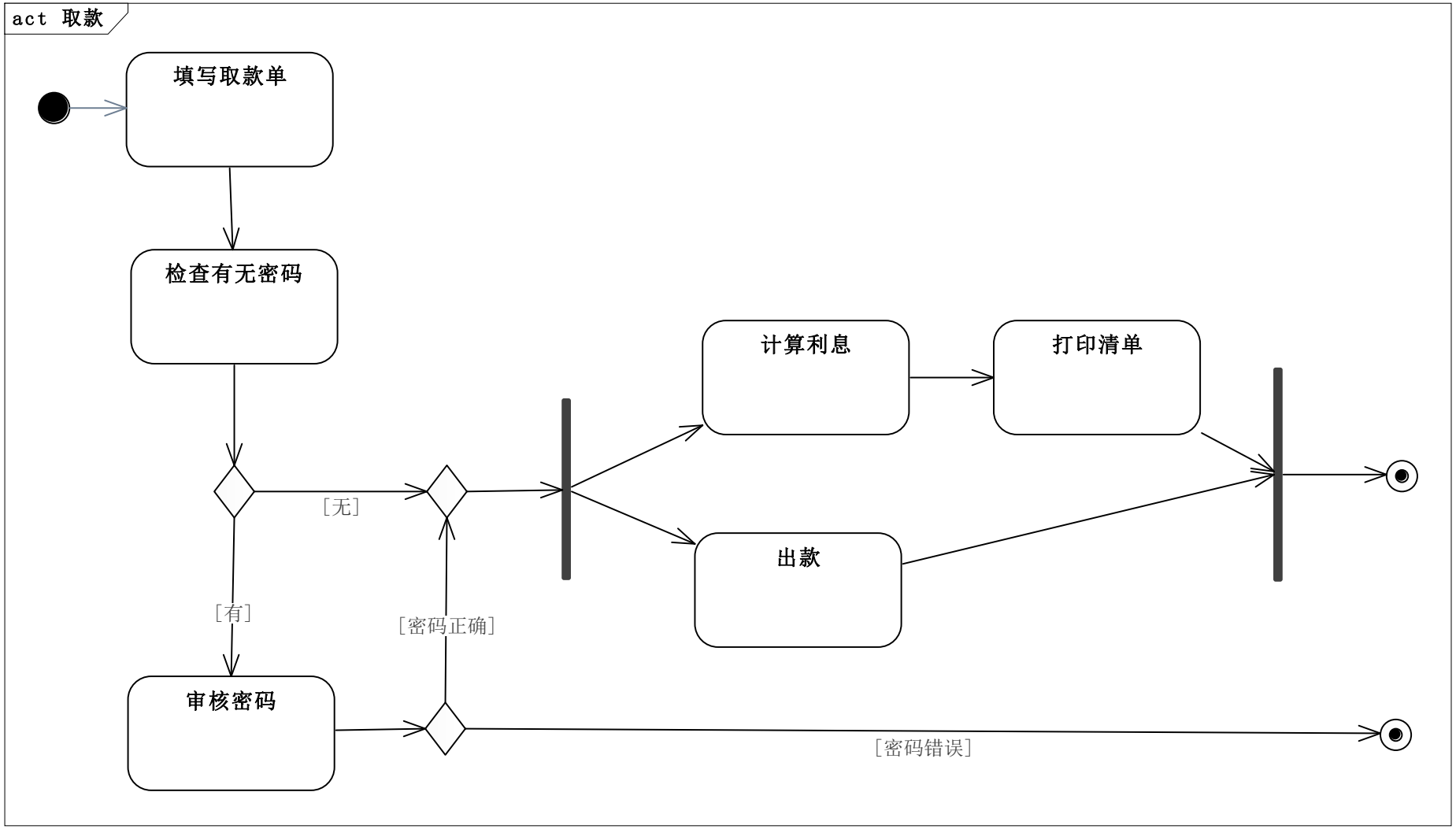
- 除了以上的对线性流程进行分区以外，使用泳道可更清晰地表示**并发**





示例：用活动图描述的Login()方法的详细设计

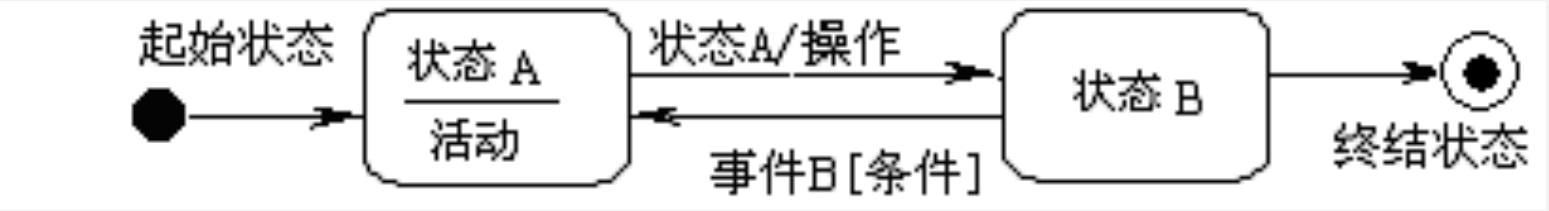
活动图基本概念



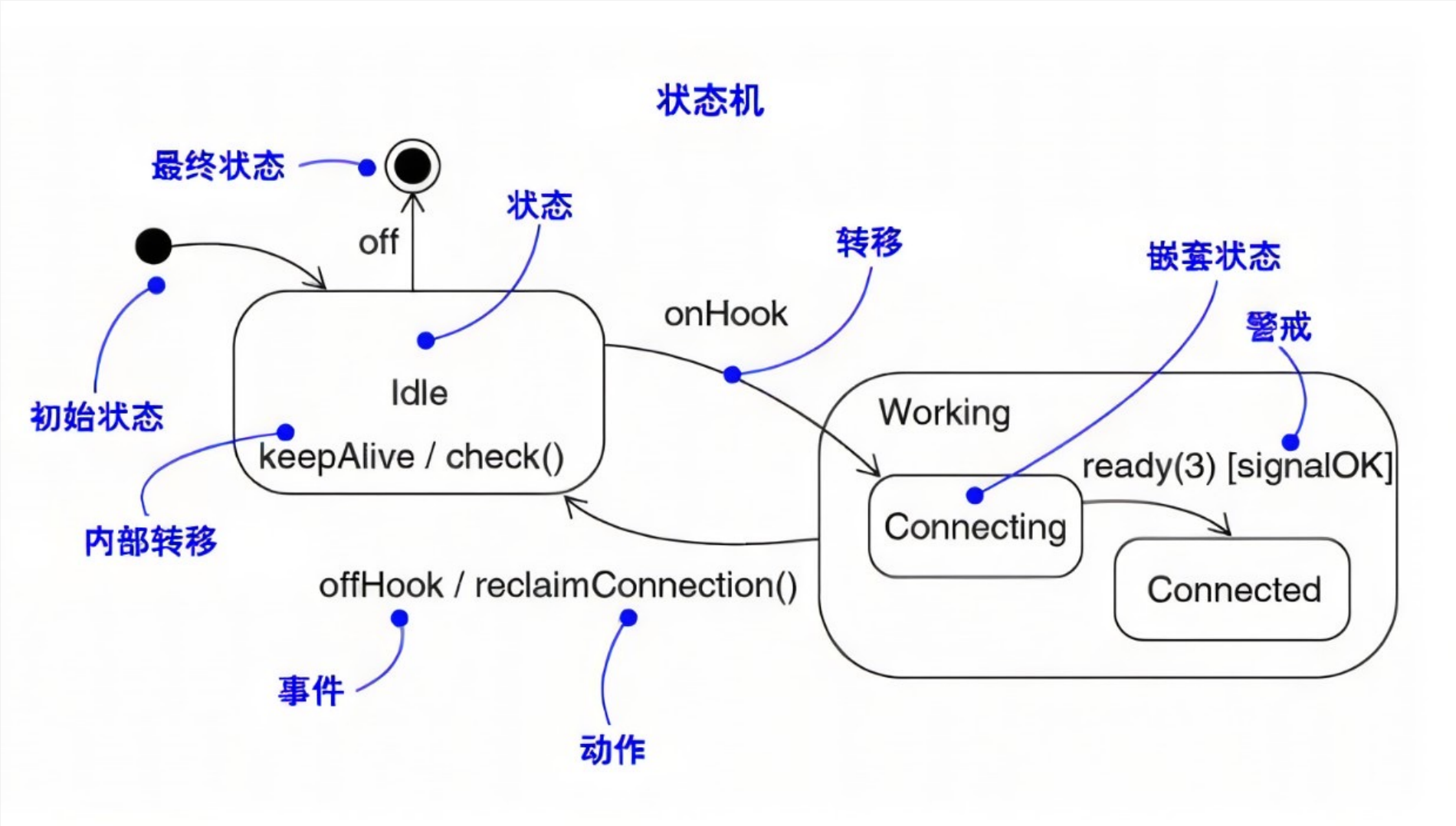
状态图

状态是一种存在状况，它具有一定的时间稳定性。即在一段有限的时间内保持对象（或系统）的外在情况和内在特性的相对稳定。

状态图提供了对象在其生命周期中可能出现的状态及其行为的描述。一个状态图包括一系列状态、事件以及状态之间的转换，其作用是能够为类图中每一个类进行动态行为说明，以获得对类的方法的细节描述的建模支持。



例子：状态图的图符


状态机
最终状态
状态
off
转移
嵌套状态
onHook
警戒
初始状态
Idle
Working
keepAlive / check()
ready(3) [signalOK]
内部转移
Connecting
Connected
offHook / reclaimConnection()
事件
动作

状态图

✓一个状态一般包括3个部分：状态名称、可选的状态变量的变量名、变量值和可选的活动表。



状态图

转移是两个状态之间的关系，一个转移由五部分组成：

- ✓ 源状态：即受转移影响的状态；如果一个对象处于源状态，当该对象接收到转移的触发事件时或满足监护条件时，就会激活状态。
- ✓ 事件触发：源状态中的对象接收这个事件使转移合法地激活，并使监护条件满足。
- ✓ 监护条件[Guard]：是一个布尔表达式，当转移因事件触发器的接收而被触发时对这个布尔表达式求值；如果表达式为真，则激活转移；如果为假，则不激活转移，如果没有其他的转移被此事件触发，则事件丢失。
- ✓ 动作：是一个可执行的原子计算，它可以直接作用于对象。
- ✓ 目标状态：转移完成后对象的状态。

状态图的主要图符

转移的图形映射



Event/Action

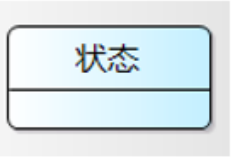
触发器（Trigger）表示会触发转移执行的触发器，是一种事件

守卫/监护条件（Guard）是一种约束，如果守卫条件为真则此转移是可触发的，否则为不可触发的转移

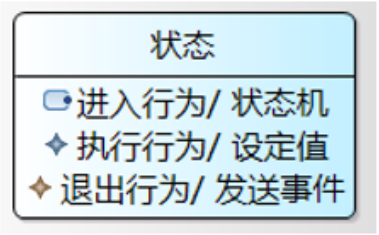
影响（Effect）是一种动作，该动作导致状态的转移

状态图的主要图符

状态部分的图形映射

状态(State) → 

区域(Region) → 

状态行为(Behavior) → 

最终状态(Final State) → 

伪状态的图形映射

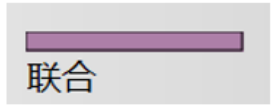
初始 → 

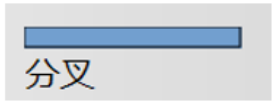
终结 → 

深历史 → 

浅历史 → 

进入点 → 

联合 → 

分叉 → 

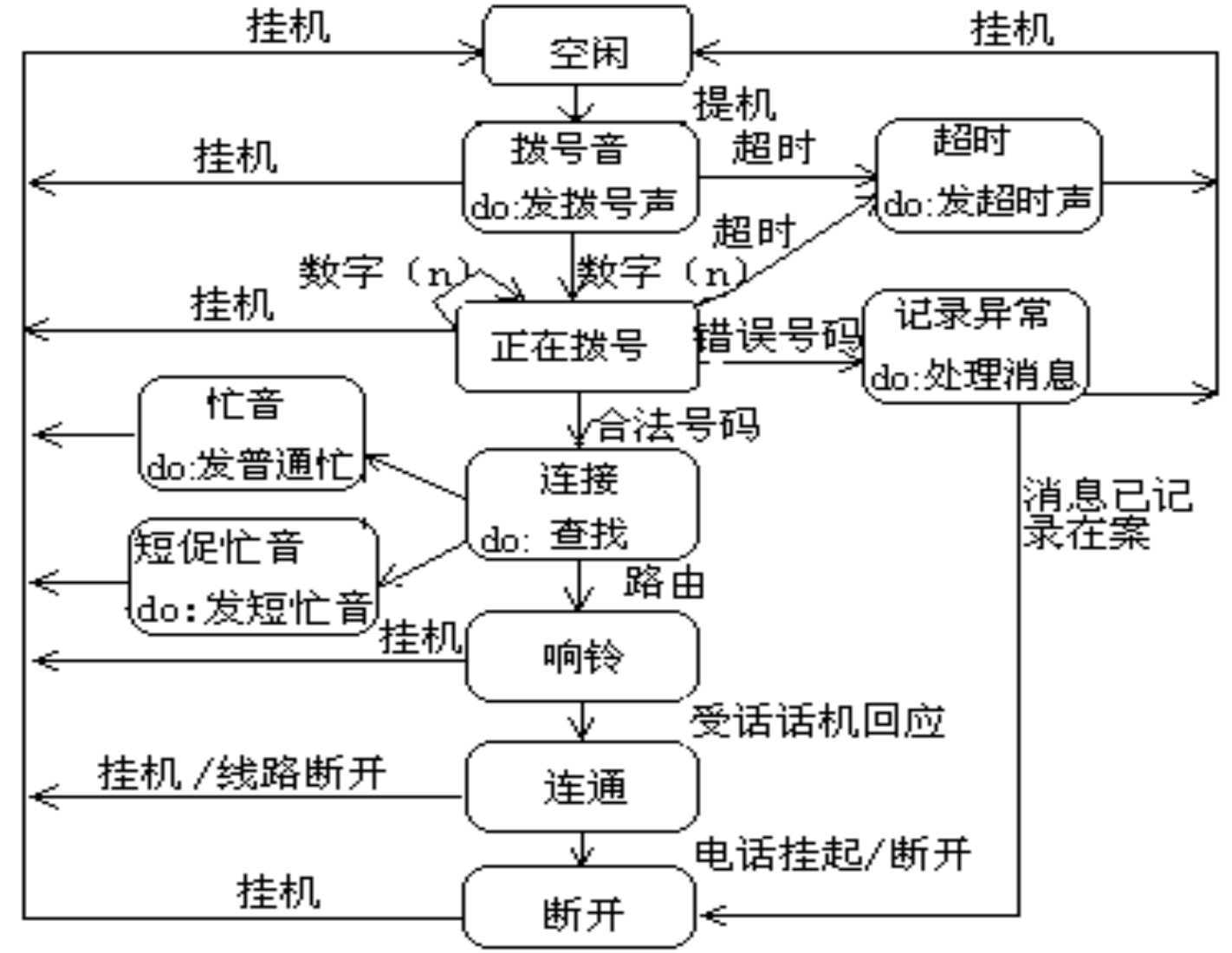
连接 → 

选择 → 

退出点 → 

状态图

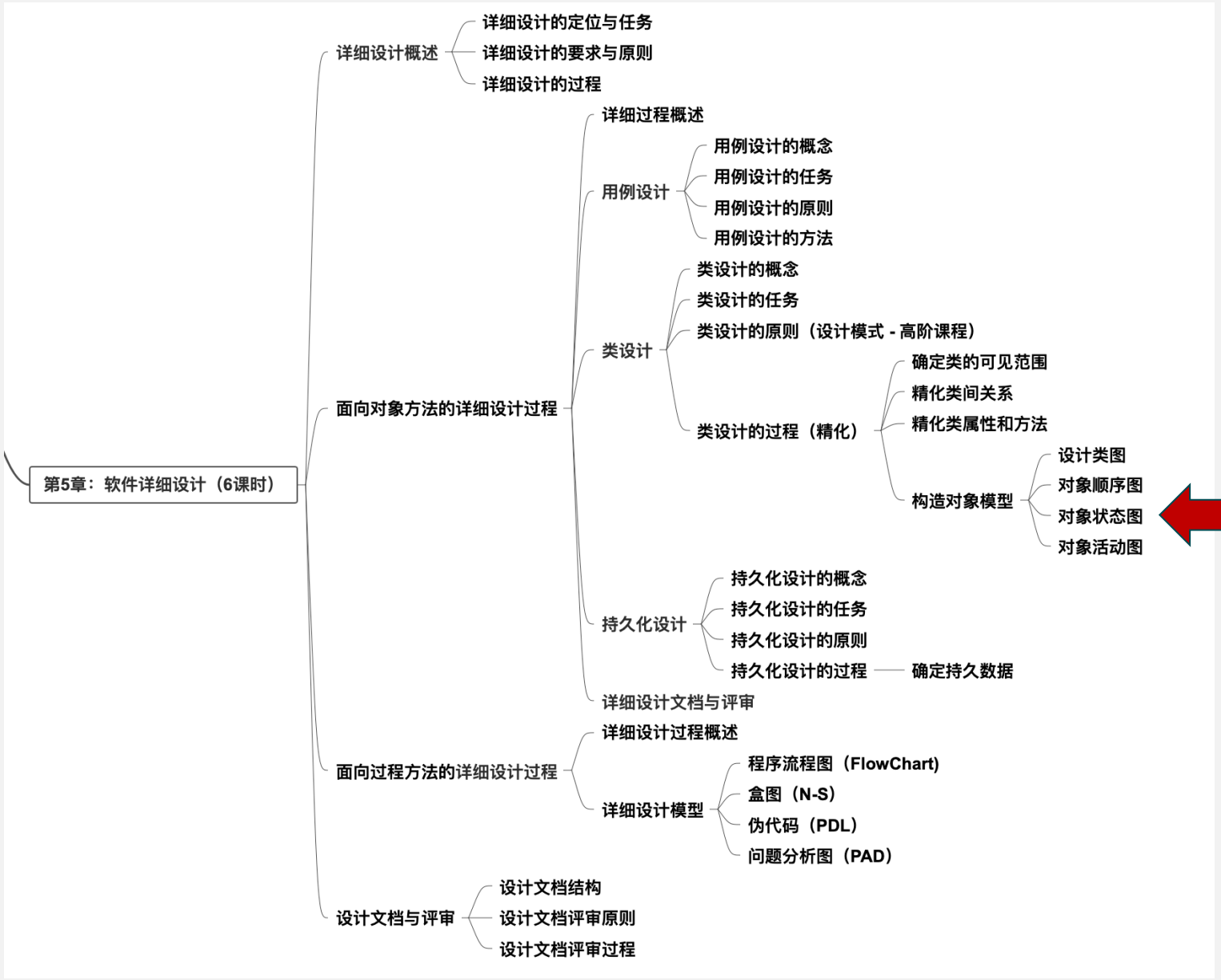
例:


空闲
挂机
挂机
提机
拨号音
do:发拨号声
挂机
超时
超时
do:发超时声
超时
数字（n）
数字（n）
挂机
正在拨号
错误号码
记录异常
do:处理消息
忙音
do:发普通忙
合法号码
连接
do: 查找
消息已记录在案
短促忙音
do:发短忙音
路由
挂机
响铃
受话话机回应
挂机/线路断开
连通
电话挂起/断开
挂机
断开

类设计输出的软件制品

- 详细的类属性、方法和类间关系设计的**类图**
- 描述类方法实现算法交互细节的**顺序图**
- 描述类方法实现算法细节的**活动图**
- 描述类状态变化的**状态图**（至少三选一）

课程大纲（回顾）

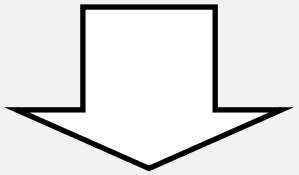

第5章：软件详细设计（6课时）
详细设计概述
详细设计的定位与任务
详细设计的要求与原则
详细设计的过程
面向对象方法的详细设计过程
详细过程概述
用例设计
用例设计的概念
用例设计的任务
用例设计的原则
用例设计的方法
类设计
类设计的概念
类设计的任务
类设计的原则（设计模式 - 高阶课程）
类设计的过程（精化）
确定类的可见范围
精化类间关系
精化类属性和方法
构造对象模型
设计类图
对象顺序图
对象状态图
对象活动图
持久化设计
持久化设计的概念
持久化设计的任务
持久化设计的原则
持久化设计的过程
确定持久数据
详细设计文档与评审
面向过程方法的详细设计过程
详细设计过程概述
详细设计模型
程序流程图（FlowChart）
盒图（N-S）
伪代码（PDL）
问题分析图（PAD）
设计文档与评审
设计文档结构
设计文档评审原则
设计文档评审过程

面向对象方法的软件设计过程（回顾）

系统需求分析

系统模型

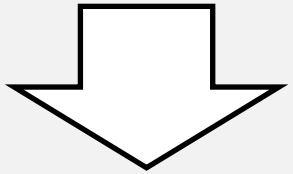
用例图+
系统顺序图（系统状态图/活动图）
+系统操作合约 + 概念类图



概要（架构）设计

组件模型

组件图 + 组件顺序图
(组件状态图/活动图)



详细（类）设计

类模型

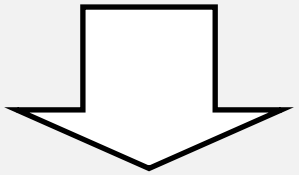
类图 + 类顺序图
(对象状态图/活动图)

面向过程方法的软件设计过程（回顾）

系统需求分析

系统模型

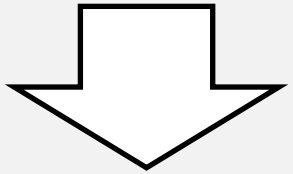
数据流图DFD + 数据字典DD + + 实体关系图E-R



概要（架构）设计

组件模型

层次图H-IPO + 结构图SD

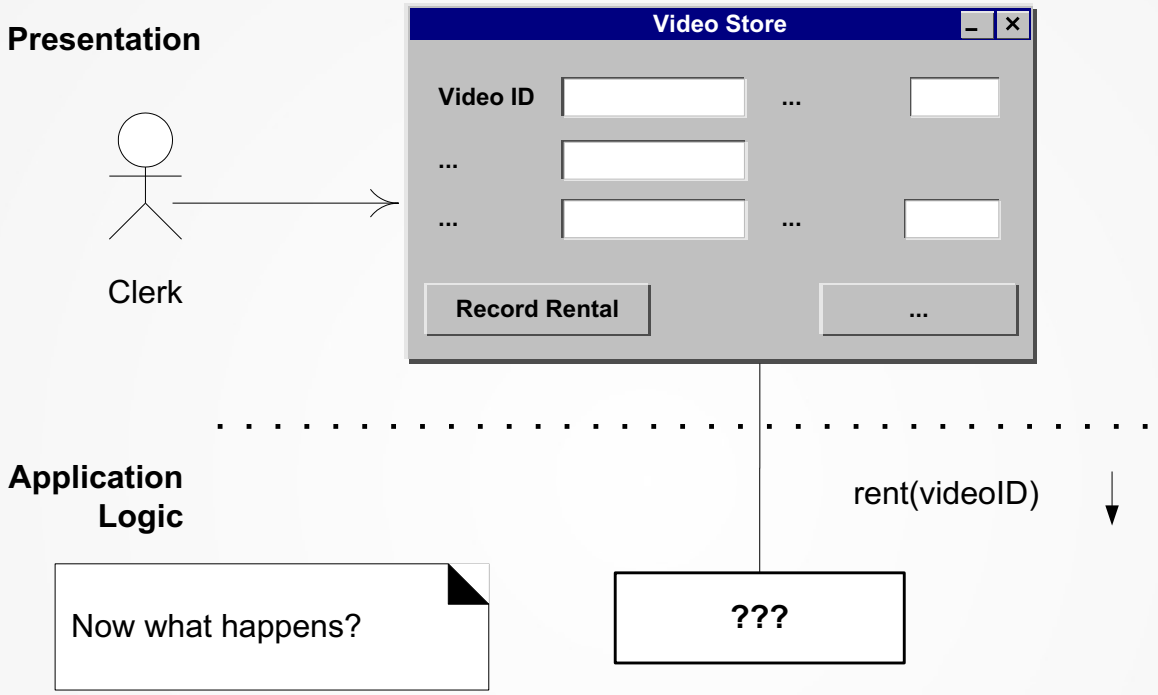


详细（函数）设计

过程模型

流程图 + N-S图 + PAD图

详细设计的动机（回顾）

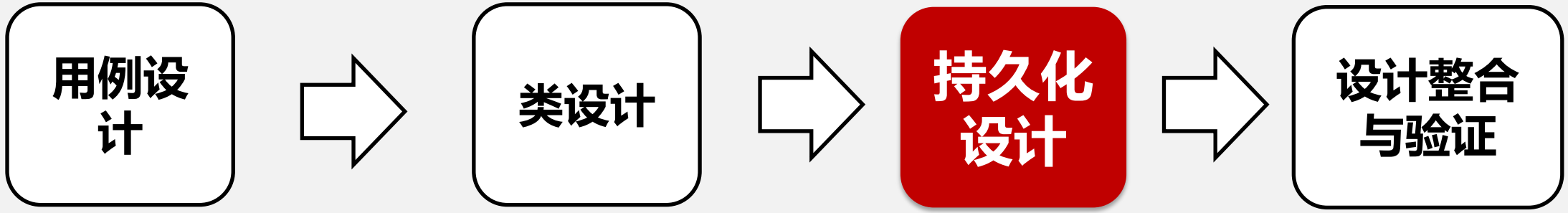


- **面向对象**
  - What object should receive this message?
  - What objects should interact to fulfill the request, and how?
- **面向过程**
  - What function should receive this message?
  - What function should be composed to fulfill the request, and how?

Part 2 详细设计过程

- 面向对象的过程概述
- 用例设计
- 类设计
- 持久化设计
- 设计整合与验证

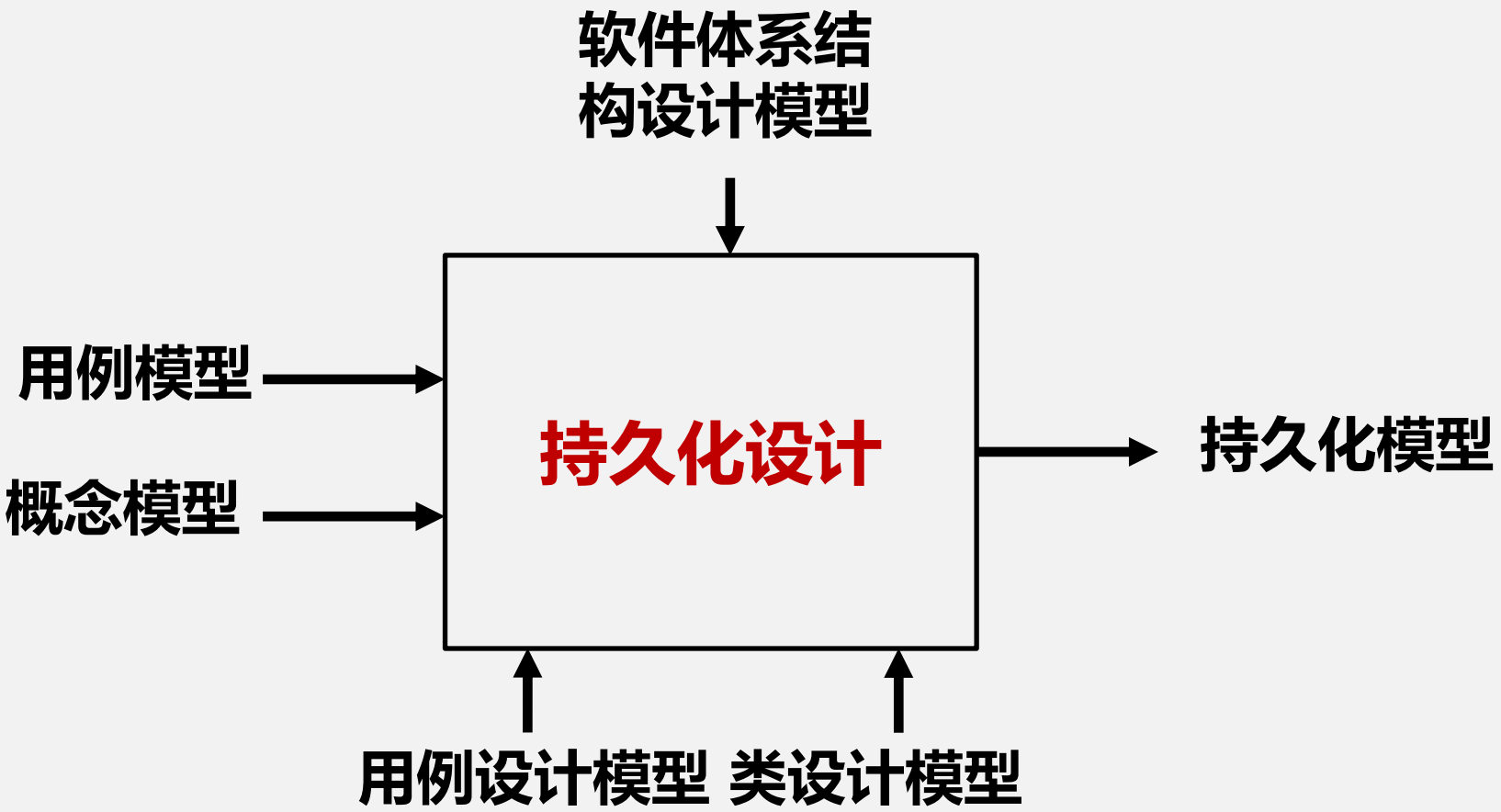
详细设计的过程（3/4）



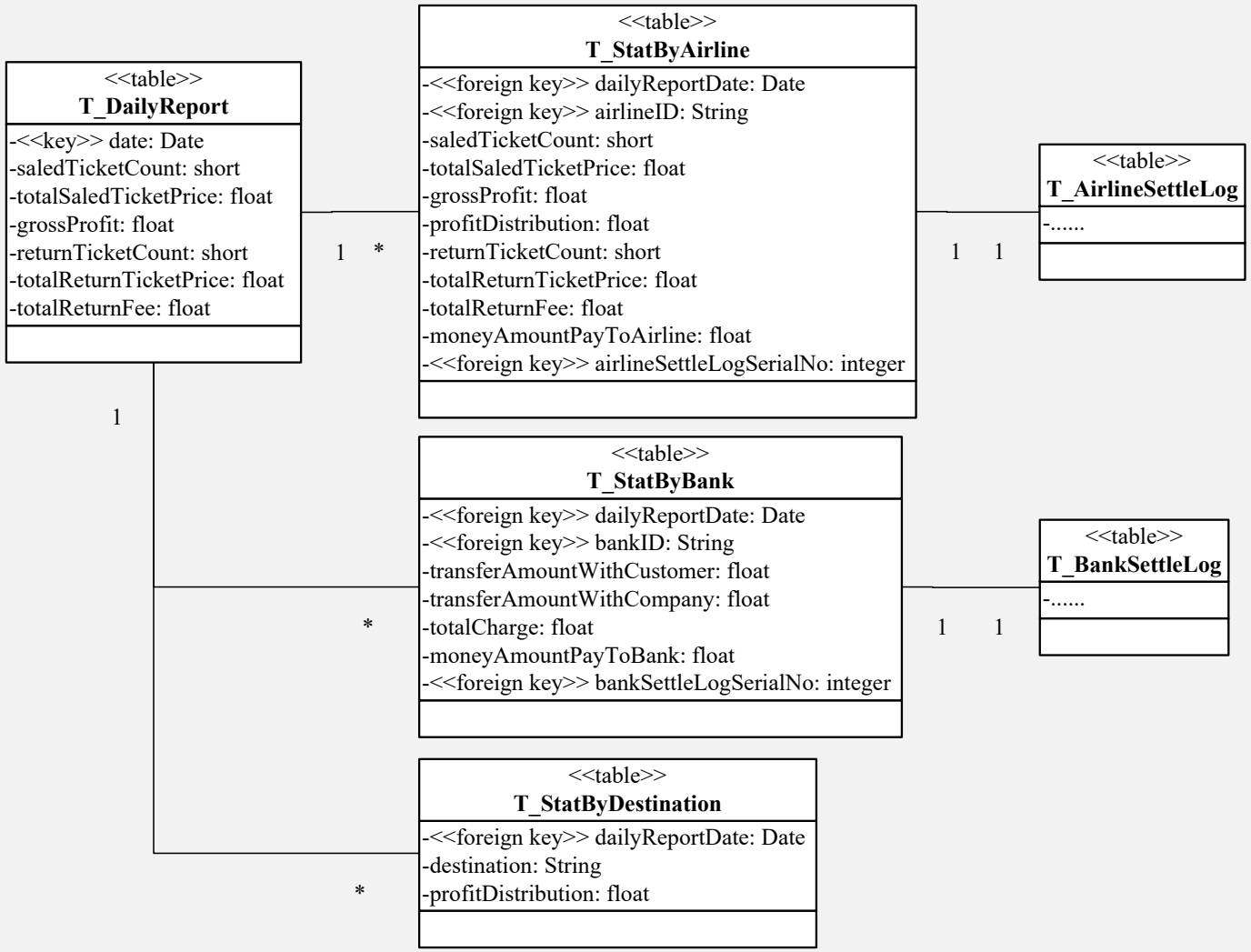
为什么要进行持久化设计

- **软件系统涉及各种信息，需要将其抽象为计算机可以理解和处理的数据**
- **有些数据需要持久保存的，存放在永久存储介质中**
  - ✓开展数据设计，以支持信息的抽象、组织、存储和读取
- **有些数据则需要存放在内存空间中，由运行的进程对其进行处理**
  - ✓在类设计中抽象和封装为类属性及其数据类型

数据设计的任务



示例：数据模型设计


<<table>>
T_DailyReport
-<<key>> date: Date
-saledTicketCount: short
-totalSaledTicketPrice: float
-grossProfit: float
-returnTicketCount: short
-totalReturnTicketPrice: float
-totalReturnFee: float
<<table>>
T_StatByAirline
-<<foreign key>> dailyReportDate: Date
-<<foreign key>> airlineID: String
-saledTicketCount: short
-totalSaledTicketPrice: float
-grossProfit: float
-profitDistribution: float
-returnTicketCount: short
-totalReturnTicketPrice: float
-totalReturnFee: float
-moneyAmountPayToAirline: float
-<<foreign key>> airlineSettleLogSerialNo: integer
<<table>>
T_AirlineSettleLog
-.....
<<table>>
T_StatByBank
-<<foreign key>> dailyReportDate: Date
-<<foreign key>> bankID: String
-transferAmountWithCustomer: float
-transferAmountWithCompany: float
-totalCharge: float
-moneyAmountPayToBank: float
-<<foreign key>> bankSettleLogSerialNo: integer
<<table>>
T_BankSettleLog
-.....
<<table>>
T_StatByDestination
-<<foreign key>> dailyReportDate: Date
-destination: String
-profitDistribution: float
1
*
1
1
1
*
1
1
1
*

持久化设计的任务

❑任务

- ✓设计需要**持久保存的数据**以及这些数据之间的关系
- ✓数据组织方式（例如关系数据库中的表、关键字、外键等）之间进行**映射**
- ✓为提高数据存储、操作性能而设计持久存储机制优化设施

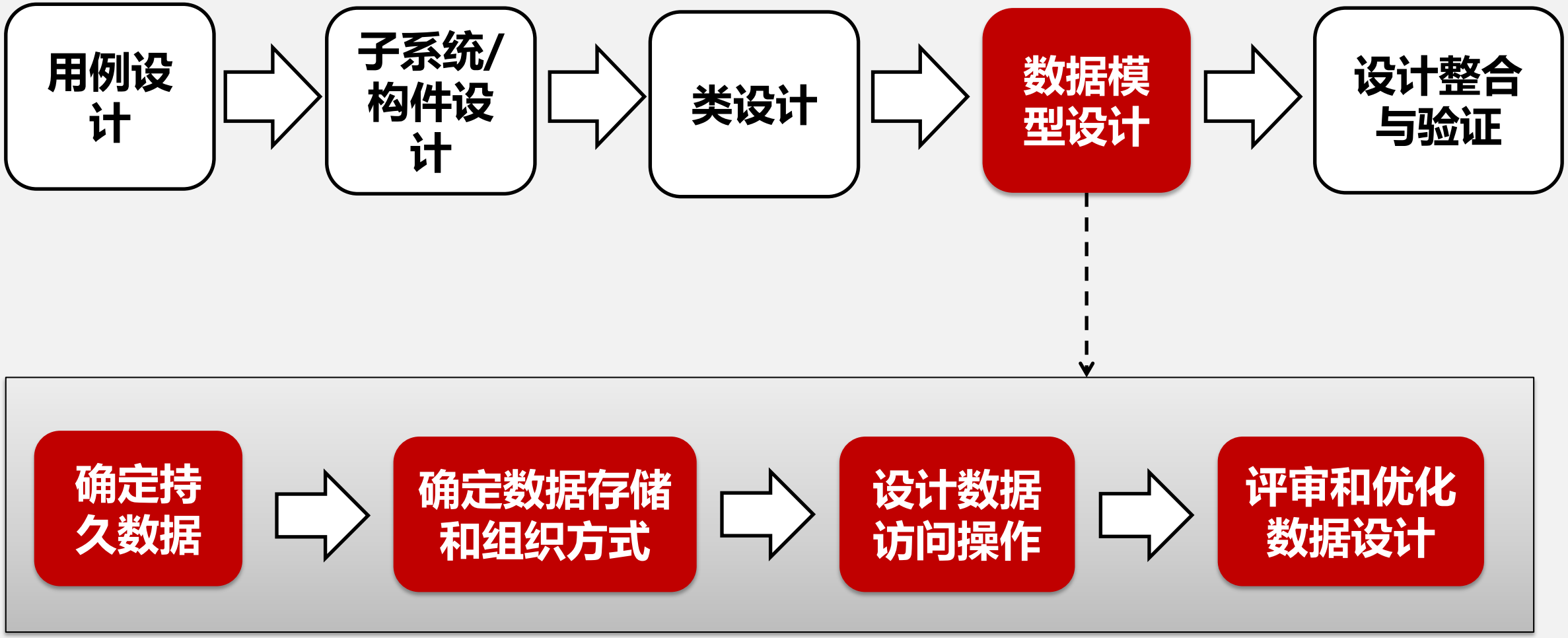
❑设计与建模

- ✓设计数据的结构、存储、组织和访问
- ✓对数据设计的结果进行建模

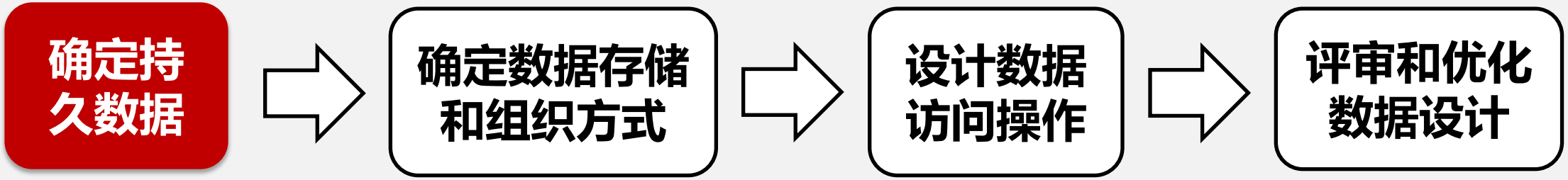
数据设计的原则

- **可追踪**
  - ✓根据软件需求、体系结构设计、用例设计等模型开展数据设计
- **无冗余**
  - ✓尽可能不要产生一些冗余、不必要的数据设计。
- **考虑和权衡时空效率**
  - ✓反复折中数据的执行效率（如操作数据需要的时间）和存储效率（如存储数据所需的空间），以满足非功能性需求
- **贯穿整个软件设计阶段**
  - ✓针对关键性、全局性的数据条目建立最初的数据模型
  - ✓数据模型应该不断丰富、演进、完善，以满足用例、子系统、构件、类等设计元素对持久数据存储的需求
- **验证数据的完整性**

数据设计过程


用例设计
子系统/构件设计
类设计
数据模型设计
设计整合与验证
确定持久数据
确定数据存储和组织方式
设计数据访问操作
评审和优化数据设计

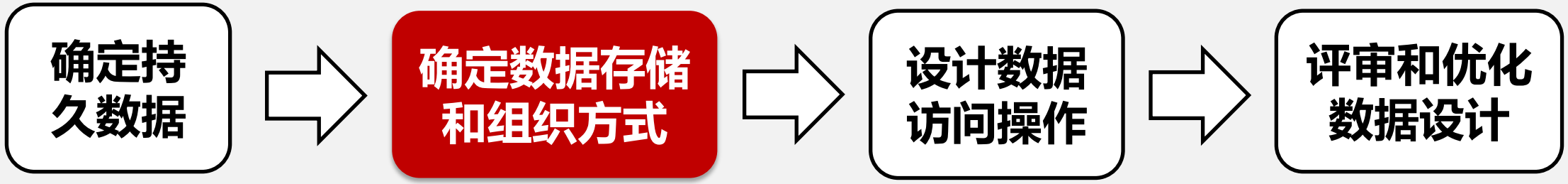
数据设计过程



确定永久数据

- **根据对需求的理解来确定哪些数据需要永久保存**
  - ✓如用户的账号和密码
  - ✓系统设置信息

数据设计过程



确定持久数据的存储和组织方式

- **将数据存储在数据文件中**
  - ✓确定数据存储的组织格式，以便将格式化和结构化的数据存放在数据文件之中
- **将数据存储在数据库中**
  - ✓设计支持数据存储的数据库表

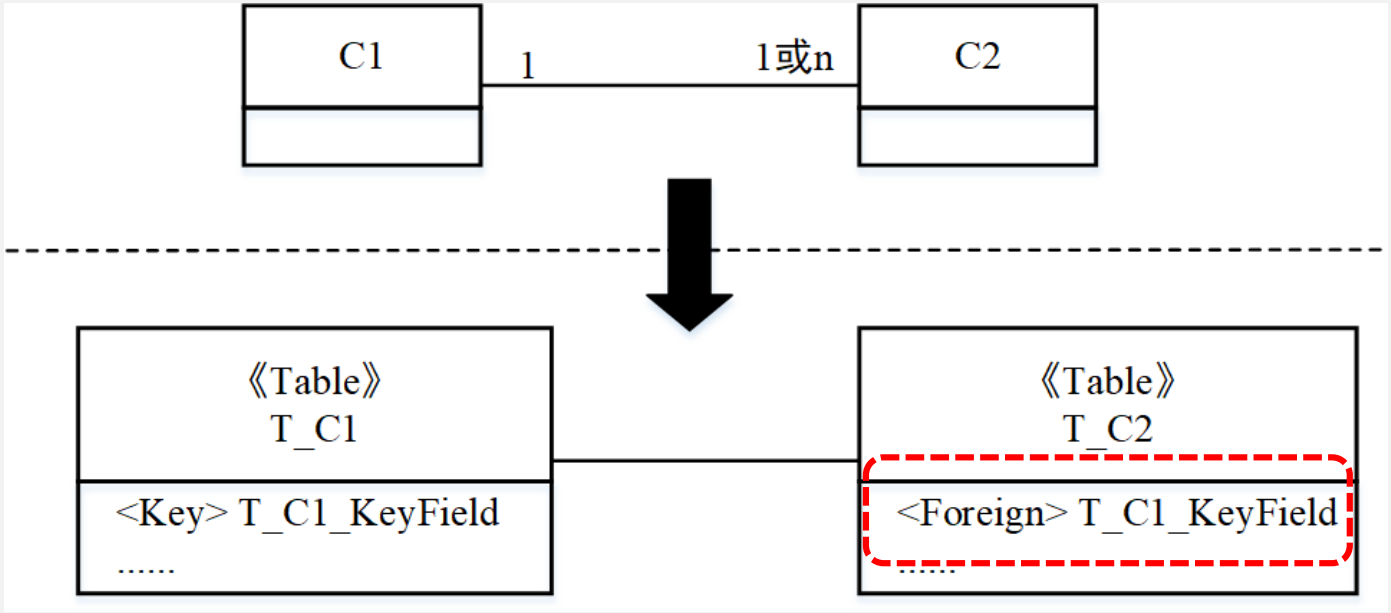
确定持久数据条目

- 确定设计模型中需要持久保存的类的对象及其属性
- 面向对象设计模型与关系数据库模型的对应关系
  - ✓类对应于“**表格**”（table）
  - ✓对象对应于“**记录**”（record）
  - ✓属性对应于表格中的“**字段**”（field）



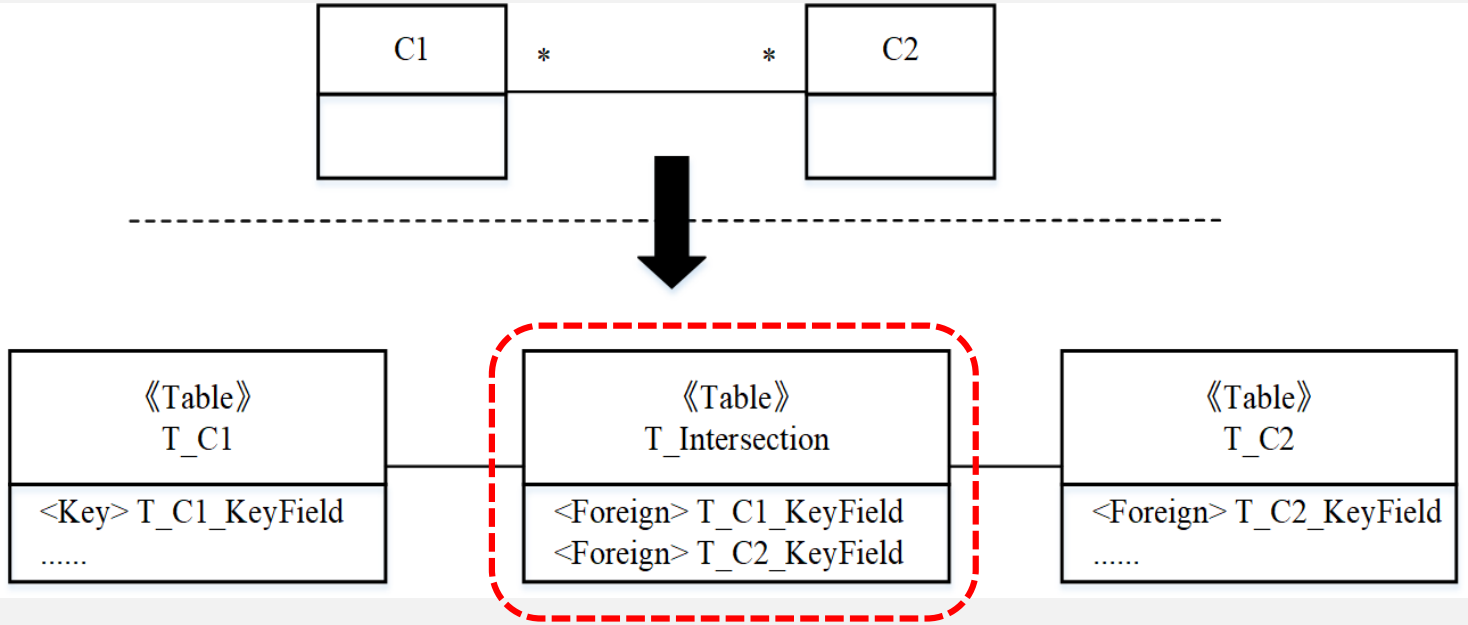
1:n关联关系的映射

- 假设类C1、C2对应的表格分别为T_C1、T_C2，只要将T_C1中关键字段纳入T_C2中作为**外键**，就可表示从T_C1到T_C2间的1:1、1:n 关联关系



n:m关联关系的映射

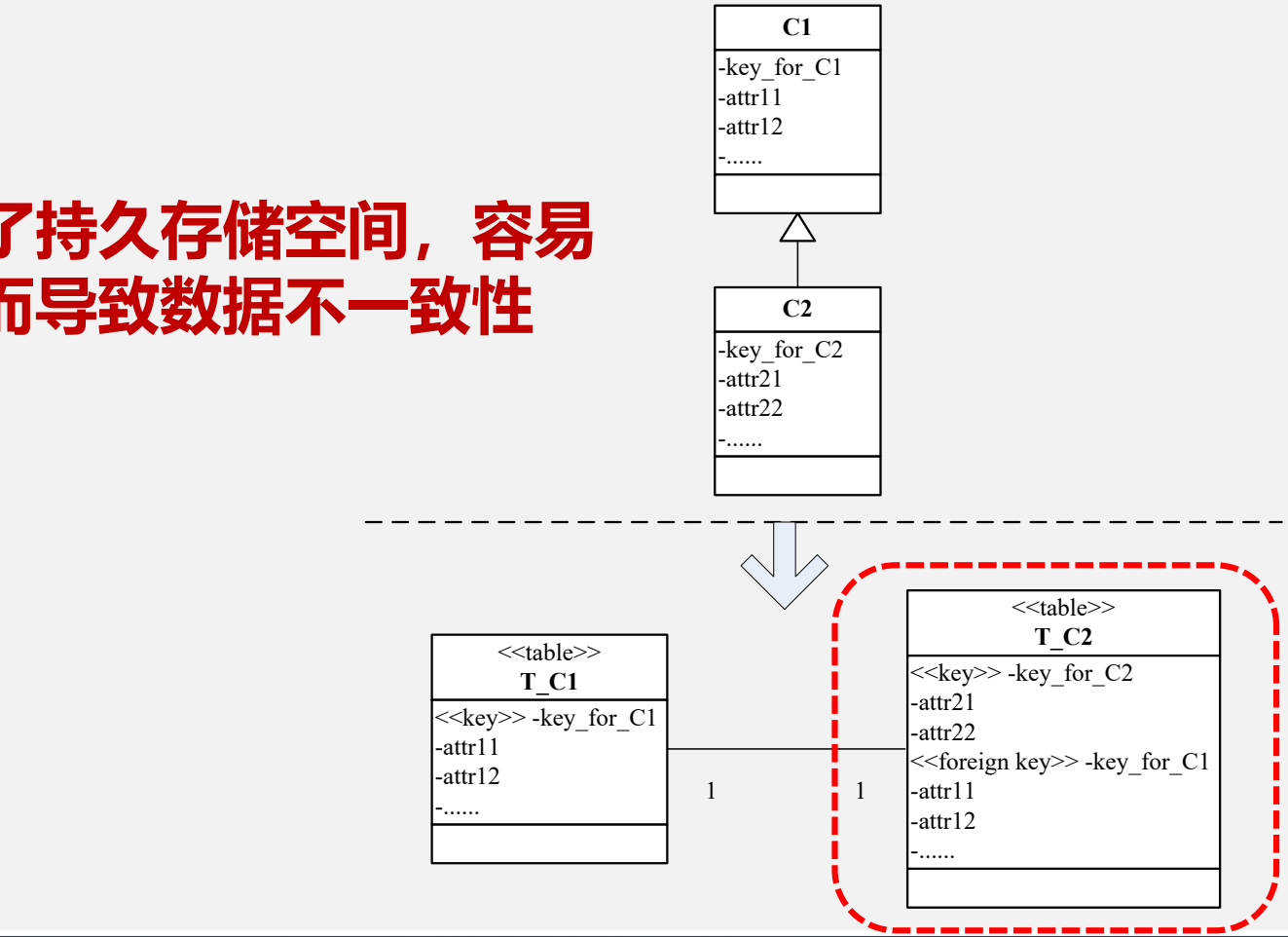
- 在T_C1、T_C2间引进新交叉表格T_Intersection，将T_C1、T_C2关键字段纳入T_Intersection中作为外键，在T_C1与T_Intersection之间、T_C2与T_Intersection之间建立一对多关系



继承关系的数据库表设计(1/2)

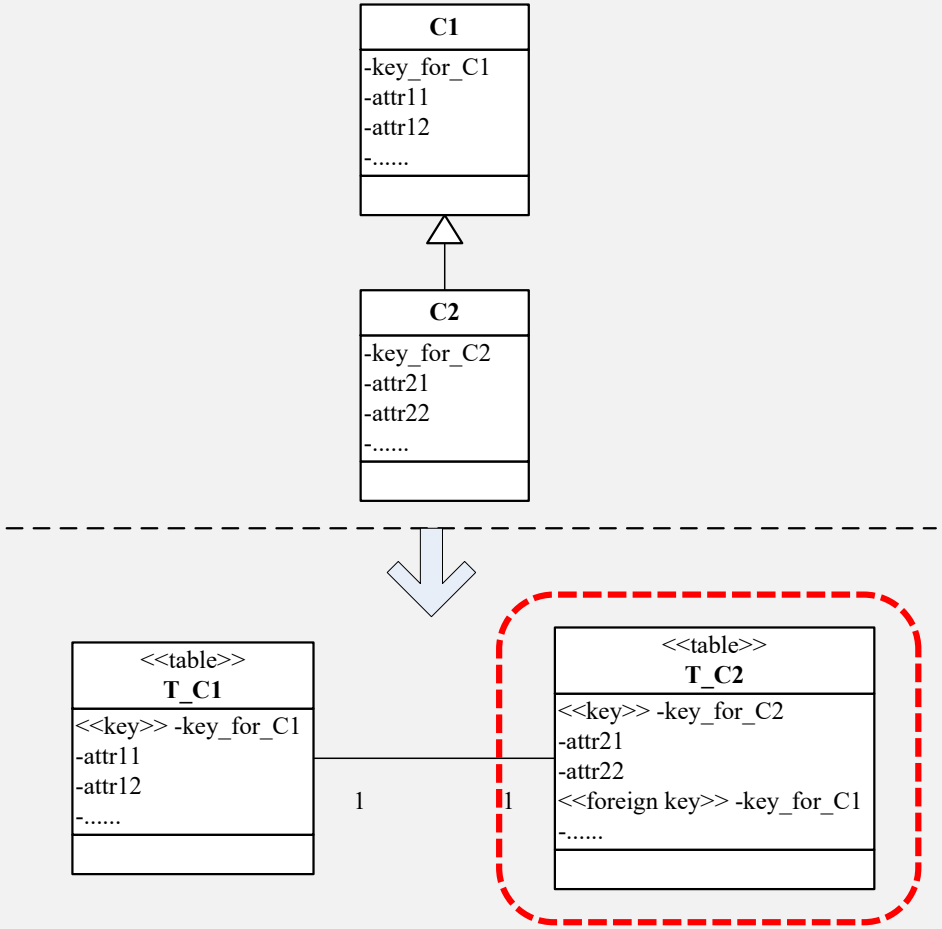
- 假设C1是C2的父类，将T_C1中的所有字段全部引入至T_C2

弊端：浪费了持久存储空间，容易因数据冗余而导致数据不一致性

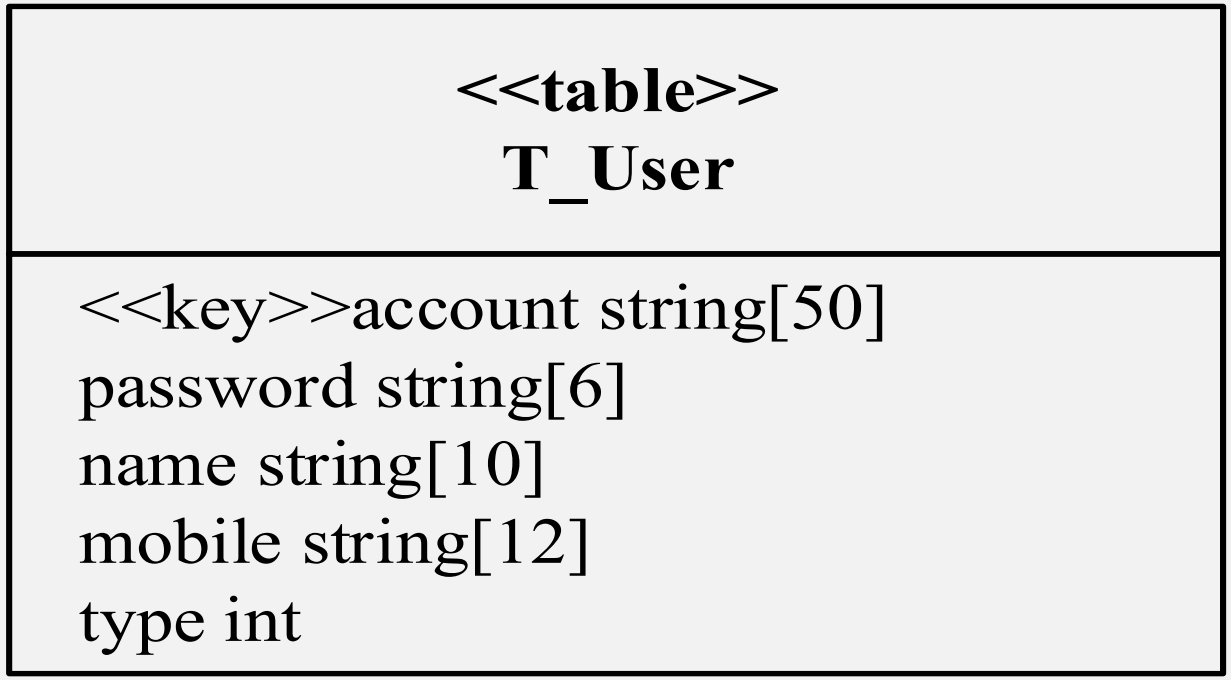


继承关系的数据库表设计(2/2)

- 假设C1是C2的父类，仅将T_C1中关键字字段纳入T_C2中作为外键
- 获取C2对象的全部属性，需要联合T_C2中的记录和对应于外键值的T_C1中的某条记录
- 避免数据冗余，但在读取C2对象时性能不如前种方法

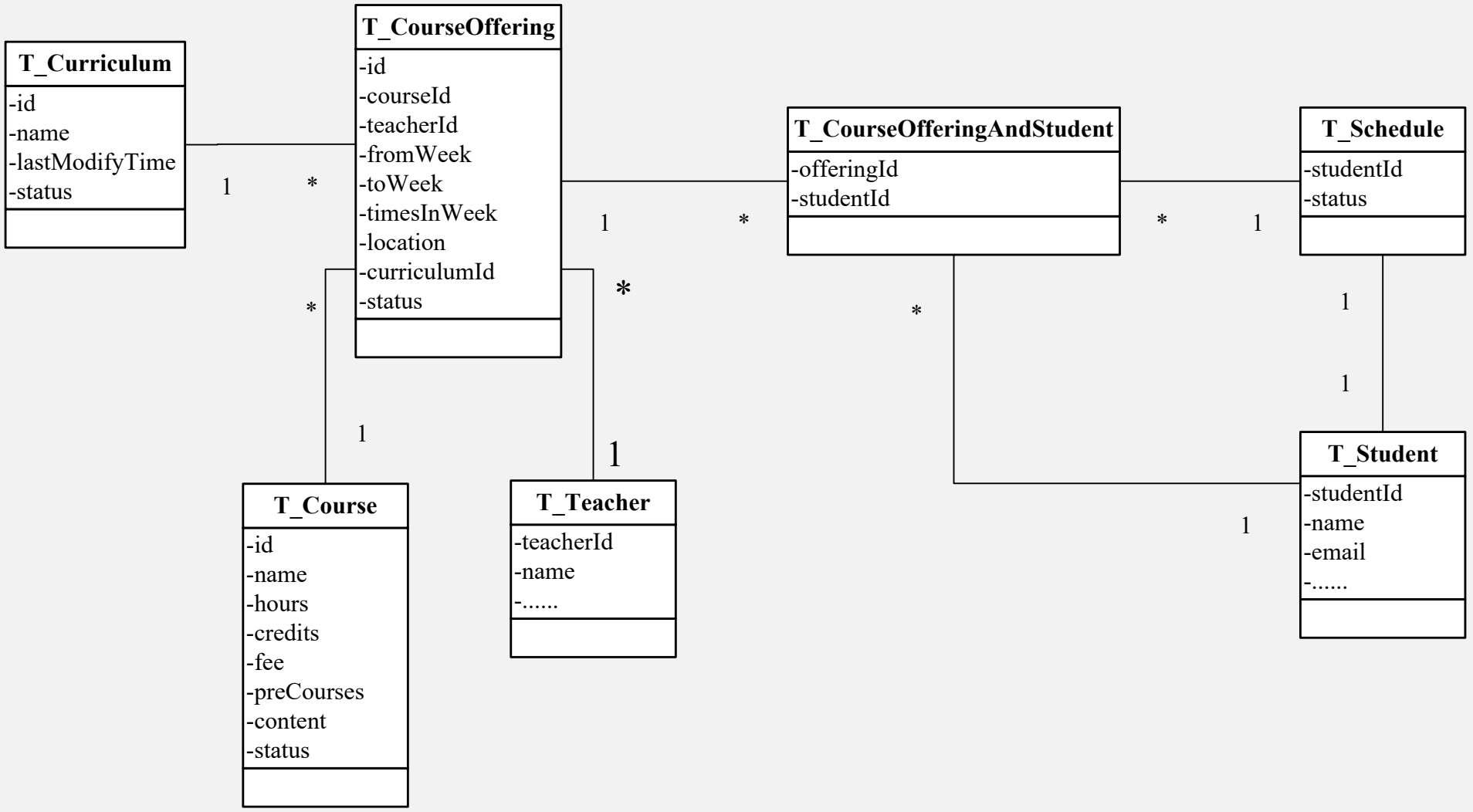


示例：设计持久数据

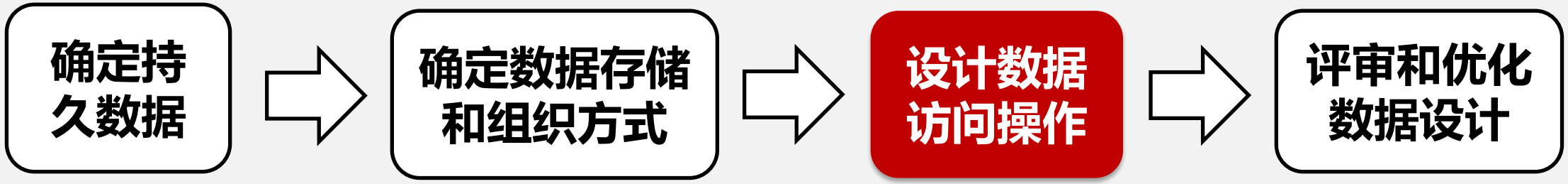


保存“User”类对象的数据库表“T_User”

示例：设计持久数据


T_Curriculum
-id
-name
-lastModifyTime
-status
T_CourseOffering
-id
-courseId
-teacherId
-fromWeek
-toWeek
-timesInWeek
-location
-curriculumId
-status
T_CourseOfferingAndStudent
-offeringId
-studentId
T_Schedule
-studentId
-status
T_Course
-id
-name
-hours
-credits
-fee
-preCourses
-content
-status
T_Teacher
-teacherId
-name
-.....
T_Student
-studentId
-name
-email
-.....
1
*

数据设计过程

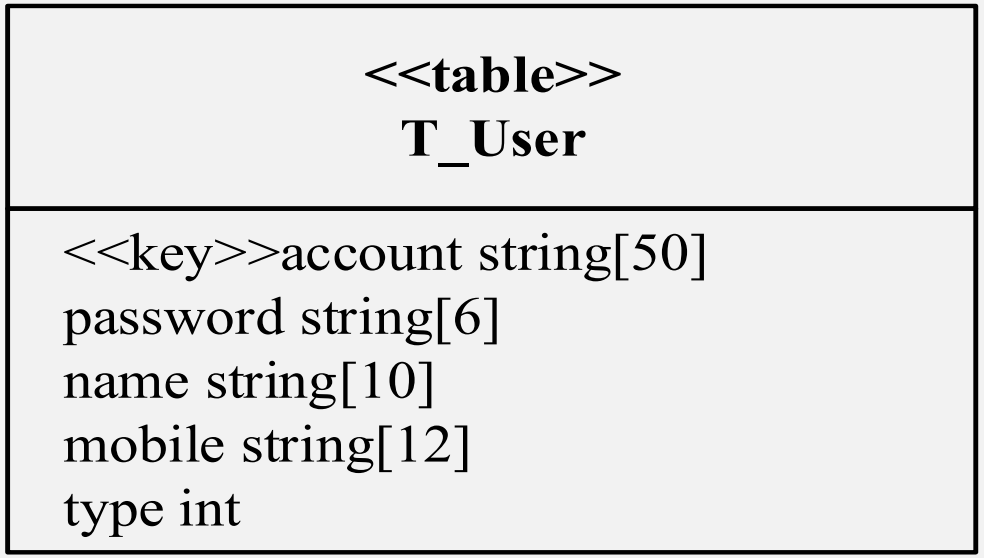


设计数据操作

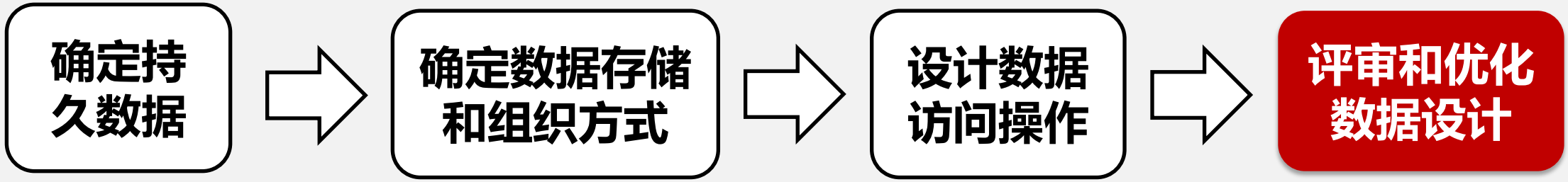
- **写入、查询、更新和删除四类基本操作以及由它们复合而成的业务数据操作**
  - ✓**写入操作**将数据从运行时的软件系统保存至数据库
  - ✓**查询操作**按照特定的选择准则从数据库提取部分数据置入运行时软件系统中的指定对象
  - ✓**更新操作**以运行时软件系统中的（新）数据替换数据库中符合特定准则的（旧）数据
  - ✓**删除操作**将符合特定准则的数据从数据库中删除
- **数据验证操作该操作**
  - ✓**验证操作**负责验证数据的完整性、相关性、一致性等等

示例：设计永久数据的操作

- boolean insertUser(User)
- boolean deleteUser(User)
- boolean updateUser(User)
- User getUserByAccount(account)
- boolean verifyUserValidity(account, password)
- UserLibrary()
- ~UserLibrary()
- void openDatabase()
- void closeDatabase()



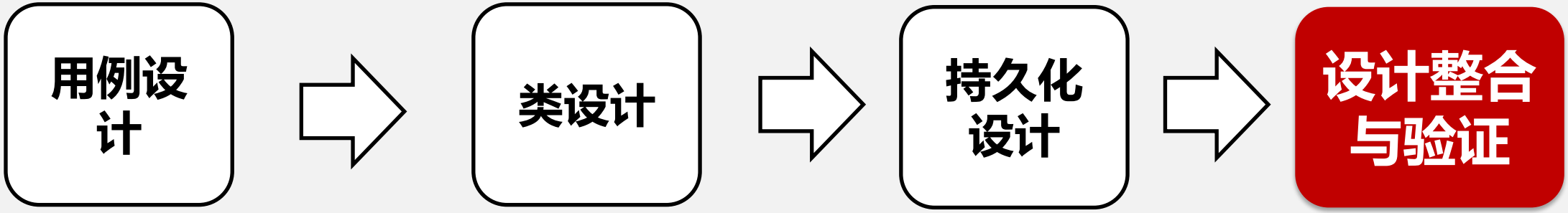
数据设计过程



评审和优化数据设计

- **正确性**
  - ✓数据设计是否满足软件需求
- **一致性**
  - ✓数据设计尤其是是数据的组织是否与相关的类设计相一致
- **时空效率**
  - ✓分析数据设计的空间利用率，以此来优化数据的组织
  - ✓根据数据操作的响应时间来分析数据操作的时效性，优化数据库以及数据访问操作
- **可扩展性**
  - ✓数据设计是否考虑和支持将来的数据持续保存的可能扩展

详细设计的过程（4/4）



软件详细设计的输出

❑模型

- ✓用UML类图、构件图、包图、状态图、顺序图等描述的详细设计模型

❑文档

- ✓软件详细设计规格说明书

设计整合

- **汇总迄今获得的所有设计模型**
  - ✓包括体系结构模型、界面设计模型、用例设计模型、子系统/构件/类设计模型、数据模型等
- **形成系统、完整的软件设计方案**

设计验证

- 验证整个设计的正确性、优化性和充分性等
- 验证设计模型之间的不一致性、冗余性等
- 发现设计方案中的问题并进行整改

北京航空航天大學
BEIHANG UNIVERSITY



Part 3

面向过程的详细设计

Part 3 面向过程的详细设计

- 详细设计概述
- 程序流程图（FlowChart)
- 盒图（N-S)
- 伪代码（PDL)
- 问题分析图（PAD)
- Jackson方法

Part 3 面向过程的详细设计

- 详细设计概述
- 程序流程图（FlowChart)
- 盒图（N-S)
- 伪代码（PDL)
- 问题分析图（PAD)
- Jackson方法

面向过程详细设计

- **软件详细设计**，又称为**软件实现设计**
- 若在**概要设计**中**将软件系统划分为各个不同的模块**，则**详细设计**就是各个**模块的实现设计**。
- 面向过程详细设计，实质上就是**面向算法分析设计**。即使是采用其他的设计方法，如面向对象设计方法，其方法本身的实现仍然是面向过程的。因此，**面向过程详细设计，是其他各种详细设计的基础。**

面向过程详细设计

- 面向过程程序设计技术，采用**自顶向下逐步求精**的设计方法和**单入口单出口**的控制结构，并且只包含**顺序、选择和循环**三种结构，设计的目标之一是**使程序的控制流程线性化**，即程序的动态执行顺序符合静态书写结构。
- 详细设计的**任务**是给出软件模块结构中**各个模块的内部过程描述**，也就是模块内部的算法设计。
- 详细设计的工具可以分为图形、表格、语言三种，包括**程序流程图、盒图（N-S图）、程序设计语言PDL、PAD图。**

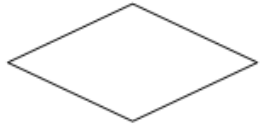
Part 3 面向过程的详细设计

- 详细设计概述
- 程序流程图（FlowChart)
- 盒图（N-S)
- 伪代码（PDL)
- 问题分析图（PAD)
- Jackson方法

程序流程图 (Flow Chart)

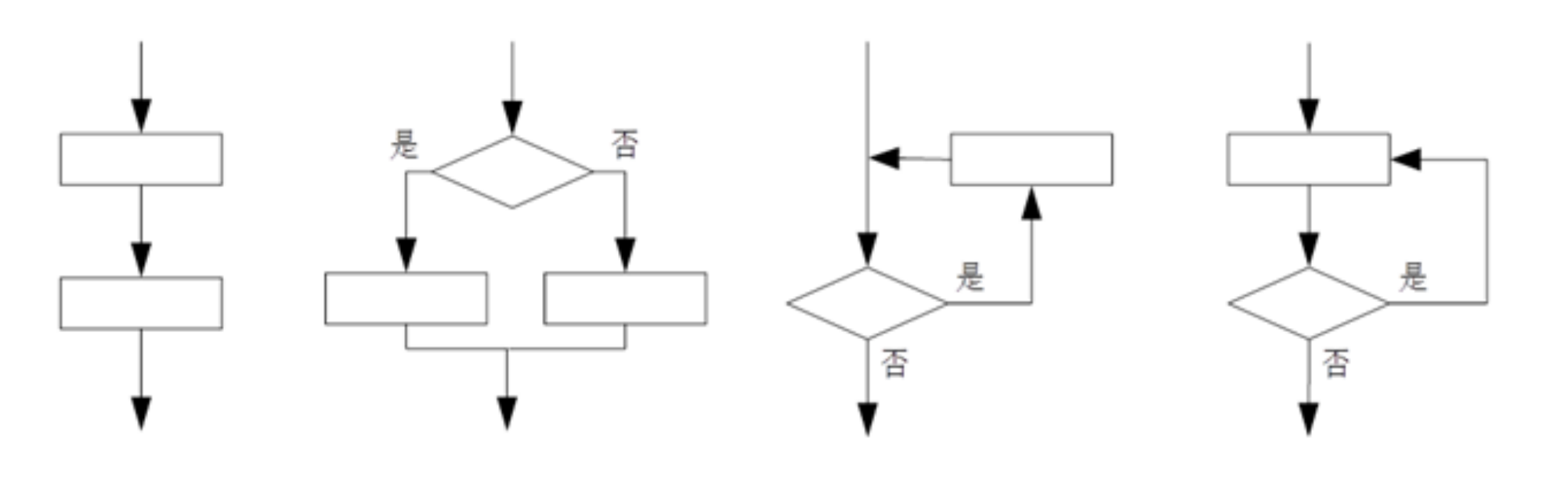
- **程序流程图**是用图形化的方式，表示程序中**一系列的操作**以及**执行的顺序。**
- 程序流程图中使用的**主要符号**包括**顺序、选择、循环结构**，它的**主要缺点**如下，程序流程图本质上不是逐步求精的好工具，它诱使程序员**过早地考虑程序的控制流程**，而不去考虑程序的全局结构，程序流程图中用箭头代表控制流，因此程序员不受任何约束，可以完全不顾结构程序设计的精神，随意转移控制，程序流程图不易表示数据结构。

程序流程图(Flowchart)

名称	图例	说明
终结符		表示流程的开始和结束
处理		表示程序的计算步骤或处理过程， 在方框内填写处理的名称或程序语句
判断		表示逻辑判断或分支， 用于决定执行后续的路径， 在菱形框内填写判断的条件
输入/输出		获取待处理的信息（输入）， 记录或显示已处理的信息（输出）
连线		连接其他的符号，表示执行顺序或数据流向

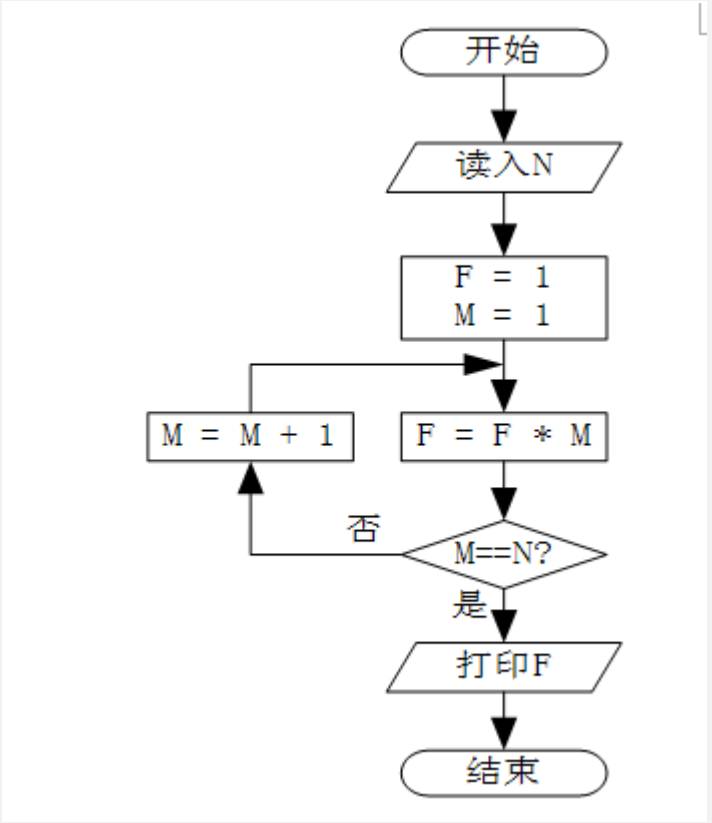
程序流程图(Flowchart)

常见的程序流程图结构有以下几种



程序流程图 (Flow Chart)

【例】 使用程序流程图，描述并打印N的阶乘，如下图所示。

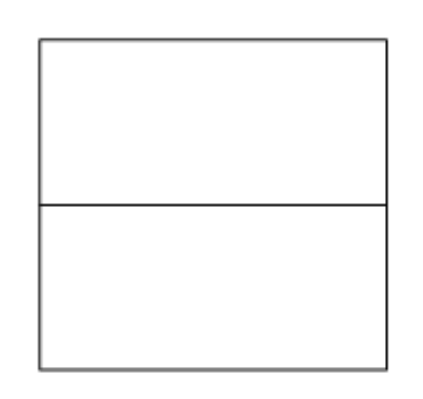


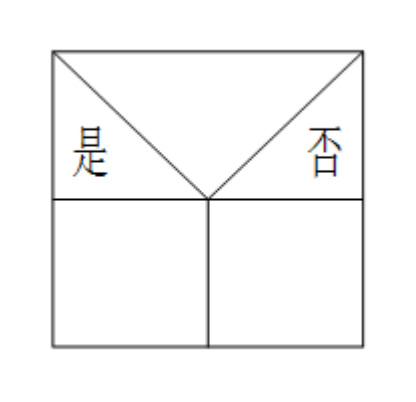
Part 3 面向过程的详细设计

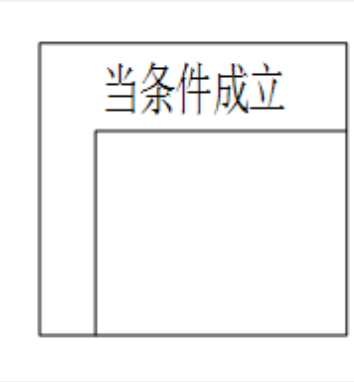
- 详细设计概述
- 程序流程图（FlowChart)
- 盒图（N-S)
- 伪代码（PDL)
- 问题分析图（PAD)
- Jackson方法

N-S图（Nassi-Schneiderman Diagram）

- Nassi-Schneiderman图是流程图的另一种表达形式，由Nassi和Schneiderman提出，简称N-S图。与流程图对应如下图所示：



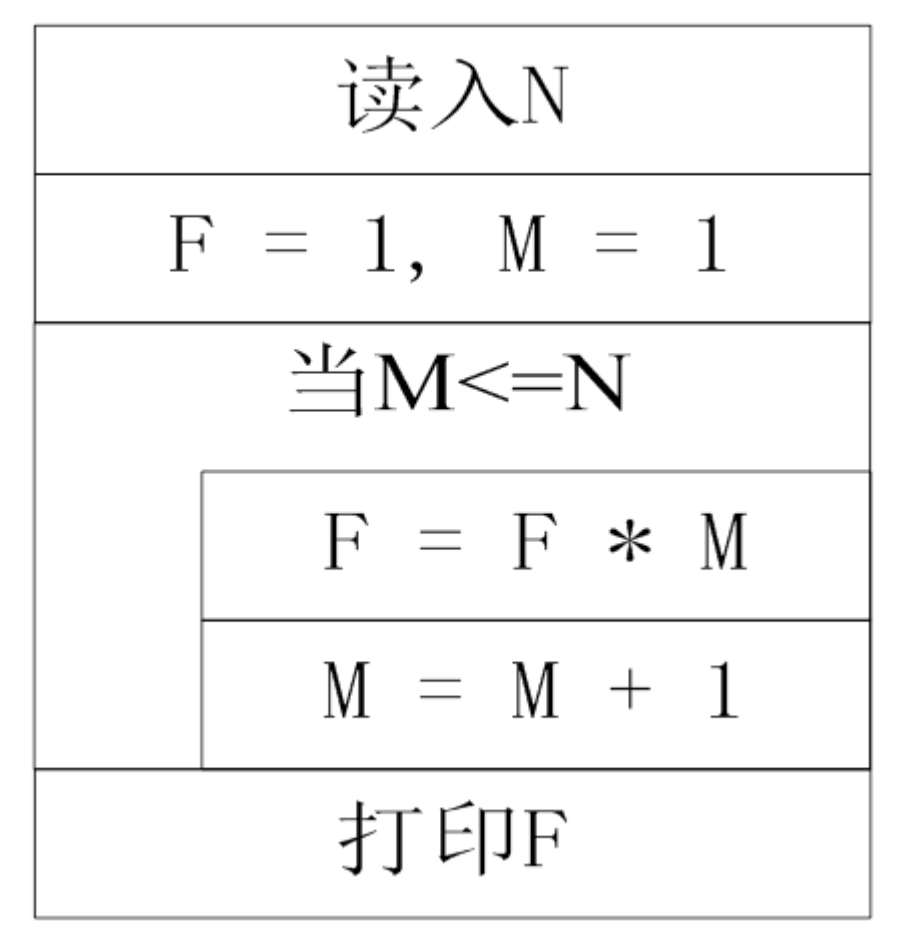






N-S图（Nassi-Schneiderman Diagram）

- 【例】 使用N-S图，描述并打印N的阶乘，如下图所示。



Part 3 面向过程的详细设计

- 详细设计概述
- 程序流程图（FlowChart)
- 盒图（N-S)
- 伪代码（PDL)
- 问题分析图（PAD)
- Jackson方法

程序设计语言PDL（Program Design Language）

- **程序设计语言PDL**也称为**伪码**，它是用正文形式表示数据结构和处理过程的设计工具。
- PDL具有以下**特点**：关键字的固定语法，提供了结构化控制结构、数据说明和模块化的手段，**自然语言的自由语法**，用于描述处理过程和判定条件，数据说明的手段，既包括简单的数据结构，又包括复杂的数据结构、模块定义和调用的技术，提供各种接口描述模式。
- 程序设计语言又称**结构化英语**，它使用**结构化编程语言的风格**描述程序算法，但**不遵循特定编程语言的语法**。

程序设计语言PDL（Program Design Language）

【例】 使用程序设计语言描述打印N的阶乘。

- 读入N
- 置F的值为1，置M的值为1
- 当M <= N时，执行:
- 使F = F * M
- 使 M = M + 1
- 打印F

决策表（Decision Table）

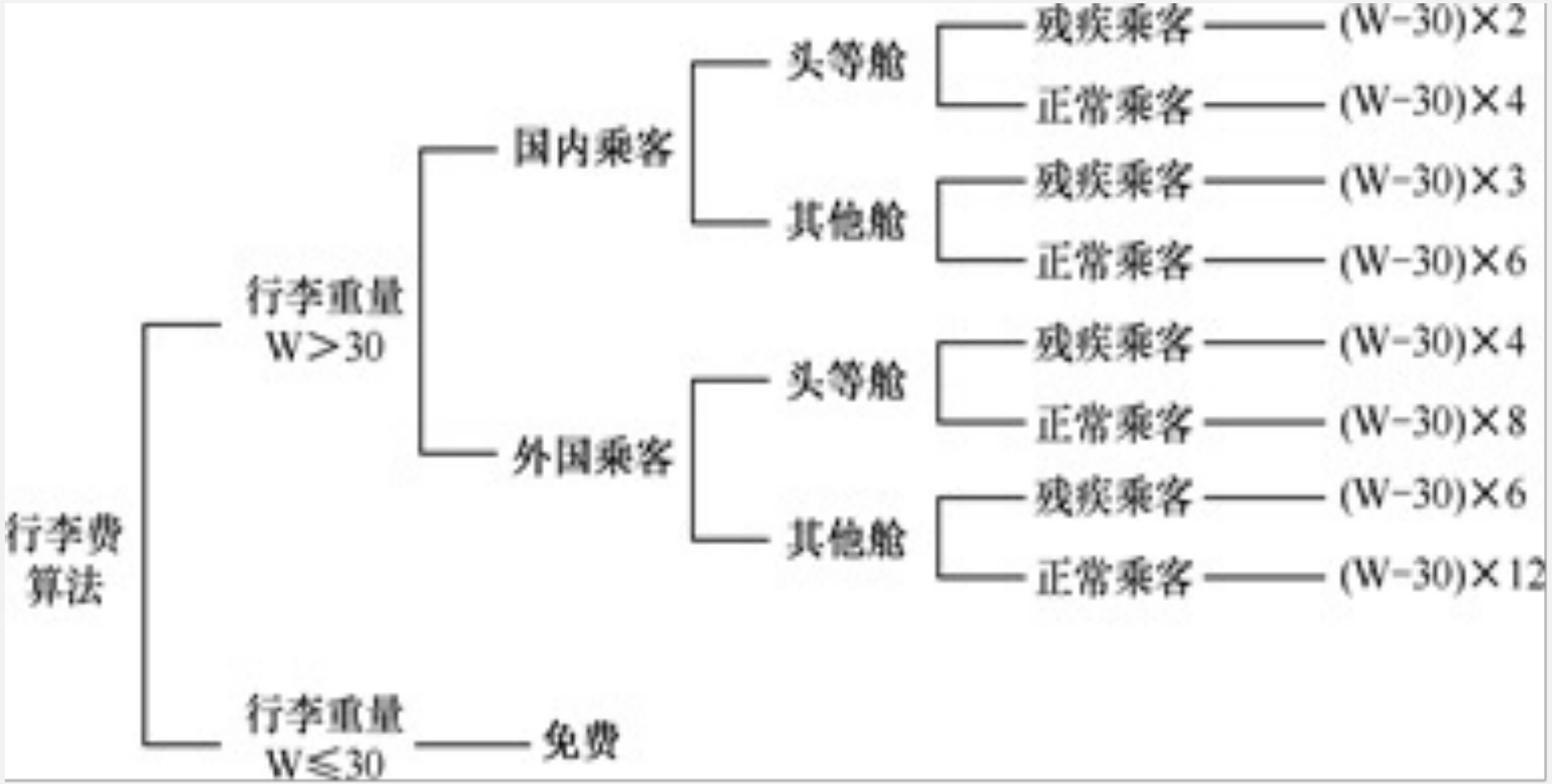
- 决策表（判定表）用于以紧凑的形式描述复杂的逻辑。决策表与if-else、switch-case语句类似，将条件判断与执行的动作联系起来。
- 决策表通常分成四个区域。

决策表（Decision Table）

【例】 **条件对应于一个变量、关系或预测**，其可能的组合在条件选择中列出。**动作是一个函数或操作。**动作选择描述**当条件满足时所执行的动作**，如下图所示。

条件	不能打印	√	√	√	√			
	红灯闪	√	√			√	√	
	不能识别打印机	√		√		√		√
动作	检查电源线			√				
	检查打印机数据线	√		√				
	检查是否安装驱动程序	√		√		√		√
	检查墨盒	√	√			√	√	
	检查是否卡纸		√		√			

判定树


行李费算法
行李重量 W＞30
国内乘客
头等舱
残疾乘客
(W-30)×2
正常乘客
(W-30)×4
其他舱
残疾乘客
(W-30)×3
正常乘客
(W-30)×6
外国乘客
头等舱
残疾乘客
(W-30)×4
正常乘客
(W-30)×8
其他舱
残疾乘客
(W-30)×6
正常乘客
(W-30)×12
行李重量 W≤30
免费

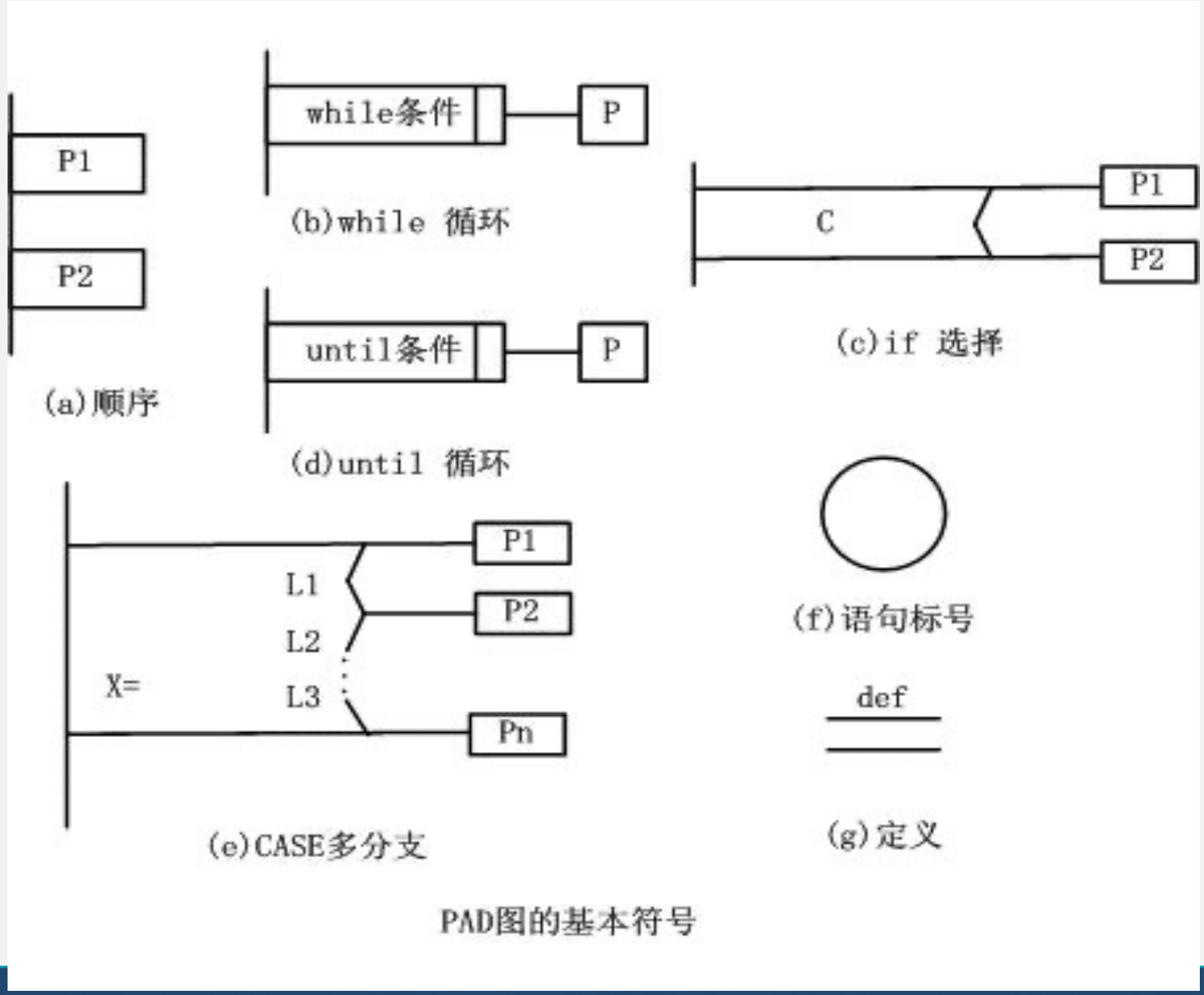
Part 3 面向过程的详细设计

- 详细设计概述
- 程序流程图（FlowChart)
- 盒图（N-S)
- 伪代码（PDL)
- 问题分析图（PAD)
- Jackson方法

PAD (Problem Analysis Diagram)

- 自1973年由日本日立公司发明PAD图形以来，已经在日本国内及中国的软件外包公司得到一定程度的推广。PAD用**二维数形结构**的图**表示程序的控制流**，将这种图转换为程序代码比较容易。
- PAD图形具有如下**优点**：
- (1). 使用**结构优化**控制结构的PAD符号，所设计出来的程序必然是程序化程序。
- (2). PAD图中**最左边的竖线是程序的主线**，即**第一层控制结构**。随着程序层次的增加，PAD图逐层向右延伸，**每增加一个层次，图形向右扩展一条竖线**。PAD图中竖线的**总条数就是程序的层次数**。

PAD (Problem Analysis Diagram)

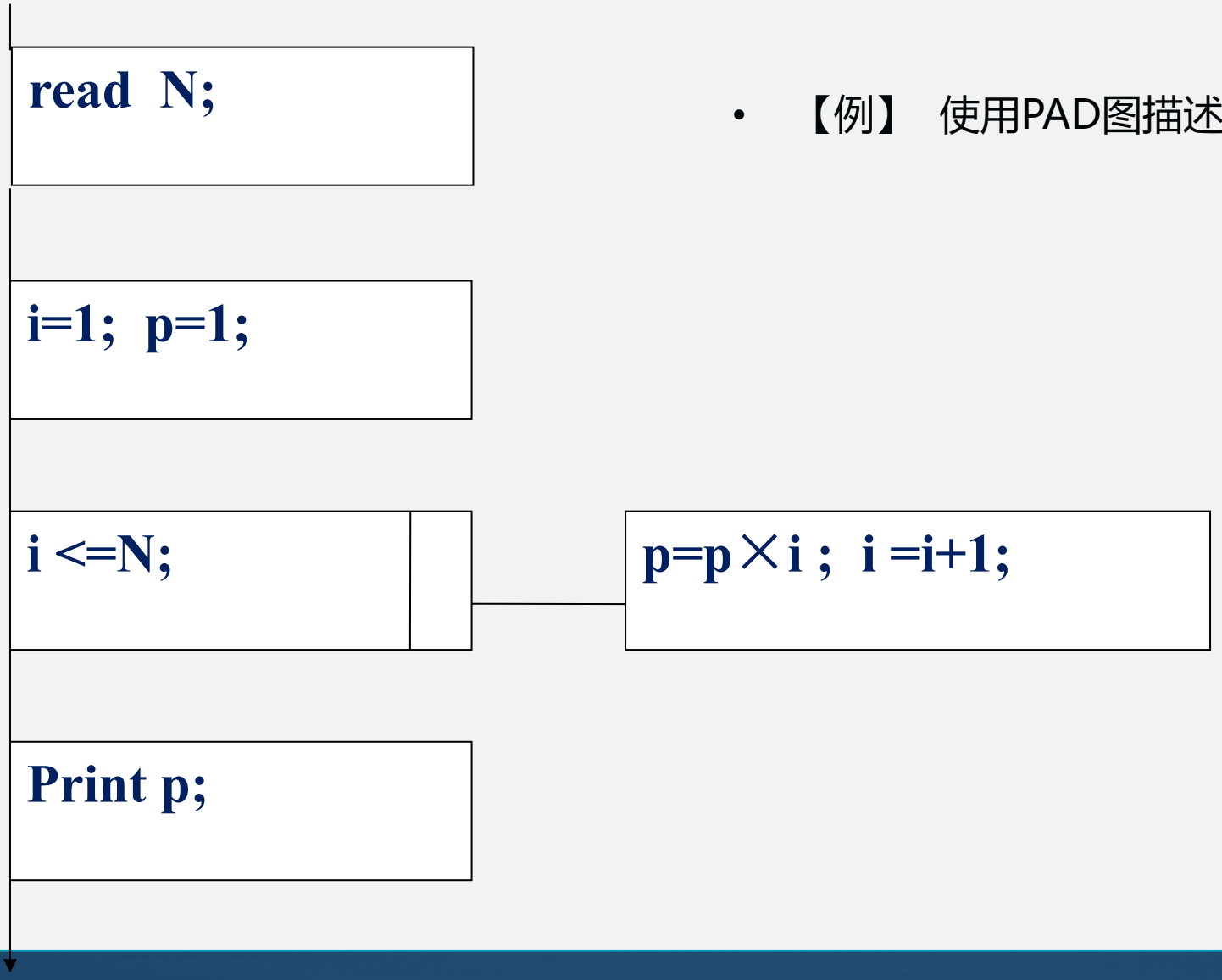


PAD图的基本符号

PAD (Problem Analysis Diagram)

- (3). 用PAD图表现程序逻辑，**易读、易懂、易记**。PAD图是二维数型结构的图形，程序从图中**最左边上端的结点**开始执行，**自上而下，从左到右顺序执行**。
- (4). 很容易将PAD图转换成高级程序**语言源程序**，有利于提高软件可靠性和软件生产率。
- (5). 既可用于**表示程序逻辑**，也可用于**描述数据结构**。
- (6). PAD图的符号**支持自顶向下、逐步求精方法**的使用。
- 由此可见，PAD是一种程序结构**可见性好、结构唯一、易于编制、易于检查和易于修改**的详细设计表现方法。

PAD (Problem Analysis Diagram)



- 【例】 使用PAD图描述打印N的阶乘，如下图所示。

Part 3 面向过程的详细设计

- 详细设计概述
- 程序流程图（FlowChart)
- 盒图（N-S)
- 伪代码（PDL)
- 问题分析图（PAD)
- Jackson方法

面向数据结构的设计方法

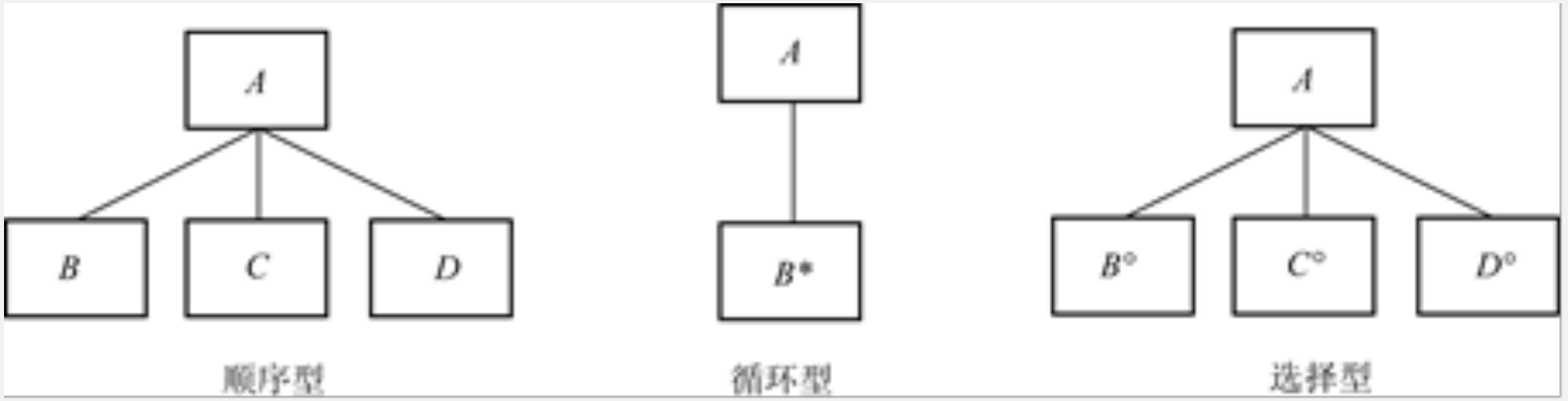
- **面向数据结构的设计方法**就是根据**数据结构**设计程序**处理过程的方法**，具体地说，面向数据结构的设计方法**按输入、输出**以及计算机内部**存储信息的数据结构**进行软件结构设计，从而把对数据结构的描述转换为对软件结构的描述。使用面向数据结构的设计方法时，**分析目标系统的数据结构**是**关键**。
- 面向数据结构的设计方法通常在**详细设计阶段**使用。比较流行的面向数据结构的设计方法包括Jackson方法和Warnier方法。在这里，主要介绍Jackson方法。

面向数据结构的设计方法

- **Jackson方法**把数据结构分为3种基本类型：**顺序型结构**、**选择型结构**和**循环型结构**。
- 它的**基本思想**是：在充分理解问题的基础上，找出**输入数据**、**输出数据**的**层次结构的对应关系**，将数据结构的层次关系映射为软件控制层次结构，然后对问题的细节进行过程性描述。

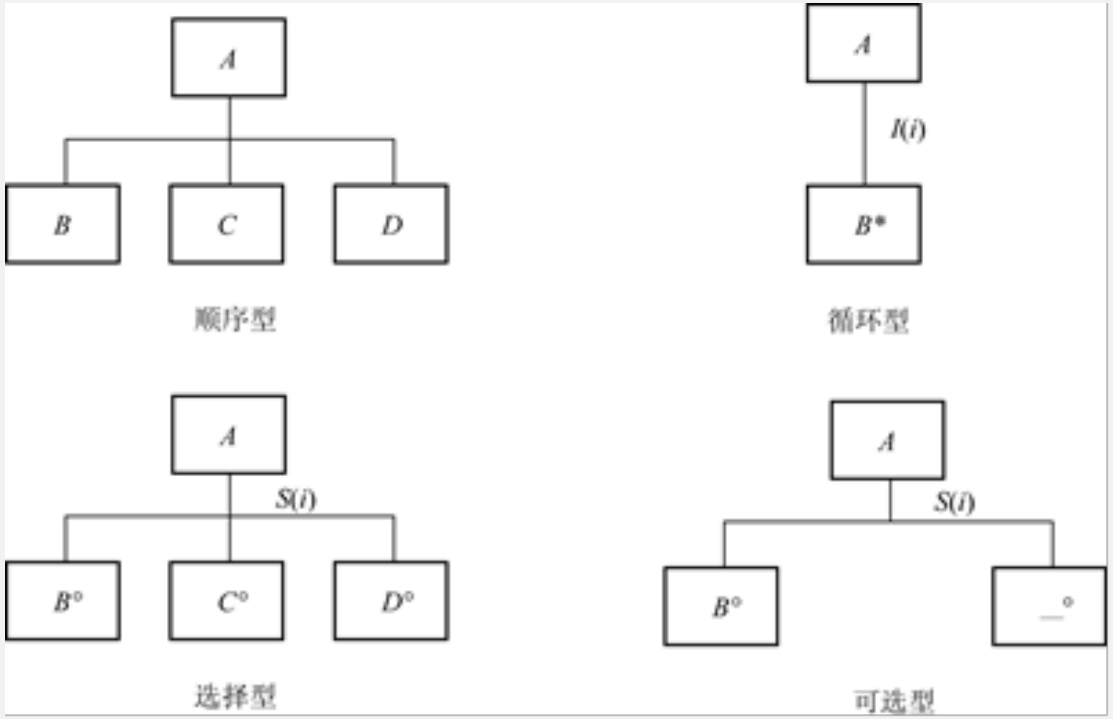
面向数据结构的设计方法

Jackson图是Jackson方法的描述工具，Jackson方法的基本逻辑结构如下图所示：



面向数据结构的设计方法

- 运用**Jackson图**表达选择型或循环型结构时，**选择条件或循环结束条件不能在图上直接**表现出来，并且框间连线为斜线，不易在打印机上输出，所以产生了改进的Jackson图，其基本逻辑符号如下图所示。



面向数据结构的设计方法

运用Jackson图进行程序设计的**优点**是：

- 1）可以**清晰地表示层次结构**，易于对自顶向下的结构进行描述。
- 2）结构**易懂、易用**，并且比较直观、形象。
- 3）不仅可以表示**数据结构**，也可以表示**程序结构**。

面向数据结构的设计方法

运用Jackson方法进行程序设计的步骤可以归纳为以下几点：

- (1) 分析并**确定输入数据和输出数据**的**逻辑结构**，并用 **Jackson结构**来表示这些**数据结构**。
- (2) 找出输入数据结构和输出数据结构中有对应关系的**数据单元**。
- (3) 按照一定的规则，从**描绘数据结构的Jackson图**导出**描绘程序结构的Jackson图**。
- (4) 列出**基本操作与条件**，并把它们分配到程序结构图的适当位置。
- (5) 用伪代码表示程序。

Jackson方法解决问题的例子

零件库房管理：

- 有“**零件表**”，用于记录**零件信息**，如：零件编号、名称、规格，其中的零件编号是零件唯一标识；
- 有“**零件进库表**”，用于记录**零件进库信息**，如：零件编号、数量。

现需要**按零件编号**对**零件进库情况进行汇总**，要求使用Jackson方法设计该问题算法。

利用Jackson方法设计该问题算法的步骤如下：

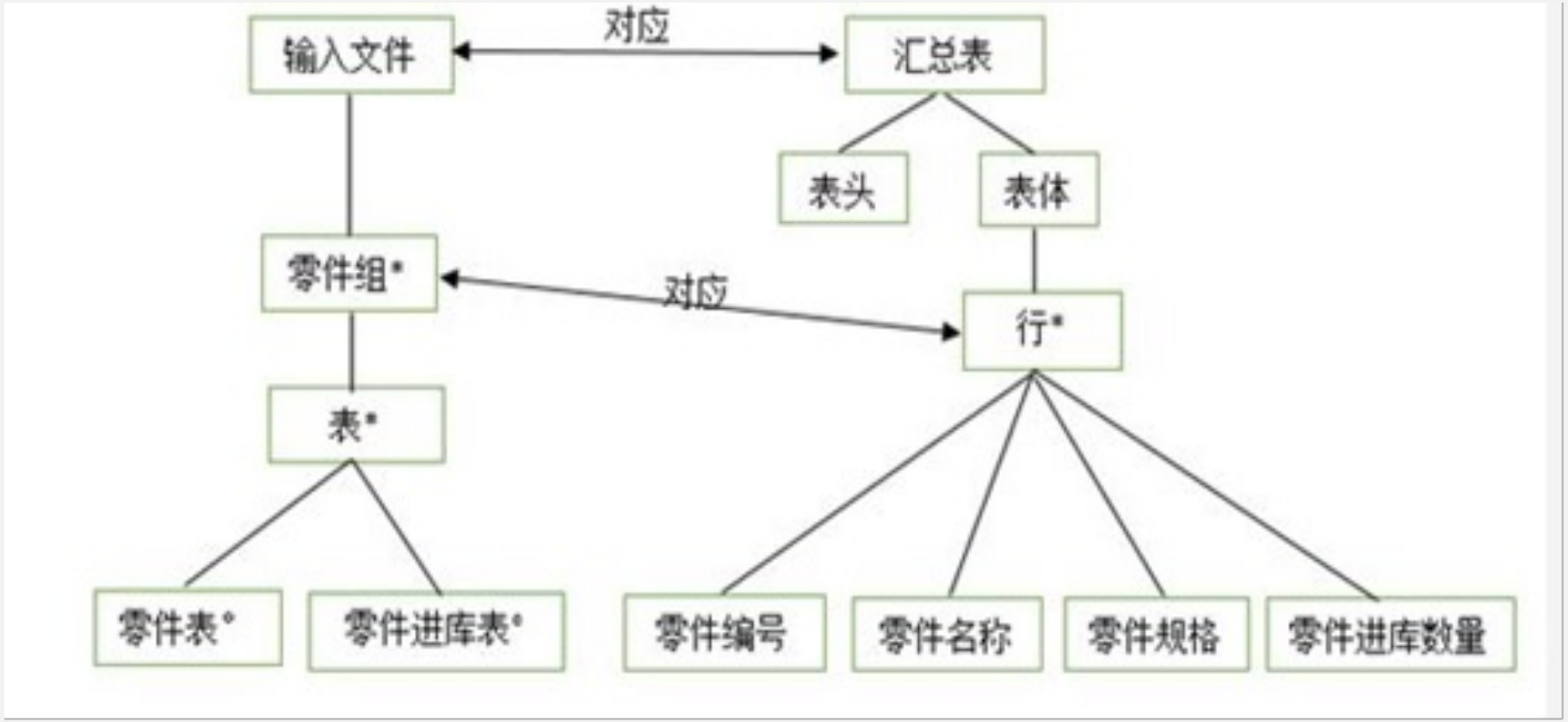
分析并确定**输入数据和输出数据的逻辑结构**，并用Jackson结构来表示这些数据结构：

1）**输入数据**：根据问题陈述，有若干通过编号来区分的不同零件，每种零件的信息由两张表来决定，把同一种零件的表放在一起组成一组，所有的组按照零件编号排序。所以输入数据是**由许多零件组组成的文件**，**每个零件组有多个表**，**每个表**记录着本零件的**基本信息和进库信息**。因此输入数据结构的Jackson图如下图的（a）所示。

2）**输出数据**：根据问题陈述，输出数据是一张如下图所示的汇总表，它**由表头和表体**两部分组成，表体中**有许多行**，一个零件的**基本信息占一行**，其输出数据结构的Jackson图如下图的（b）所示。

面向数据结构的设计方法

- （a）输入数据结构　　（b）输出数据结构



面向数据结构的设计方法

进库汇总表			
零件编号	零件名称	零件规格	零件进库数量
P1	螺丝	xxx	300
P2	钉子	yyy	150
….	….	….	….

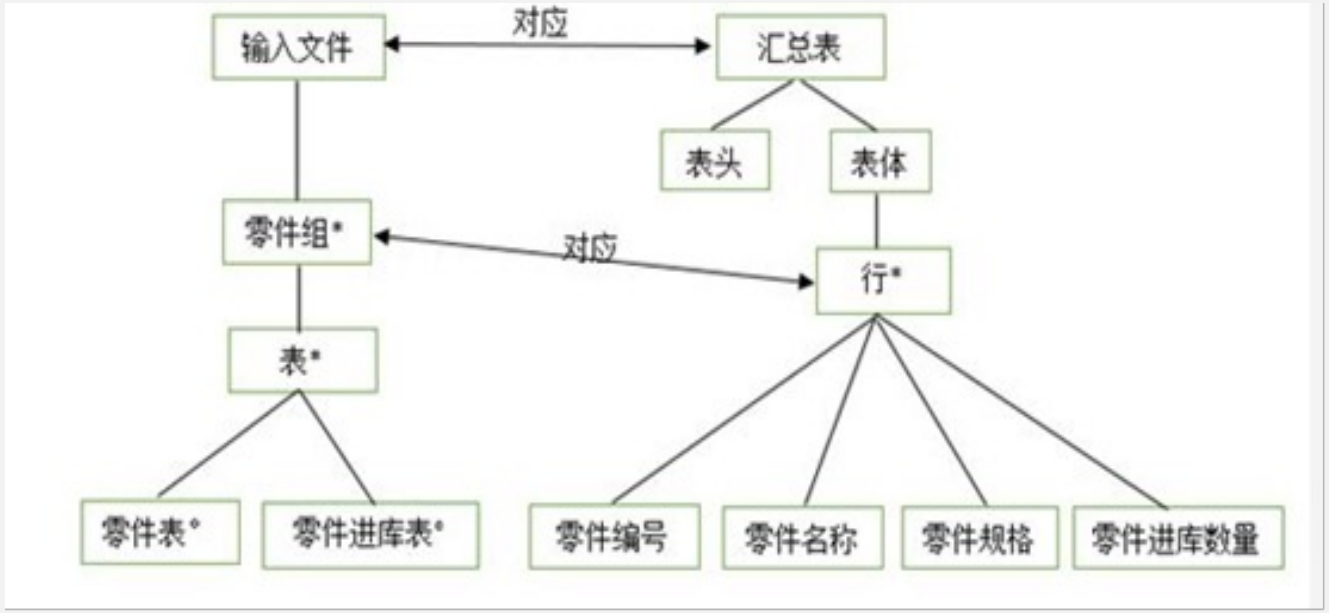
面向数据结构的设计方法

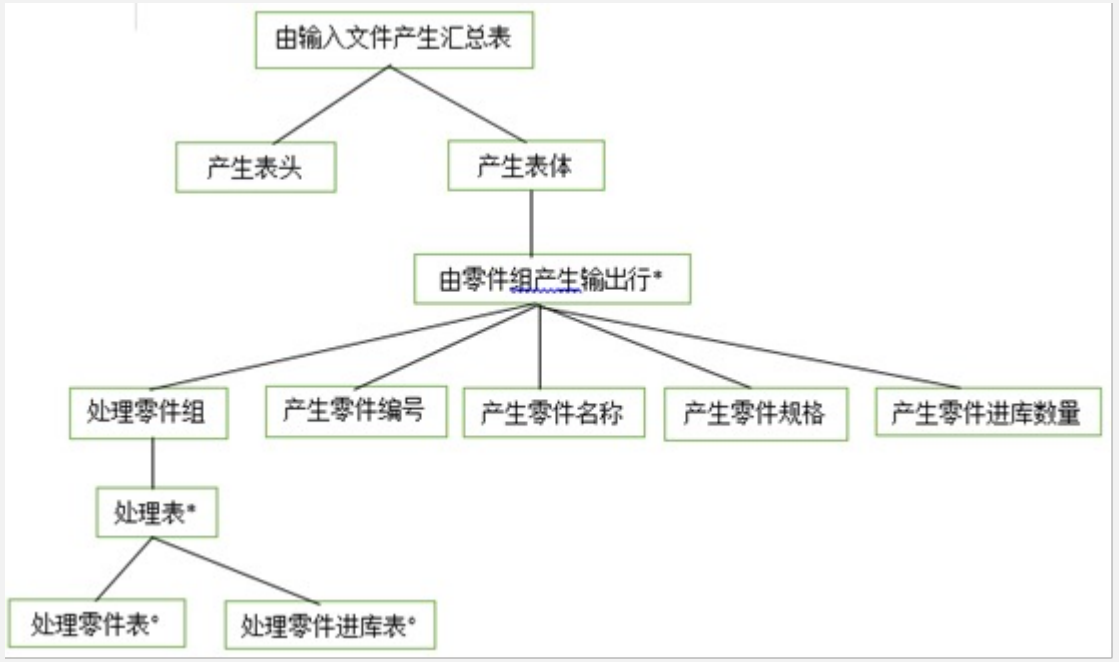
- 找出输入、输出数据结构中有对应关系的数据单元；
- **汇总表由输入文件产生**，有直接的因果关系，因此顶层的数据单元是对应的。表体的**每一行数据**由输入文件的**每一个“零件组”计算而来**，行数与组数相同，且行的排列次序与组的排列次序一致，都按零件号排序。因此“零件组”与“行”两个单元对应，以下再无对应的单元。如上图所示。

从描绘数据结构Jackson图导出描绘程序结构的Jackson图

- 找出对应关系后，根据**以下规则导出程序结构**：对于输入数据结构与输出数据结构中的**数据单元**，**每对有对应关系的数据**按照它们所在的层次，在程序结构图适当位置合**画一个处理框**，无对应关系的数据单元，各画一个处理框。
- 按照以上规则，画出的程序结构图如下图所示。
- 在下图的程序结构的**第4层增加了一个“处理零件组”的框**，因为改进的Jackson图规定顺序执行的处理中不允许混有重复执行和选择执行的处理。增加了这样一个框，使符合该规定，同时也提高了结构图的易读性。

面向数据结构的设计方法





面向数据结构的设计方法

- 列出基本操作与条件，并把它们分配到程序结构图的适当位置；
- 为了对程序结构作补充，要列出求解问题的所有操作和条件，然后分配到程序结构图的适当位置，就可得到完整的程序结构图。

1）本问题的基本操作列出如下：

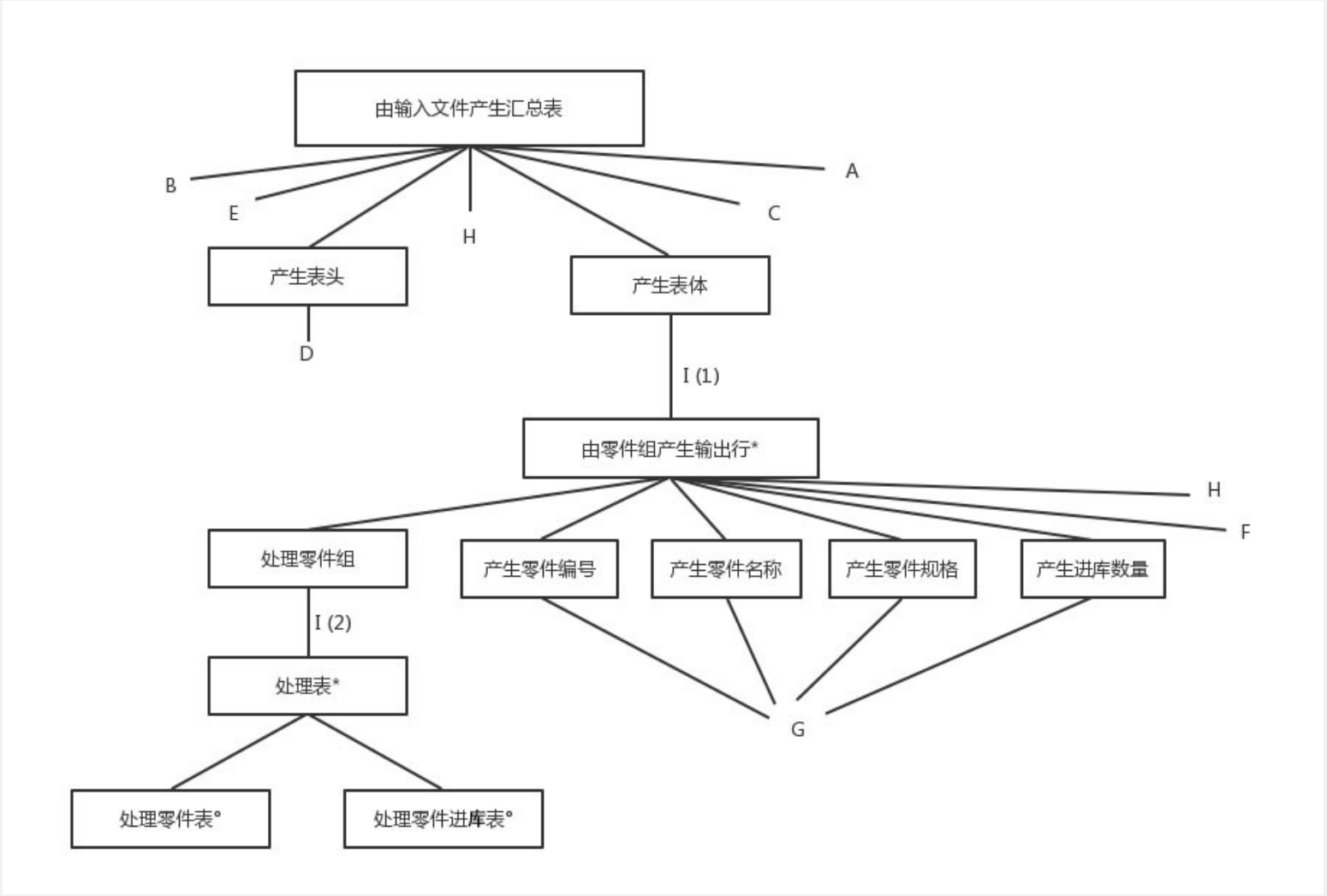
A、终止　B、打开文件　C、关闭文件　D、打印字符行　E、 读一张表　F、产生行结束符　G、列出信息　H、置零件组开始标志

2）列出条件如下：

I（1）：输入条件未结束　　I（2）：零件组未结束

面向数据结构的设计方法

- 将操作与条件分配到适当位置的程序结构图如下图所示：



面向数据结构的设计方法

- 在分配操作时注意：为了能获得重复和选择的条件，Jackson建议至少超前读一个记录，以便使得程序不论在什么时候判定，总有数据已经读入，并做好使用准备。因此在上图中，将操作E（读一张表）放在打开文件之后，同时在处理完一张表后再读一次。

面向数据结构的设计方法

❑用伪代码表示程序

- ❑打开文件
- ❑读一张表
- ❑产生表头 seq
- ❑打印字符行
- ❑产生表头end
- ❑置零件组开始标志
- ❑产生表体 item while输入文件未结束
- ❑由零件组产生输出行seq
- ❑处理零件组 item while零件组未结束
- ❑处理表sel
- ❑处理零件表
- ❑处理零件进库表
- ❑处理表end
- ❑读一张表
- ❑处理零件组end

- 产生零件编号seq
- 列出信息
- 产生零件编号end
- 产生零件名称seq
- 列出信息
- 产生零件名称end
- 产生零件规格seq
- 列出信息
- 产生零件规格end
- 产生进库数量seq
- 列出信息
- 产生进库数量end
- 换行
- 置零件组开始标志
- 由零件组产生输出行end
- 产生表体end
- 关闭文件
- 终止

北京航空航天大學
BEIHANG UNIVERSITY



Part 4

设计文档与评审

3.4 撰写设计文档

1、引言

1.1 编写目的

1.2 读者对象

1.3 软件系统概述

1.4 文档概述

1.5 定义

1.6 参考资料

2、软件设计约束和原则

2.1 软件设计约束

2.2 软件设计原则

3. 软件设计方案

3.1 体系结构设计

3.2 用户界面设计

3.3 用例设计

3.4 子系统/构件设计

3.5 类设计

3.6 数据设计

3.7 部署设计

4. 实施指南

优化和评审用例设计方案

- **尽可能重用已有软件资产来实现用例并依此构建用例实现方案**
  - ✓如开源软件、云服务、遗留系统、软件开发包等等
- **合并设计元素**
  - ✓将具有相同或相似职责的多个设计元素进行整合，归并为一个设计元素。
  - ✓将设计元素（如类等）中具有相同或相似功能的多个方法整合为一个方法，以减少不必要的冗余设计。
- **重组设计元素**
  - ✓借助继承或代理机制，对设计元素进行必要的组织和重组，以发现不同类之间的一般和特殊关系，抽象出公共的方法。
  - ✓确保所有用例实现方案在软件系统中和谐共存，无逻辑冲突

设计评审人员

- **用户（客户）**
  - ✓ 评估和分析软件设计是否正确地实现了他们所提出的软件需求
- **软件设计人员**
  - ✓ 根据评审的意见来修改设计方案
- **程序员**
  - ✓ 能否正确理解设计文档、是否提供足够详细的设计方案以指导编码
- **软件需求分析人员**
  - ✓ 是否实现了他们所定义的软件需求
- **质量保证人员**
  - ✓ 发现软件设计模型和文档中的质量问题，并进行质量保证
- **测试工程师**
  - ✓ 以软件设计文档为依据，设计软件测试用例，开展软件测试
- **配置管理工程师**
  - ✓ 对软件设计规格说明书和设计模型进行配置管理

评审设计文档(1)

- **规范性**
  - ✓是否遵循文档规范，是否按规范的要求和方式来撰写文档
- **简练性**
  - ✓语言表述是否简洁不啰嗦、易于理解。
- **正确性**
  - ✓设计方案是否正确实现了软件功能性需求和非功能性需求
- **可实施性**
  - ✓设计元素是否已充分细化和精化，模型是否易于理解，所选定的程序设计语言是否可以实现该设计模型

评审设计文档(2)

- **可追踪性**
  - ✓各项需求是否在设计文档中都可找到相应的实现方案，设计文档中的设计内容是否对应于需求条目
- **一致性**
  - ✓设计模型间、文档不同段落间、文档的文字表达与设计模型间是否一致。
- **高质量**
  - ✓是否充分考虑了软件设计原则，设计模型是否具有良好的质量属性，如有效性、可靠性、可扩展性、可修改性等

评审步骤

- 阅读和汇报软件设计规格说明书
- 收集和整理问题
- 讨论和达成一致，并进行修改
- 纳入配置

小结

- **详细设计是要给出可指导编码的详细设计方案**
  - ✓依据软件需求、体系结构和用户界面设计模型
- **详细设计的任务**
  - ✓用例设计、子系统设计、构件设计、数据设计、类设计
- **详细设计的描述和输出**
  - ✓UML的顺序图、类图、状态图、活动图、构件图等
  - ✓软件设计规格说明书
- **软件设计的整合、验证与评审**
  - ✓形成系统的软件设计方案
  - ✓发现和修改方案中存在的问题

问题和讨论

